CAMPUS TERRA

Integración y evaluación de modelos de visión, lenguaje y acción en tareas de pick-and-place con un UR5e

Trabajo Fin de Grado
Para la obtención del título de
Grado en Robótica

Autor:
Jeanfranco Abchi Soto

Tutores:
Dr. Juan Antonio Corrales Ramón
Dpto. de Electrónica y Computación

Dr. Jawad Masood
AIMEN Centro Tecnológico

Convocatoria:
Septiembre 2026/2027

USC
UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

ESCOLA POLITÉCNICA SUPERIOR DE ENXEÑARÍA

UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

TRABAJO FIN DE GRADO

INTEGRACIÓN Y EVALUACIÓN DE MODELOS DE VISIÓN, LENGUAJE Y ACCIÓN EN TAREAS DE PICK-AND-PLACE CON UN UR5E

AUTOR:
JEANFRANCO ABCHI SOTO

TUTORES:
Dr. JUAN ANTONIO CORRALES RAMÓN
Dpto. de Electrónica y Computación

Dr. JAWAD MASOOD
AIMEN Centro Tecnológico

Este trabajo se presenta para cumplir los requisitos normativos exigidos para obtener el Grado en Robótica

en la

Escola Politécnica Superior de Enxeñaría.

Fecha de la convocatoria: Septiembre 2026/2027

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA
ESCOLA POLITÉCNICA SUPERIOR DE ENXEÑARÍA

Autorización

Yo, JEANFRANCO ABCHI SOTO, estudiante del Grado en Robótica de la Escola Politécnica Superior de Enxeñaría de la Universidade de Santiago de Compostela, con DNI 30857115Q, autorizo la ubicación de una copia de mi Trabajo Fin de Grado en la biblioteca para que pueda ser consultada por las personas interesadas.

Firmado:

Fecha:

Resumo

INTEGRACIÓN E AVALIACIÓN DE MODELOS DE VISIÓN, LINGUAXE E ACCIÓN EN TAREFAS DE PICK-AND-PLACE CUN UR5E

por
JEANFRANCO ABCHI SOTO

Palabras chave: *Modelos VLA, manipulación robótica, grounding visual-lingüístico, percepción 3D, cross-embodiment*

Esta investigación trata a integración e avaliación de modelos de visión, linguaxe e acción (VLA) preadestrados nunha célula real de manipulación baseada nun robot UR5e. Avaliáronse OpenVLA, π_0-DROID e $\pi_{0.5}$-DROID sen *fine-tuning* específico, abordando a súa transferencia *cross-embodiment* mediante unha arquitectura híbrida con percepción 3D, *grounding* visual-lingüístico, adaptación residual das saídas de acción e control determinista.

A experimentación abarcou dúas tarefas de *pick-and-place* con esixencias semánticas diferentes: a primeira avalía a selección de obxecto e zona de destino, incluíndo relacións espaciais, mentres que a segunda avalía a selección do destino segundo a forma do obxecto, asociando os obxectos cos contedores correspondentes. Realizáronse 66 episodios físicos e *replays* adicionais para analizar a robustez lingüística e a ablación de SigLIP.

OpenVLA con apoio externo de SigLIP foi a configuración máis robusta: obtivo 10/10 e 11/12 seleccións correctas nas Tarefas 1 e 2, fronte a 3/10 e 3/12 sen SigLIP. Na Tarefa 1 alcanzou 9/10 de éxito operacional, fronte a 7/10 para π_0-DROID e 5/10 para $\pi_{0.5}$-DROID. Os principais fallos concentráronse no razoamento espacial na primeira tarefa e no *grounding* na segunda.

Resumen

INTEGRACIÓN Y EVALUACIÓN DE MODELOS DE VISIÓN, LENGUAJE Y ACCIÓN EN TAREAS DE PICK-AND-PLACE CON UN UR5E

por
JEANFRANCO ABCHI SOTO

Palabras clave: *Modelos VLA, manipulación robótica, grounding visual-lingüístico, percepción 3D, cross-embodiment*

Esta investigación trata la integración y evaluación de modelos de visión, lenguaje y acción (VLA) preentrenados en una célula real de manipulación basada en un robot UR5e. Se evaluaron OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID sin *fine-tuning* específico, abordando su transferencia *cross-embodiment* mediante una arquitectura híbrida con percepción 3D, *grounding* visual-lingüístico, adaptación residual de las salidas de acción y control determinista.

La experimentación abarcó dos tareas de *pick-and-place* con exigencias semánticas diferentes: la primera evalúa la selección de objeto y zona de destino, incluyendo relaciones espaciales, mientras que la segunda evalúa la selección del destino según la forma del objeto, asociándolos con los contenedores correspondientes. Se realizaron 66 episodios físicos y *replays* adicionales para analizar la robustez lingüística y la ablación de SigLIP.

OpenVLA con apoyo externo de SigLIP fue la configuración más robusta: obtuvo 10/10 y 11/12 selecciones correctas en las Tareas 1 y 2, frente a 3/10 y 3/12 sin SigLIP. En la Tarea 1 alcanzó 9/10 de éxito operacional, frente a 7/10 para π_0-DROID y 5/10 para $\pi_{0.5}$-DROID. Los principales fallos se concentraron en el razonamiento espacial en la primera tarea y en el *grounding* en la segunda.

Abstract

INTEGRATION AND EVALUATION OF VISION-LANGUAGE-ACTION MODELS IN PICK-AND-PLACE TASKS WITH A UR5E

by
JEANFRANCO ABCHI SOTO

Keywords: *Vision-Language-Action models, robotic manipulation, visual-language grounding, 3D perception, cross-embodiment*

This research investigates the integration and evaluation of pretrained Vision-Language-Action (VLA) models in a real robotic manipulation cell based on a UR5e robot. OpenVLA, π_0-DROID, and $\pi_{0.5}$-DROID were evaluated without platform-specific *fine-tuning*, addressing their *cross-embodiment* transfer through a hybrid architecture with 3D perception, visual-language *grounding*, residual adaptation of action outputs, and deterministic control.

The experiments included two *pick-and-place* tasks with different semantic requirements: the first evaluates object and target-region selection, including spatial relations, whereas the second evaluates target selection according to object shape by associating them with the corresponding containers. A total of 66 physical episodes were performed, together with additional *replays* to analyze linguistic robustness and the ablation of SigLIP.

OpenVLA with external SigLIP support was the most robust configuration: it achieved 10/10 and 11/12 correct selections in Tasks 1 and 2, compared with 3/10 and 3/12 without SigLIP. In Task 1, it achieved 9/10 operational success, compared with 7/10 for π_0-DROID and 5/10 for $\pi_{0.5}$-DROID. The main failures were concentrated in spatial reasoning in the first task and *grounding* in the second one.

Agradecimientos

Quiero agradecer al equipo de AIMEN la oportunidad de realizar este Trabajo de Fin de Grado en sus instalaciones y de poder desarrollar allí toda la parte experimental del proyecto. Ha sido una experiencia que me ha permitido trabajar con medios y problemas reales de robótica, y aprender mucho más allá de lo estrictamente académico.

También quiero agradecer especialmente a mi abuelo todo lo que ha hecho por mí a lo largo de mi formación. Fue doctor en Educación y siempre estuvo dispuesto a ayudarme con cualquier cuestión académica, desde mucho antes de este trabajo. Su forma de entender el estudio, el esfuerzo y la importancia de hacer bien las cosas ha tenido una influencia muy grande en mí, y una parte de haber llegado hasta aquí se la debo a él.

Índice general

Autorización III

Resumo V

Resumen VII

Abstract IX

Agradecimientos XI

Índice de figuras XVII

Índice de tablas XIX

Lista de abreviaturas y acrónimos XXI

Lista de símbolos XXIII

Glosario de términos XXV

1. Introducción 1
- 1.1. Contexto y motivación . . . 1
- 1.2. Planteamiento del problema . . . 2
- 1.3. Objetivos . . . 3
  - 1.3.1. Objetivo general . . . 3
  - 1.3.2. Objetivos específicos . . . 3
- 1.4. Alcance y delimitación del trabajo . . . 3
- 1.5. Metodología general . . . 4

2. Estado del arte 7
- 2.1. Introducción . . . 7
- 2.2. Evolución hacia las políticas robóticas generalistas . . . 7
  - 2.2.1. Políticas condicionadas por visión y lenguaje . . . 7
  - 2.2.2. Aprendizaje a partir de múltiples robots . . . 8
- 2.3. Conjuntos de datos robóticos . . . 8
  - 2.3.1. DROID . . . 8
- 2.4. Modelos de visión, lenguaje y acción . . . 9
  - 2.4.1. OpenVLA . . . 9
  - 2.4.2. π_0 . . . 10
  - 2.4.3. $\pi_{0.5}$ . . . 11
- 2.5. Evaluación y despliegue de modelos VLA . . . 11
  - 2.5.1. Transferencia entre plataformas robóticas . . . 11

2.5.2. Robustez lingüística y asociación visual . . . 12
2.5.3. Evaluación física y seguridad . . . 13
2.6. Posicionamiento del trabajo . . . 14

3. Marco teórico **15**
3.1. Introducción . . . 15
3.2. Aprendizaje profundo y modelos preentrenados . . . 15
3.2.1. Redes neuronales y aprendizaje a partir de datos . . . 15
3.2.2. Preentrenamiento y transferencia . . . 16
3.3. Modelos multimodales de visión y lenguaje . . . 17
3.3.1. Arquitectura *transformer* y mecanismo de atención . . . 17
3.3.2. Codificadores visuales y alineamiento imagen–texto . . . 18
3.4. Políticas de visión, lenguaje y acción . . . 19
3.4.1. De los modelos visión–lenguaje a las políticas VLA . . . 19
3.4.2. Representación y generación de acciones . . . 20
3.4.3. Predicción autorregresiva y discretización . . . 21
3.4.4. *Flow matching* y bloques temporales de acciones . . . 21
3.4.5. *Grounding* visual-lingüístico en la selección de entidades . . . 23
3.5. Geometría, cinemática y percepción tridimensional . . . 24
3.5.1. Espacio articular y espacio cartesiano . . . 24
3.5.2. Cinemática directa e inversa . . . 24
3.5.3. Jacobiano y cinemática inversa diferencial . . . 25
3.5.4. Efector final y punto central de la herramienta . . . 26
3.5.5. Sistemas de coordenadas y transformaciones homogéneas . . . 26
3.5.6. Percepción 3D y calibración extrínseca *eye-to-hand* . . . 27
3.6. Transferencia de políticas VLA a sistemas robóticos . . . 27
3.6.1. Espacios de observación y acción . . . 28
3.6.2. Transferencia entre *embodiments* . . . 28
3.6.3. Arquitecturas *end-to-end*, modulares e híbridas . . . 29
3.6.4. Adaptación y acotación de acciones . . . 29

4. Metodología **31**
4.1. Introducción . . . 31
4.2. Configuraciones y checkpoints evaluados . . . 31
4.3. Plataforma y arquitectura del sistema . . . 32
4.3.1. Entorno software y dependencias . . . 33
4.4. Calibración y percepción tridimensional . . . 34
4.4.1. Calibraciones geométricas . . . 34
4.4.2. Obtención de candidatos y posiciones 3D . . . 35
4.5. *Grounding* e integración de los modelos . . . 36
4.6. Adaptación de las acciones al UR5e . . . 37
4.7. Trayectoria y seguridad . . . 38
4.8. Tareas implementadas . . . 39
4.9. Métricas y análisis . . . 40

5. Resultados y discusión **43**
5.1. Introducción . . . 43
5.2. Tarea 1: colocación de objetos sobre zonas . . . 43
5.2.1. Resultados físicos . . . 43
5.2.2. Ablación de OpenVLA sin SigLIP . . . 44

5.2.3. Robustez lingüística . . . 45
5.2.4. Comportamiento de las acciones . . . 45
5.3. Tarea 2: clasificación por forma . . . 47
5.3.1. Resultados físicos . . . 47
5.3.2. Ablación de SigLIP . . . 48
5.3.3. Robustez lingüística . . . 48
5.3.4. Comportamiento de las acciones . . . 49
5.4. Discusión . . . 50

6. Conclusiones y líneas futuras **53**
6.1. Conclusiones . . . 53
6.2. Líneas futuras . . . 54

A. Resultados experimentales completos **55**
A.1. Tarea 1: colocación de objetos sobre zonas . . . 55
A.2. Tarea 2: clasificación por forma . . . 57
A.3. Contribución traslacional aplicada por la interfaz VLA . . . 59

B. Ensayos complementarios de robustez y ablación **61**
B.1. Robustez lingüística . . . 61
B.1.1. Tarea 1: colocación de objetos sobre zonas . . . 61
B.1.2. Tarea 2: clasificación por forma . . . 62
B.2. Ablación del adaptador externo SigLIP . . . 63
B.2.1. Ablación en la Tarea 1 . . . 63
B.2.2. Ablación en la Tarea 2 . . . 64
B.3. Trazabilidad de los replays . . . 65

C. Implementación software y reproducibilidad **67**
C.1. Organización del código . . . 67
C.2. Secuencia general de un episodio . . . 68
C.3. Grounding y neutralización de candidatos . . . 68
C.3.1. Selección y comprobaciones del grounding externo . . . 69
C.4. Adaptación de las acciones VLA . . . 70
C.4.1. Adaptación de OpenVLA . . . 70
C.4.2. Adaptación de los bloques de acciones DROID . . . 71
C.4.3. Acotación de los residuales . . . 72
C.5. Cinemática inversa mediante DLS . . . 72
C.6. Supervisión previa a la ejecución física . . . 73
C.7. Ejecución física y control del efector . . . 74
C.8. Soporte para replays . . . 75

Bibliografía **77**

Índice de figuras

1.1. Flujo general de la metodología 5

3.1. Arquitectura del *Transformer* 18
3.2. Arquitectura de OpenVLA 20
3.3. Arquitectura general de π_0 22
3.4. Visión general de $\pi_{0.5}$ 22

4.1. Vista general de la célula experimental 33
4.2. Montaje de calibración extrínseca 35
4.3. Ejemplos de percepción visual 36
4.4. Escenas de las dos tareas 40

5.1. Resultados físicos por configuración en la tarea objeto–zona. 43
5.2. Grounding correcto y relación física en la tarea objeto–zona. 44
5.3. Selección correcta de la pareja objeto–destino con OpenVLA + SigLIP y en la consolidación de OpenVLA sin SigLIP en la Tarea 1. 45
5.4. Robustez lingüística por configuración en los 150 *replays* de la tarea objeto–zona 45
5.5. Tasa de acierto agregada por tipo de variación lingüística en la tarea objeto–zona. 46
5.6. Éxito operacional y cumplimiento estricto condicionados a un *grounding* correcto en la tarea objeto–zona. 46
5.7. Métricas temporales internas de los bloques de acción en la tarea objeto–zona. 46
5.8. Resultados físicos por configuración en la tarea de clasificación por forma. 47
5.9. Resultados físicos por disposición de la escena en la tarea de clasificación por forma. 47
5.10. Comparación del *grounding* de OpenVLA con y sin el apoyo externo de SigLIP en la tarea de clasificación por forma. 48
5.11. Robustez lingüística por configuración en la tarea de clasificación por forma. 49
5.12. Tasa de acierto agregada por tipo de variación lingüística en la tarea de clasificación por forma. 49
5.13. Resultado de la acción condicionado a un *grounding* correcto en la tarea de clasificación por forma. 49
5.14. Métricas temporales internas de los bloques de acción en la tarea de clasificación por forma. 50

Índice de tablas

4.1. Formas de salida de las configuraciones evaluadas. 36

A.1. Casos físicos definidos para la Tarea 1. 56
A.2. Resultados individuales de los 30 episodios físicos de la Tarea 1. 56
A.3. Tipos de caso físicos definidos para la Tarea 2. 57
A.4. Resultados individuales de los 36 episodios físicos de la Tarea 2. 58
A.5. Magnitud media de la contribución traslacional aplicada por la interfaz VLA en los episodios que superaron correctamente el *grounding*. . . 59

B.1. Resultados de robustez lingüística por configuración en la Tarea 1. . . 62
B.2. Desglose por configuración y tipo de variación lingüística en la Tarea 1. 62
B.3. Resultados de robustez lingüística por configuración en la Tarea 2. . . 62
B.4. Desglose por configuración y tipo de variación lingüística en la Tarea 2. 63
B.5. Ablación de SigLIP en los diez casos de la Tarea 1. 64
B.6. Ablación de SigLIP en los doce casos de OpenVLA de la Tarea 2. 64

C.1. Archivos de código incluidos en el material complementario. 67

Lista de abreviaturas y acrónimos

A-AI	*Action Acceleration Instability*, inestabilidad de aceleración de la acción
A-PI	*Action Position Instability*, inestabilidad de posición de la acción
A-VI	*Action Velocity Instability*, inestabilidad de velocidad de la acción
DROID	*Distributed Robot Interaction Dataset*
GPU	*Graphics Processing Unit*
RGB	*Red, Green, Blue*
RLDS	*Reinforcement Learning Datasets*
ROS 2	*Robot Operating System 2*
TCP	*Tool Center Point*
TFG	Trabajo Fin de Grado
VLA	*Vision-Language-Action*
VLM	*Vision-Language Model*

Lista de símbolos

$\mathbf{A}$	Matriz que contiene un bloque temporal de acciones
$\mathbf{a}_t$	Acción correspondiente al instante interno t del bloque
H	Horizonte o número de acciones del bloque
D	Número de componentes empleadas al calcular una métrica
${}^{B}\mathbf{T}_{C}$	Transformación homogénea del sistema de la cámara al de la base
${}^{C}\mathbf{p}$	Punto expresado en el sistema de coordenadas de la cámara
${}^{B}\mathbf{p}$	Punto expresado en el sistema de coordenadas de la base del UR5e
$\Delta\mathbf{p}_k^{\mathrm{VLA}}$	Residual traslacional construido por la interfaz a partir de la salida del modelo en la fase k
$\mathbf{p}_k^{\mathrm{geom}}$	Ancla geométrica calculada mediante percepción tridimensional
$\mathbf{p}_k^{\mathrm{exec}}$	Posición cartesiana finalmente empleada en la ejecución
$\mathbf{b}_k$	Límite cartesiano aplicado al residual en la fase k

Glosario de términos

Término	Definición utilizada en esta memoria
Ablación	Comparación experimental en la que se elimina o modifica un componente concreto del sistema para estudiar su contribución.
Action chunk	Bloque temporal de varias acciones generado conjuntamente por una política robótica.
Action expert	Módulo neuronal especializado de la familia π que recibe representaciones multimodales y genera acciones robóticas continuas mediante *flow matching*.
Backpropagation	Procedimiento utilizado durante el entrenamiento de redes neuronales para calcular los gradientes de la función de pérdida respecto a los parámetros del modelo.
Checkpoint	Conjunto de parámetros de un modelo guardado después de su entrenamiento y empleado posteriormente para inferencia o adaptación.
Cross-embodiment	Transferencia de una política entre plataformas con diferencias de robot, sensores, efector final o representación de acciones.
Desnormalización	Operación que transforma una salida normalizada del modelo de vuelta al espacio numérico definido por las estadísticas y convenciones asociadas a su checkpoint.
Embedding	Representación vectorial de un elemento visual, lingüístico o de otro tipo utilizada por el modelo para procesar y relacionar información.
Embodiment	Realización física de un sistema robótico, determinada por aspectos como su estructura cinemática, sensores, efector final e interfaz de acciones.
Encoder-decoder	Arquitectura formada por un codificador que genera una representación de la entrada y un decodificador que utiliza dicha representación para producir la salida.
End-to-end	Enfoque en el que un modelo aprende directamente la relación entre las observaciones disponibles y la acción, sin separar explícitamente todas las etapas intermedias.
Espacio de acciones	Conjunto de variables y convenciones mediante las que una política representa sus acciones, incluyendo su significado, unidades, normalización y forma de ejecución.
Eye-to-hand	Configuración en la que la cámara permanece fija respecto al entorno y observa al robot y la superficie de trabajo desde una posición externa.

Término	Definición utilizada en esta memoria
Fine-tuning	Ajuste adicional de un modelo preentrenado mediante datos específicos de una tarea o plataforma.
Flow matching	Método generativo que aprende un campo vectorial continuo para transformar una distribución inicial en una distribución objetivo y que, en la familia π, se utiliza para generar acciones continuas.
Grounding	Asociación entre una expresión lingüística, una entidad visual concreta y, en este trabajo, su posición tridimensional.
Inferencia	Ejecución de un modelo ya entrenado para obtener una predicción a partir de una entrada.
Layout	Disposición física concreta de los objetos y destinos dentro de la escena experimental.
Multi-head attention	Variante del mecanismo de atención que utiliza varias proyecciones aprendidas en paralelo para representar diferentes relaciones entre los elementos de una secuencia.
Parser	Componente determinista que normaliza expresiones de la instrucción y comprueba su compatibilidad semántica con las entidades seleccionadas.
Pick-and-place	Tarea robótica de manipulación en la que un objeto se recoge desde una posición inicial y se deposita en un destino determinado.
Política robótica	Función o modelo que relaciona observaciones y objetivos con acciones destinadas a controlar un robot.
Predicción autorregresiva	Estrategia de generación en la que cada nueva componente se predice condicionada por las generadas anteriormente y por la información de entrada disponible.
Replay	Reproducción de una inferencia sobre observaciones guardadas, sin repetir necesariamente el movimiento físico del robot.
Residual cartesiano	Corrección traslacional acotada que se añade a una referencia geométrica obtenida por percepción.
Site	Elemento de referencia definido en un modelo MuJoCo para representar un punto de interés del sistema y calcular magnitudes cinemáticas asociadas, como la pose o el jacobiano.
Softmax	Función que transforma un conjunto de valores numéricos en pesos positivos normalizados cuya suma es uno, utilizada, entre otros casos, para obtener los pesos de atención en un *transformer*.
Token	Unidad discreta utilizada por un modelo generativo para representar y producir información secuencial. En OpenVLA, las componentes discretizadas de acción se convierten en tokens antes de ser generadas por el modelo.
Transformer	Arquitectura neuronal basada en mecanismos de atención que permite modelar relaciones entre los elementos de una secuencia y que constituye la base de numerosos modelos multimodales y VLA.

Término	Definición utilizada en esta memoria
Zero-shot	Aplicación de un modelo a una tarea o plataforma sin entrenamiento específico con ejemplos propios de esa condición.

1 Introducción

1.1. Contexto y motivación

En los últimos años, el desarrollo de grandes conjuntos de demostraciones robóticas y de modelos capaces de procesar conjuntamente visión y lenguaje natural ha impulsado nuevas formas de interacción con los robots. Dentro de esta línea se encuentran los modelos de visión, lenguaje y acción, denominados VLA por sus siglas en inglés (*Vision-Language-Action*). Los mismos buscan relacionar la observación visual, la instrucción recibida y la acción que debe realizarse, permitiendo plantear sistemas en los que el usuario describe el objetivo de la tarea sin programar de forma explícita cada movimiento.

Este Trabajo de Fin de Grado (TFG) se desarrolla en AIMEN Centro Tecnológico y estudia la integración de estos modelos en una célula de manipulación real compuesta por un robot UR5e, una pinza OnRobot 3FG15 y un sensor tridimensional Photoneo MotionCam-3D-S. La evaluación comprende dos tareas de *pick-and-place*, que son el traslado de objetos hacia zonas de destino, incluyendo relaciones espaciales, y la clasificación de objetos mediante su colocación en recipientes asociados a su forma geométrica.

AIMEN Centro Tecnológico es una entidad especializada en investigación, desarrollo e innovación aplicada a materiales y tecnologías de fabricación avanzada. Dentro de sus líneas de I+D+i se encuentra específicamente la automatización y la robótica, con actividades orientadas, entre otros ámbitos, a la robótica móvil y colaborativa para aplicaciones industriales, los sistemas robotizados en entornos complejos y las arquitecturas de control aplicadas a procesos de fabricación e intralogística [1]. Este marco tecnológico resulta directamente relacionado con el presente trabajo, que estudia la integración de modelos de visión, lenguaje y acción en una célula robótica real orientada a tareas de manipulación.

La motivación del trabajo no se limita a conseguir que el robot desplace objetos, sino que pretende analizar qué información de los modelos puede transferirse de forma útil a una plataforma diferente de aquellas utilizadas durante su entrenamiento, cómo debe combinarse con percepción tridimensional y control robótico, y en qué etapas se producen los fallos.

Este TFG no plantea sustituir completamente los módulos clásicos de robótica, sino que adopta una arquitectura en la que los modelos participan en la interpretación de la instrucción, la asociación con los elementos de la escena y la generación de información de acción, mientras que aspectos como la cinemática inversa y las comprobaciones de seguridad se gestionan mediante componentes específicos y verificables.

Esta separación permite aprovechar las capacidades multimodales de los modelos sin aplicar directamente sus salidas al robot. En una plataforma física, las acciones

generadas deben adaptarse al sistema de referencia, al espacio de trabajo, a los límites articulares, al efector final y a las condiciones de seguridad.

1.2. Planteamiento del problema

El problema abordado surge de la necesidad de evaluar si los modelos VLA preentrenados pueden integrarse de forma trazable, segura y eficiente en una célula robótica destinada, a largo plazo, a tareas industriales de mayor complejidad, como el desensamblaje de baterías. Antes de aplicar estas políticas a operaciones de separación y manipulación de componentes, resulta necesario estudiar su comportamiento en un entorno controlado que permita identificar con precisión el origen de los posibles fallos. Por este motivo, el trabajo utiliza tareas de *pick-and-place* como banco de pruebas inicial, en el que a partir de una instrucción en lenguaje natural y una observación visual de la escena, el sistema debe identificar el objeto y el destino solicitados, obtener referencias métricas compatibles con el UR5e y transformar la información generada por cada modelo en una ejecución física segura y verificable. Esta integración no es directa, ya que los modelos fueron entrenados con plataformas, interfaces y representaciones de acción diferentes de las utilizadas en este trabajo.

La dificultad principal reside en la transferencia entre *embodiments*. OpenVLA, π_0 y $\pi_{0.5}$ presentan interfaces y estructuras de salida diferentes, y sus acciones están condicionadas por plataformas robóticas que no coinciden con el flujo de trabajo implementado. OpenVLA genera una acción por inferencia, mientras que los modelos de la familia π producen bloques temporales de acciones [2]-[4]. En las configuraciones DROID concretamente desplegadas en este trabajo, `pi0_droid` devuelve un bloque de 10 acciones y `pi05_droid` uno de 15 acciones, de acuerdo con las configuraciones de inferencia definidas en OpenPI [5]. Estas longitudes corresponden a las interfaces utilizadas en el proyecto y no deben interpretarse como propiedades universales de las arquitecturas. Las salidas obtenidas no pueden compararse ni aplicarse directamente sin estudiar su significado, escala y compatibilidad con el robot de destino.

A esta diferencia se añade la necesidad de combinar información semántica y geometría métrica. La imagen y la instrucción permiten decidir qué elemento debe manipularse, pero la ejecución requiere conocer posiciones tridimensionales en el sistema de coordenadas de la base del UR5e. Por ello, la integración incluye una percepción 3D calibrada y una estrategia que conecta la información obtenida de los modelos con referencias cartesianas y configuraciones articulares válidas.

Dado lo anterior, surgen las siguientes interrogantes: ¿cómo se comportan las interfaces de entrada y salida de los modelos VLA seleccionados (OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID) al desplegarse fuera de sus configuraciones de origen? ¿Qué calibración y arquitectura de percepción tridimensional permite expresar de forma fiable la detección de la escena en el sistema de referencia del UR5e? ¿Cuál sería la estrategia de transferencia para adaptar las acciones generadas por estos modelos a la cinemática, el espacio de trabajo y el efector final del UR5e? ¿Cuál sería el protocolo experimental que permita evaluar, de forma diferenciada, la interpretación lingüística, la selección visual, la generación de acciones y la ejecución física en las tareas de *pick-and-place* estudiadas? ¿Cuáles serían las diferencias de comportamiento y modos de

fallo que presentan los tres modelos al operar bajo un mismo protocolo experimental?

1.3. Objetivos

1.3.1. Objetivo general

El objetivo general de este TFG es integrar y evaluar comparativamente los modelos VLA OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID en tareas de *pick-and-place* con un robot UR5e.

1.3.2. Objetivos específicos

Para alcanzar el objetivo general se establecen los siguientes objetivos específicos:

1. Implantar OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID, caracterizando sus entradas, salidas y requisitos de inferencia.
2. Integrar el sistema de percepción tridimensional Photoneo con el UR5e, de forma que las posiciones detectadas puedan expresarse en el sistema de coordenadas del robot.
3. Validar una estrategia de transferencia entre los modelos y la plataforma experimental, teniendo en cuenta las diferencias de robot, configuración sensorial, efector final y espacio de acciones.
4. Ejecutar un protocolo experimental sobre dos tareas de manipulación que permita analizar de forma diferenciada la interpretación lingüística, la percepción visual, la generación de acciones, la ejecución física y la verificación del resultado.
5. Comparar el comportamiento de los tres modelos, identificando sus capacidades, diferencias y modos de fallo en las condiciones estudiadas.

1.4. Alcance y delimitación del trabajo

El estudio se centra en tres configuraciones preentrenadas: OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID. No se realiza entrenamiento desde cero ni *fine-tuning* con demostraciones propias del UR5e, sino que se evalúa un despliegue *zero-shot* respecto a esta plataforma, entendido como la utilización de los checkpoints sin entrenamiento específico con datos propios de la célula. La integración requiere, no obstante, una interfaz externa para su transferencia *cross-embodiment*. El sistema de percepción tridimensional utilizado fue el que mejores cualidades técnicas estaba a disposición para este TFG en la empresa.

La plataforma experimental queda delimitada a un UR5e de seis grados de libertad, una pinza OnRobot 3FG15, una Photoneo fija en configuración *eye-to-hand*, una Jetson AGX Orin y un ordenador central. Los experimentos se realizan sobre una cinta transportadora detenida, utilizada como superficie de trabajo, con una calibración cámara–robot fija y agarres de orientación cenital. Se estudian las dos tareas controladas previamente mencionadas y un conjunto definido de objetos e instrucciones. No se incluyen obstáculos dinámicos, objetos deformables ni tareas de ensamblaje.

Como se ha señalado, la incompatibilidad entre los espacios de acción nativos de los modelos y el del UR5e impide aplicar directamente sus salidas al robot. En particular, la implementación oficial de OpenPI normaliza los estados y las acciones mediante estadísticas calculadas sobre los datos de entrenamiento y almacenadas junto al *checkpoint*; su reutilización requiere respetar las definiciones del espacio de acciones empleadas durante el preentrenamiento [6]. Por este motivo, las predicciones de las configuraciones DROID no se interpretaron como comandos directamente ejecutables por el UR5e. La arquitectura final utilizó únicamente una contribución traslacional construida por la interfaz a partir de dichas salidas, escalada y limitada como corrección cartesiana. La orientación del efector se mantiene cenital, mientras que la apertura y el cierre de la pinza, la generación de puntos intermedios, la cinemática inversa y las comprobaciones de seguridad se resuelven mediante módulos deterministas externos a los modelos.

Además, no se realizó una campaña física equivalente con $\Delta\mathbf{p}^{\mathrm{VLA}} = 0$; por tanto, los experimentos permiten comparar las configuraciones integradas evaluadas, pero no aislar cuantitativamente la aportación del residual VLA frente a una ejecución basada únicamente en la referencia geométrica.

En consecuencia, los resultados corresponden al comportamiento de los sistemas integrados y no a un control directo de todos los grados de libertad por parte de los VLA. Las conclusiones se restringen a los *checkpoints*, el hardware y el protocolo experimental utilizados en este trabajo.

1.5. Metodología general

El trabajo sigue un desarrollo incremental. En primer lugar se estudian las interfaces de los modelos y se prepara su despliegue remoto en la Jetson AGX Orin. Paralelamente se configura la comunicación con el UR5e y la pinza, y posteriormente, se integra la captura tridimensional. Por último, se procede con la calibración del TCP y la calibración extrínseca Photoneo–UR5e que permiten expresar los puntos de la nube en el sistema de referencia del robot.

El flujo completo de la metodología se resume en la Figura 1.1, que sirve como guía de las principales etapas desarrolladas posteriormente.

A partir de una captura nueva se generan candidatos visuales y posiciones tridimensionales para los objetos y destinos. La instrucción y la observación se procesan con el modelo correspondiente y la información resultante se relaciona con los candidatos de la escena. Durante el desarrollo se incorporó un adaptador externo basado en SigLIP, un modelo preentrenado para establecer compatibilidad entre pares imagen–texto [7]. En este trabajo los candidatos visuales son extraídos de la escena para estudiar y reforzar la correspondencia entre la instrucción y las regiones asociadas a los posibles objetos y destinos. Esta incorporación se analiza mediante condiciones con y sin dicho adaptador, evitando atribuir a OpenVLA aislado los resultados del sistema completo.

La información de acción se adapta posteriormente al UR5e. Las componentes traslacionales se combinan con posiciones métricas obtenidas mediante Photoneo, se construyen puntos de paso y se resuelve la cinemática inversa de forma secuencial. Antes de autorizar el movimiento se comprueban el estado inicial, la continuidad

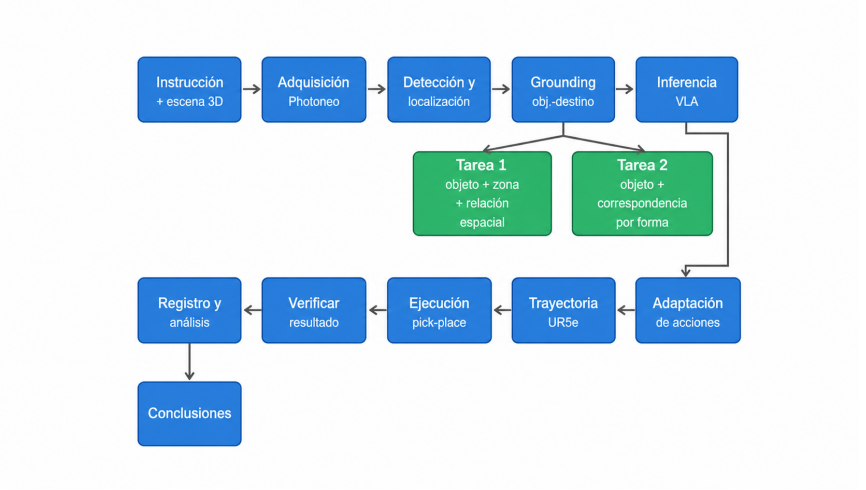


FIGURA 1.1: Flujo general de la metodología empleada en el trabajo.

articular, los límites operativos y las condiciones de seguridad. La apertura y el cierre de la pinza 3FG15 y la verificación final se gestionan de forma independiente de la salida nativa de los modelos.

2 Estado del arte

2.1. Introducción

El aprendizaje de políticas robóticas ha evolucionado desde soluciones diseñadas para una tarea y una plataforma concretas hacia modelos entrenados sobre conjuntos amplios de demostraciones y capaces de responder a diferentes instrucciones, objetos y escenarios. Dentro de esta evolución, los VLA buscan relacionar observaciones visuales e instrucciones lingüísticas con acciones robóticas [8].

Este capítulo revisa las principales contribuciones que condujeron a la aparición de estas políticas generalistas. En primer lugar, se analiza la evolución desde los modelos entrenados para conjuntos restringidos de tareas hasta el aprendizaje a partir de datos procedentes de múltiples robots. A continuación, se estudian Open X-Embodiment y DROID como fuentes de datos relevantes, así como OpenVLA, π_0 y $\pi_{0.5}$, los tres modelos considerados en este trabajo.

Finalmente, se revisan estudios relacionados con la transferencia entre plataformas, la robustez frente a variaciones lingüísticas, la evaluación física y la seguridad.

2.2. Evolución hacia las políticas robóticas generalistas

2.2.1. Políticas condicionadas por visión y lenguaje

Una de las primeras propuestas de gran escala para integrar observaciones visuales e instrucciones lingüísticas en una única política robótica fue RT-1. El modelo empleó una arquitectura basada en *transformers* y se entrenó con aproximadamente 130 000 episodios de manipulación correspondientes a más de 700 tareas ejecutadas sobre una misma familia de robots. Sus autores analizaron la capacidad de reutilizar representaciones entre comportamientos y de generalizar ante nuevos objetos, fondos y condiciones visuales [9].

Los experimentos mostraron que una política entrenada conjuntamente sobre numerosas tareas podía superar a alternativas especializadas en determinadas condiciones de generalización. Sin embargo, el conocimiento adquirido por RT-1 procedía principalmente de demostraciones robóticas y continuaba vinculado a la plataforma y al espacio de acciones utilizados durante la recopilación de los datos.

RT-2 amplió este planteamiento mediante el ajuste conjunto de modelos de visión y lenguaje con datos generales de imagen y texto y con trayectorias robóticas. Los autores denominaron VLA a esta clase de modelos y representaron las acciones de forma compatible con la generación lingüística del modelo [8].

Los resultados presentados para RT-2 mostraron mejoras en tareas que requerían reconocer propiedades, relaciones y conceptos no incluidos explícitamente en las

demostraciones robóticas. Esta aportación evidenció que el conocimiento semántico adquirido por los modelos visuales y lingüísticos podía contribuir a la ejecución de tareas físicas. No obstante, la acción generada seguía dependiendo de las convenciones de representación y de la plataforma empleada durante el entrenamiento, por lo que la mejora semántica no resolvía por sí sola la transferencia a otros robots.

2.2.2. Aprendizaje a partir de múltiples robots

Open X-Embodiment abordó la fragmentación de los datos robóticos mediante la agregación y normalización de conjuntos recopilados por diferentes instituciones. La colección reunió demostraciones procedentes de 22 plataformas robóticas y 527 habilidades, proporcionando un formato compartido para estudiar el entrenamiento a partir de experiencias heterogéneas [10].

Sobre esta colección se desarrollaron los modelos RT-X. Los experimentos publicados mostraron que la combinación de datos de distintas plataformas podía producir transferencia positiva y mejorar el comportamiento sobre robots individuales frente al entrenamiento con datos de una única fuente [10]. Aun así, la normalización de los conjuntos no eliminaba las diferencias cinemáticas, sensoriales y de control existentes entre los robots participantes.

Octo continuó esta línea mediante una política generalista y de código abierto entrenada con aproximadamente 800 000 trayectorias de Open X-Embodiment. El modelo fue diseñado para aceptar distintas combinaciones de observaciones, instrucciones e interfaces de acción. Sus autores evaluaron su adaptación a nueve plataformas y propusieron utilizarlo como punto de partida para realizar *fine-tuning* en nuevos dominios [11].

Los resultados de Octo reforzaron la hipótesis de que la diversidad de datos puede facilitar la adaptación a nuevas tareas y plataformas. Al mismo tiempo, el procedimiento de ajuste empleado mostró que una política generalista no es necesariamente independiente del cuerpo robótico. Las diferencias entre sensores, efectores, frecuencias de control y representaciones de acción continuaron requiriendo adaptaciones específicas.

Estas contribuciones establecieron el contexto para el desarrollo de OpenVLA, π_0 y $\pi_{0.5}$. Los tres modelos aprovechan datos heterogéneos y representaciones multimodales, pero siguen enfoques distintos respecto a la arquitectura, el entrenamiento y la generación de acciones.

2.3. Conjuntos de datos robóticos

2.3.1. DROID

DROID (*Distributed Robot Interaction Dataset*) es un conjunto de datos de manipulación robótica recogido en entornos reales y variados. La publicación original describe aproximadamente 65 000 trayectorias, equivalentes a unas 350 horas de interacción, obtenidas en 564 escenas y 86 tareas mediante 50 operadores distribuidos entre distintas instituciones [12].

A diferencia de Open X-Embodiment, DROID mantiene una configuración de hardware común entre las instituciones participantes. La plataforma de adquisición está

formada por un manipulador Franka Panda de siete grados de libertad, cámaras estereoscópicas externas, una cámara montada en la muñeca y un sistema de teleoperación mediante realidad virtual [12].

Esta estandarización permitió recopilar demostraciones con una interfaz sensorial y de acción relativamente uniforme, manteniendo al mismo tiempo una elevada diversidad de laboratorios, oficinas y entornos domésticos. Las trayectorias incluyen observaciones visuales, estados del robot, acciones e instrucciones lingüísticas asociadas a las tareas. El conjunto se distribuye tanto en formato bruto como mediante una representación compatible con RLDS (*Reinforcement Learning Datasets*) [12], [13]. RLDS proporciona una estructura común para almacenar episodios como secuencias ordenadas de observaciones, acciones y demás información asociada a la interacción. Su utilidad en este contexto es facilitar que conjuntos de datos procedentes de distintas fuentes puedan almacenarse, transformarse y reutilizarse mediante una interfaz consistente [13].

Los experimentos publicados junto con DROID analizaron la generalización frente a cambios de escena, distractores y nuevas instancias de objetos. Los autores observaron que la incorporación de demostraciones diversas mejoraba la robustez de las políticas evaluadas frente a alternativas entrenadas con datos menos variados [12].

DROID resulta relevante para la familia π porque OpenPI distribuye configuraciones y *checkpoints* asociados a este conjunto de datos [5]. Sus convenciones de observación y acción corresponden a la plataforma DROID, por lo que su transferencia a otros robots requiere una interfaz específica.

2.4. Modelos de visión, lenguaje y acción

2.4.1. OpenVLA

OpenVLA fue presentado como un modelo VLA de código abierto con aproximadamente siete mil millones de parámetros, entrenado con 970 000 demostraciones robóticas reales procedentes de una selección de conjuntos incluidos en Open X-Embodiment [2].

El conjunto utilizado para su preentrenamiento no corresponde a un único robot. OpenVLA emplea una mezcla de datos procedentes de Open X-Embodiment, que reúne demostraciones obtenidas con diferentes *embodiments* [2], [10]. La configuración pública de entrenamiento de OpenVLA incluye, entre otros, conjuntos asociados a los robots de RT-1 de Google, manipuladores KUKA, WidowX, Jaco, UR5 y Franka, además de datos procedentes de otras plataformas como FANUC o Stretch [14]. Por tanto, OpenVLA no debe interpretarse como una política aprendida para un manipulador concreto, sino como una política preentrenada sobre una mezcla heterogénea de robots, tareas y entornos.

La publicación de OpenVLA buscó facilitar el acceso a una política generalista que pudiera ser desplegada, estudiada y adaptada por grupos externos. Su arquitectura combina el modelo de lenguaje Llama 2 con dos representaciones visuales complementarias. DINOv2 es un modelo visual preentrenado mediante aprendizaje autosupervisado, orientado a obtener características reutilizables de las imágenes sin depender de etiquetas específicas para cada tarea [15]. SigLIP, por su parte, aprende el alineamiento entre imágenes y textos y proporciona representaciones sensibles a su

compatibilidad semántica [7]. OpenVLA integra ambas fuentes de información visual antes de generar las acciones mediante una formulación autorregresiva basada en componentes discretizadas [2]. Los fundamentos de estas representaciones y de la generación de acciones se desarrollan en el Capítulo 3.

Los autores compararon OpenVLA con otras políticas de código abierto y con modelos generalistas previos. Los resultados mostraron un comportamiento competitivo en las tareas evaluadas y una mejora respecto a determinadas alternativas de tamaño similar. También estudiaron procedimientos de adaptación mediante *fine-tuning*, técnicas de bajo rango y cuantización [2].

Estos experimentos mostraron que OpenVLA podía utilizarse como base para adaptaciones posteriores. Sin embargo, los resultados se obtuvieron bajo interfaces y plataformas concretas, por lo que no garantizan que la salida del modelo pueda ejecutarse sin adaptación sobre un robot diferente.

2.4.2. π_0

El modelo π_0 fue presentado como una política generalista basada en un modelo de visión y lenguaje preentrenado y un módulo específico para la generación de acciones [3]. Su base multimodal procede de PaliGemma, un modelo visión–lenguaje abierto construido a partir de un codificador visual SigLIP y un modelo de lenguaje Gemma, diseñado para transferirse posteriormente a diferentes tareas multimodales [16]. Sobre esta base, π_0 incorpora un módulo aprendido especializado en la generación de acciones, denominado *action expert*, que produce bloques temporales de acciones continuas mediante *flow matching* [3].

El preentrenamiento de π_0 combinó conjuntos robóticos abiertos con más de 10 000 horas de datos recogidos por Physical Intelligence. Estos últimos abarcaron 68 tipos de tarea y siete configuraciones robóticas: un UR5e de un solo brazo, un sistema bimanual formado por dos UR5e, un Franka, un sistema bimanual Trossen basado en ALOHA, una configuración bimanual ARX/AgileX, una plataforma móvil bimanual Trossen/ARX y una plataforma móvil Fibocom [3]. El modelo fue entrenado conjuntamente sobre estas configuraciones, que incluyen manipuladores de uno y dos brazos y robots móviles con espacios de estado y acción diferentes.

La denominación de siete configuraciones no implica necesariamente siete modelos de robot completamente distintos. En algunos casos, los autores agrupan dentro de una misma configuración variantes de hardware con propiedades cinemáticas próximas, como ARX y AgileX o las versiones móviles basadas en Trossen y ARX [3]. Esta diversidad forma parte del planteamiento *cross-embodiment* de π_0.

Los autores evaluaron el modelo en tareas de manipulación que requerían destreza, adaptación a distintas instrucciones y transferencia entre configuraciones. Los resultados respaldaron el uso conjunto de datos heterogéneos, aunque las plataformas continuaban necesitando definiciones específicas de observación, estado y acción.

La implementación abierta utilizada para trabajar con esta familia se distribuye mediante OpenPI [17]. OpenPI no constituye un modelo VLA adicional, sino el conjunto de herramientas publicado por Physical Intelligence para configurar, entrenar y ejecutar las políticas de la familia π. También proporciona una interfaz de inferencia remota que permite mantener la política en un equipo dedicado y enviarle las

observaciones desde el sistema encargado de la ejecución robótica [18]. Esta última posibilidad es la que se aprovecha posteriormente en la arquitectura desarrollada en este trabajo.

2.4.3. $\pi_{0.5}$

El modelo $\pi_{0.5}$ amplió el planteamiento de π_0 con el objetivo de mejorar la generalización ante nuevas escenas y tareas de mayor duración [4]. La propuesta combinó demostraciones robóticas con fuentes multimodales heterogéneas, entre ellas anotaciones semánticas, instrucciones verbales y datos procedentes de la web.

Al igual que π_0, $\pi_{0.5}$ no se plantea como una política asociada exclusivamente a un único manipulador, sino que su entrenamiento combina datos robóticos *cross-embodiment* con demostraciones obtenidas en distintos entornos y plataformas, además de información multimodal no robótica [4]. La mezcla utilizada incorpora datos de manipulación con robots estáticos, aproximadamente 400 horas de manipulación móvil y fuentes adicionales destinadas a reforzar la predicción semántica y la generalización a entornos nuevos.

Esto debe distinguirse de la configuración concreta utilizada posteriormente en este TFG. El *checkpoint* `pi05_droid` distribuido mediante OpenPI está ajustado al dominio DROID, cuya plataforma de adquisición utiliza un Franka Panda [12], [17]. Por tanto, la familia $\pi_{0.5}$ no es una política "de Franka", aunque la configuración DROID empleada en los experimentos sí hereda las convenciones de observación y acción de ese *embodiment*.

El sistema completo presentado por sus autores combina predicciones semánticas de alto nivel con bloques de acciones de bajo nivel. Los experimentos se realizaron principalmente con manipuladores móviles y tareas domésticas de larga duración en viviendas no observadas durante el entrenamiento. Los resultados mostraron capacidad para completar secuencias prolongadas y responder a instrucciones abiertas en entornos nuevos [4].

Estas capacidades corresponden al sistema y a las configuraciones evaluadas en el trabajo original. Por tanto, no pueden atribuirse automáticamente a cualquier *checkpoint* derivado de la misma familia.

2.5. Evaluación y despliegue de modelos VLA

2.5.1. Transferencia entre plataformas robóticas

Los trabajos de Open X-Embodiment, Octo y π_0 muestran que el entrenamiento con datos de varios robots puede producir transferencia positiva y facilitar la adaptación a nuevos dominios [3], [10], [11]. Sin embargo, ninguno de estos resultados implica que los comandos generados sean directamente intercambiables entre plataformas.

Las publicaciones revisadas recurren a interfaces específicas, normalizaciones, ajustes posteriores o redefiniciones del espacio de acciones para adaptar las políticas a cada sistema. Esto refleja que la transferencia entre *embodiments* continúa siendo un problema de integración y no únicamente de generalización visual o lingüística.

La transferencia también afecta a la forma de utilizar las secuencias temporales generadas por una política. ACT, por ejemplo, emplea *action chunking* y agregación temporal para ejecutar predicciones por bloques [19]. Por otra parte, se han propuesto métodos de transferencia entre robots que alinean los espacios de estado y acción de diferentes manipuladores mediante representaciones latentes compartidas [20]. Estos trabajos muestran que tanto el tratamiento temporal de las acciones como la correspondencia entre espacios de acción deben considerarse al transferir una política entre plataformas.

OpenVLA-OFT es una variante de OpenVLA obtenida mediante una estrategia de *fine-tuning* optimizada que modifica, entre otros aspectos, la forma de decodificar y representar las acciones e incorpora generación por bloques para mejorar la eficiencia y el rendimiento durante la adaptación [21]. Resulta relevante en este trabajo como ejemplo de que la representación y adaptación de la acción pueden modificar de forma importante el comportamiento de una política VLA.

Un estudio posterior sobre OpenVLA y OpenVLA-OFT en una plataforma UR5e examinó específicamente la transferencia desde los dominios de entrenamiento hacia un robot físico diferente [22]. Los autores observaron discrepancias entre las acciones predichas y el sistema de coordenadas físico, así como diferencias entre determinados indicadores obtenidos *offline* y el comportamiento real del robot.

El estudio concluyó que el despliegue debía considerarse como un problema de alineamiento del sistema completo, incluyendo los datos, el modelo, la representación de acciones y la interfaz de control [22]. Este resultado es especialmente relevante para trabajos que utilizan políticas preentrenadas sobre robots distintos de las plataformas de origen.

2.5.2. Robustez lingüística y asociación visual

VLABench propuso un entorno de evaluación de manipulación condicionado por lenguaje que reúne 100 categorías de tareas. El banco de pruebas incluye instrucciones con objetivos implícitos, relaciones espaciales, conocimiento común y comportamientos de varios pasos [23].

Los resultados publicados muestran que el éxito en tareas elementales no garantiza una comprensión robusta ante instrucciones semánticamente complejas o tareas de mayor horizonte. Los modelos evaluados presentaron dificultades al combinar razonamiento lingüístico, conocimiento visual y generación de movimientos prolongados.

LIBERO-Para estudió específicamente la robustez frente a paráfrasis mediante variaciones controladas de las expresiones de acción y de las referencias a objetos. En siete configuraciones pertenecientes a cuatro familias de arquitecturas, el rendimiento disminuyó entre 22,8 y 51,9 puntos porcentuales al emplear instrucciones reformuladas [24].

El análisis identificó la variación léxica de los objetos como uno de los principales factores de degradación. También mostró que numerosos fallos correspondían a trayectorias que divergían desde su inicio, en lugar de pequeños errores acumulados durante la ejecución. Estos resultados se obtuvieron en simulación, por lo que requieren validación adicional sobre sistemas físicos.

ICBench estudió el comportamiento de diferentes modelos VLA ante instrucciones contradictorias manteniendo fija la observación visual. Los autores observaron que algunos modelos continuaban ejecutando acciones visualmente plausibles aunque la instrucción fuera incompatible con la escena [25].

El trabajo denominó a este comportamiento ceguera lingüística y propuso un mecanismo de recalibración de la atención durante la inferencia. Los resultados mostraron que aumentar la influencia de la instrucción podía reducir parte de estos fallos sin volver a entrenar completamente el modelo [25].

En conjunto, estos estudios indican que la tasa de éxito final no permite identificar por sí sola si el fallo procede de la interpretación lingüística, de la selección visual o de la generación de la trayectoria. También justifican la evaluación separada de las distintas etapas de una política multimodal.

2.5.3. Evaluación física y seguridad

La evaluación de políticas robóticas sobre plataformas reales requiere preparar las escenas, ejecutar los movimientos, comprobar los resultados y restablecer las condiciones iniciales. Estas operaciones dificultan la realización de campañas extensas y pueden introducir variabilidad entre episodios.

AutoEval abordó este problema mediante un sistema orientado a automatizar la detección del éxito y el reinicio de las escenas [26]. Los autores mostraron que estas operaciones podían reducir la intervención humana y facilitar evaluaciones prolongadas. Su propuesta también destacó la necesidad de utilizar criterios de éxito y procedimientos de preparación consistentes.

La seguridad de los modelos VLA continúa siendo una línea abierta. Una revisión reciente identificó riesgos asociados a perturbaciones visuales y lingüísticas, errores de percepción, generación de trayectorias inseguras y falta de mecanismos de recuperación [27].

La literatura revisada propone complementar las políticas con restricciones externas, comprobaciones durante la ejecución, sistemas de monitorización y mecanismos de recuperación. Sin embargo, gran parte de las evaluaciones de seguridad continúa realizándose en simulación, por lo que sus garantías no pueden trasladarse directamente a una célula física [27].

Estos trabajos muestran que la evaluación física no debe limitarse a contabilizar ejecuciones correctas e incorrectas. También resulta necesario registrar las etapas alcanzadas y distinguir los fallos de interpretación, percepción, planificación, agarre, movimiento y verificación.

En este trabajo se optó por realizar la validación principal sobre el sistema físico porque el problema estudiado no se limita al comportamiento de la política de forma aislada, sino a su transferencia e integración con una plataforma diferente de sus configuraciones de origen. Una evaluación exclusivamente simulada permitiría controlar con mayor facilidad las condiciones experimentales, pero no reproduciría completamente aspectos como los errores de calibración, las diferencias entre espacios de acción, el comportamiento real del efector, la comunicación con el robot o la verificación física de la colocación. Estos factores forman parte del problema de despliegue analizado en este TFG y han sido identificados también como relevantes

en evaluaciones de modelos VLA sobre plataformas reales [22]. Por ello, la simulación puede complementar este tipo de estudios, pero no sustituir la validación física cuando el objetivo es analizar el funcionamiento del sistema integrado.

2.6. Posicionamiento del trabajo

La literatura revisada muestra avances significativos en el entrenamiento de políticas generalistas, la combinación de datos procedentes de múltiples robots y la interpretación de instrucciones lingüísticas. También existen trabajos específicos sobre transferencia entre plataformas, robustez frente a paráfrasis, contradicciones lingüísticas, evaluación automática y seguridad.

Sin embargo, los trabajos analizados no ofrecen una comparación directa de OpenVLA, π_0 y $\pi_{0.5}$ bajo un mismo protocolo físico y sobre una plataforma distinta de sus configuraciones de origen. Las diferencias entre robots, interfaces, tareas y métricas dificultan atribuir las variaciones de rendimiento exclusivamente al modelo utilizado.

La elección de OpenVLA, π_0 y $\pi_{0.5}$ no pretende abarcar de forma exhaustiva el conjunto de modelos VLA disponibles, sino comparar alternativas que resultan especialmente útiles para el problema estudiado. OpenVLA representa un enfoque abierto basado en la generación autorregresiva de componentes de acción discretizadas [2], mientras que π_0 y $\pi_{0.5}$ pertenecen a una familia diferente, en la que las acciones se generan de forma continua mediante *flow matching* y pueden producirse como bloques temporales [3], [4]. La inclusión conjunta de π_0 y $\pi_{0.5}$ permite además estudiar dos generaciones de una misma familia, mientras que OpenVLA introduce un planteamiento distinto tanto en su arquitectura como en la representación de la acción.

A esta complementariedad se añadió un criterio práctico de reproducibilidad. Los tres modelos disponen de implementaciones, *checkpoints* e interfaces de inferencia accesibles, lo que permitió desplegarlos, inspeccionar sus entradas y salidas y conservar las predicciones generadas durante los experimentos [14], [17]. Esta condición resulta relevante en un trabajo que no se limita a comparar el éxito final de la tarea, sino que busca identificar qué ocurre durante la interpretación, el *grounding*, la generación de acciones y su posterior transferencia al robot.

En este contexto, el presente TFG compara OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID sobre una célula basada en un UR5e. El objetivo no es reproducir las arquitecturas de control originales ni demostrar un control VLA completo de todos los grados de libertad, sino estudiar qué información puede transferirse y en qué etapas aparecen las principales dificultades.

Para ello, el protocolo diferencia la interpretación lingüística, la asociación entre instrucciones y entidades visuales, la generación y adaptación de acciones, la ejecución física y la verificación del resultado. Asimismo, se estudian las secuencias temporales de la familia π y el efecto de un adaptador visual–lingüístico en parte del flujo de OpenVLA.

3 Marco teórico

3.1. Introducción

Los modelos VLA combinan conceptos procedentes del aprendizaje profundo, la representación multimodal y el control robótico. Para comprender su utilización sobre una plataforma física resulta necesario considerar tanto los mecanismos mediante los que procesan la información visual y lingüística como las restricciones geométricas y cinemáticas asociadas al robot que finalmente ejecuta las acciones.

El Capítulo 2 presentó la evolución de las políticas robóticas generalistas, los principales conjuntos de datos y los modelos relacionados con este trabajo. En cambio, este capítulo se centra en los fundamentos necesarios para comprender su funcionamiento y su transferencia a una célula de manipulación. Se revisan primero los principios del aprendizaje mediante redes neuronales, el preentrenamiento y las arquitecturas multimodales, prestando especial atención al mecanismo de atención y a los codificadores visuales. A continuación se describen las políticas VLA, sus principales formas de representar acciones y el concepto de *grounding* visual-lingüístico.

La segunda parte introduce los fundamentos geométricos necesarios para trasladar las predicciones de una política a un manipulador físico, se describen los espacios articular y cartesiano, la cinemática del manipulador y las transformaciones entre sistemas de coordenadas y, por último, se analiza el problema de transferir políticas preentrenadas entre plataformas robóticas con diferentes espacios de observación y acción.

3.2. Aprendizaje profundo y modelos preentrenados

3.2.1. Redes neuronales y aprendizaje a partir de datos

Una red neuronal artificial puede describirse como una composición de transformaciones parametrizadas cuyos parámetros se ajustan a partir de los datos durante el entrenamiento [28]. En una capa densamente conectada, una entrada se transforma mediante

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}, \tag{3.1}$$

y posteriormente se aplica una función de activación no lineal,

$$\mathbf{h}^{(l)} = \phi\left(\mathbf{z}^{(l)}\right). \tag{3.2}$$

La combinación de múltiples capas permite aprender representaciones progresivamente más complejas sin definir manualmente todas las características utilizadas por el sistema. Durante el entrenamiento, los parámetros se modifican para reducir una función de pérdida que cuantifica la diferencia entre las predicciones del modelo y el objetivo considerado. La retropropagación permite calcular los gradientes necesarios para realizar estas actualizaciones mediante un algoritmo de optimización [28].

El aprendizaje no tiene por qué depender exclusivamente de ejemplos etiquetados. Los modelos actuales pueden adquirir representaciones mediante objetivos supervisados, autosupervisados o multimodales y reutilizarlas posteriormente en otras tareas. Este principio es especialmente relevante para los modelos VLA considerados en este trabajo, que parten de representaciones previamente aprendidas sobre grandes cantidades de información visual, lingüística y robótica.

3.2.2. Preentrenamiento y transferencia

Entrenar modelos profundos con capacidad de generalización requiere grandes cantidades de datos y recursos de cómputo. El preentrenamiento busca reutilizar ese coste aprendiendo primero representaciones sobre conjuntos amplios y empleándolas posteriormente en otras tareas o dominios.

La transferencia de aprendizaje parte de la idea de que parte del conocimiento adquirido para una tarea puede resultar útil en otra relacionada. En lugar de inicializar todos los parámetros de un nuevo modelo de forma arbitraria, se parte de parámetros previamente aprendidos. Dependiendo de la aplicación, algunas partes pueden conservarse y otras pueden ajustarse mediante *fine-tuning*.

OpenVLA y Octo contemplan procedimientos de adaptación a nuevas plataformas mediante ajuste posterior [2], [11]. En este trabajo, en cambio, los *checkpoints* evaluados se utilizan sin realizar *fine-tuning* con demostraciones propias del UR5e. De este modo, se analiza hasta qué punto la información aprendida previamente puede reutilizarse mediante una interfaz externa de integración.

El preentrenamiento sobre datos extensos y heterogéneos permite obtener representaciones reutilizables más allá de una única tarea o dominio. En las políticas VLA consideradas en este trabajo, este principio se materializa mediante el aprendizaje previo a partir de información visual, lingüística y robótica, que posteriormente se reutiliza durante el despliegue [2]-[4].

Conviene distinguir, no obstante, la transferencia de aprendizaje de la transferencia entre *embodiments*. La primera se refiere principalmente a la reutilización del conocimiento aprendido por un modelo. La segunda aparece cuando una política debe utilizarse sobre una realización física diferente, con otros sensores, cinemática, efector o espacio de acciones. En este trabajo no se modifican mediante entrenamiento los parámetros de las políticas, pero sí se estudia esta segunda forma de transferencia.

3.3. Modelos multimodales de visión y lenguaje

Una política condicionada simultáneamente por una imagen y una instrucción debe representar información procedente de modalidades diferentes dentro de un espacio computacional compatible. Los modelos multimodales proporcionan mecanismos para construir estas representaciones y establecer relaciones entre elementos visuales y lingüísticos.

3.3.1. Arquitectura *transformer* y mecanismo de atención

La arquitectura *transformer* fue propuesta inicialmente para el procesamiento de secuencias y sustituyó la dependencia recurrente de arquitecturas anteriores por mecanismos de atención [29]. Su componente fundamental es la autoatención, que permite construir la representación de cada elemento utilizando información de otros elementos de la misma secuencia.

Sea una secuencia representada mediante la matriz $\mathbf{X}$. A partir de ella se calculan tres proyecciones aprendidas:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{3.3}$$

denominadas consultas (*queries*), claves (*keys*) y valores (*values*). Conceptualmente, una consulta representa la información que un elemento busca en el resto de la secuencia; las claves permiten determinar qué elementos son compatibles con dicha consulta, y los valores contienen la información que se combinará para producir la nueva representación.

La atención del producto escalar se expresa como:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}, \tag{3.4}$$

donde d_k es la dimensión de las claves [29]. El producto $\mathbf{Q}\mathbf{K}^T$ proporciona puntuaciones de compatibilidad entre los elementos. La función *softmax* las transforma en pesos normalizados y la multiplicación posterior por $\mathbf{V}$ produce una combinación ponderada de los valores.

El factor $\sqrt{d_k}$ limita el crecimiento de los productos escalares cuando aumenta la dimensión de las representaciones. De este modo se evita que los valores introducidos en la función *softmax* alcancen magnitudes excesivas que dificulten su comportamiento numérico.

La autoatención permite que cada elemento establezca relaciones directas con los demás elementos de la secuencia sin tener que propagar la información secuencialmente a través de estados recurrentes. Además, las representaciones de los diferentes elementos pueden calcularse de forma paralela durante el entrenamiento. Como contrapartida, una atención global requiere comparar los elementos por pares, por lo que su coste aumenta cuadráticamente con la longitud de la secuencia.

Los *transformers* emplean normalmente atención multi-cabeza o *multi-head attention*. En lugar de realizar una única operación con **Q**, **K** y **V**, se utilizan varias proyecciones aprendidas en paralelo. Cada cabeza puede representar relaciones diferentes y sus resultados se concatenan y proyectan posteriormente [29].

La atención por sí sola tampoco introduce información sobre el orden de los elementos. Por este motivo se incorporan representaciones posicionales a los *embeddings* de entrada. En el *transformer* original se utilizaron codificaciones posicionales sinusoidales [29]; otras arquitecturas emplean representaciones posicionales aprendibles.

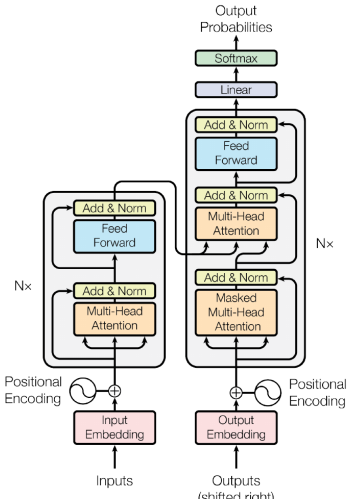


FIGURA 3.1: Arquitectura original del *Transformer*. Fuente: [29].

La Figura 3.1 representa la arquitectura encoder–decoder propuesta originalmente. Los modelos posteriores pueden reutilizar solo parte de esta estructura o introducir modificaciones sustanciales, por lo que la figura debe interpretarse como fundamento del mecanismo y no como la arquitectura exacta de los modelos VLA evaluados.

En modelos multimodales, las representaciones visuales y lingüísticas deben integrarse de forma que puedan participar conjuntamente en el procesamiento posterior. La arquitectura concreta utilizada para realizar esta integración varía entre modelos; por ello, el mecanismo de atención presentado en esta sección debe entenderse como un fundamento común y no como una descripción única de las arquitecturas VLA consideradas [2], [3].

3.3.2. Codificadores visuales y alineamiento imagen–texto

Las imágenes no se presentan naturalmente como una secuencia unidimensional de elementos. Vision Transformer (ViT) mostró que podían adaptarse al procesamiento mediante *transformers* dividiendo la imagen en regiones de tamaño fijo, denominadas *patches* [30].

Para una imagen de dimensiones $H \times W$ y *patches* cuadrados de lado P, el número de regiones resultante es:

$$N = \frac{HW}{P^2}. \tag{3.5}$$

Cada región se aplana y se transforma mediante una proyección lineal en un vector de dimensión fija. La secuencia visual puede representarse de forma simplificada como:

$$\mathbf{Z}_0 = [\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_N] + \mathbf{E}_{pos}, \tag{3.6}$$

donde $\mathbf{E}_{pos}$ contiene información posicional. En la formulación original de ViT se emplean *embeddings* posicionales aprendibles [30]. Estas representaciones pueden procesarse entonces mediante bloques de autoatención de forma análoga a una secuencia de *tokens*.

El desarrollo posterior de modelos visuales a gran escala ha permitido obtener representaciones transferibles sin necesidad de entrenar un clasificador específico para cada aplicación. DINOv2, por ejemplo, utiliza aprendizaje autosupervisado para obtener características visuales reutilizables en diferentes tareas [15].

Otra línea de trabajo busca alinear directamente imagen y lenguaje. En este caso, un codificador visual y un codificador textual producen representaciones que pueden compararse dentro de un espacio común. Una medida de semejanza habitual entre una imagen I y un texto T es la similitud coseno:

$$s(I, T) = \frac{\mathbf{z}_I^T \mathbf{z}_T}{\|\mathbf{z}_I\|_2 \|\mathbf{z}_T\|_2}, \tag{3.7}$$

donde $\mathbf{z}_I$ y $\mathbf{z}_T$ son las representaciones de ambas modalidades.

SigLIP, como referencia, pertenece a esta familia de modelos de alineamiento imagen–texto. Su entrenamiento emplea una pérdida sigmoide aplicada a pares de imagen y texto, evitando la normalización global mediante *softmax* utilizada en otros enfoques contrastivos [7]. Una vez preentrenado, puede utilizarse para comparar la compatibilidad relativa entre contenido visual y diferentes descripciones.

El alineamiento semántico no equivale, sin embargo, a la localización espacial. Un codificador imagen–texto puede estimar la compatibilidad de una imagen o de una región previamente delimitada con una expresión lingüística, pero no proporciona necesariamente los límites espaciales de una instancia ni sus coordenadas tridimensionales. Esta diferencia será importante al introducir el *grounding* utilizado en tareas robóticas.

3.4. Políticas de visión, lenguaje y acción

3.4.1. De los modelos visión–lenguaje a las políticas VLA

Los modelos visión–lenguaje relacionan contenido visual y lingüístico, pero su salida habitual continúa perteneciendo al dominio semántico. Una política de visión, lenguaje y acción amplía este planteamiento incorporando una representación explícita de la acción robótica.

De forma general, una política VLA puede representarse como:

$$\mathbf{a} \sim \pi_\theta \left(\mathbf{o}, \mathbf{l}, \mathbf{s} \right), \tag{3.8}$$

donde **o** representa la observación visual, **l** la instrucción lingüística, **s** un posible estado adicional del robot y **a** la acción predicha. La formulación exacta depende de la arquitectura: no todas las políticas utilizan explícitamente los mismos tipos de observación, estado o representación de acciones.

Los modelos considerados en este trabajo permiten ilustrar dos estrategias diferentes de generación de acciones. OpenVLA formula la predicción de acción como una tarea de generación autorregresiva sobre componentes discretizadas [2], mientras que la familia π genera acciones continuas mediante un experto específico basado en *flow matching* [3], [4].

La Figura 3.2 muestra la arquitectura de OpenVLA y sirve como referencia para comprender la integración entre representación visual, lenguaje y generación de acciones.

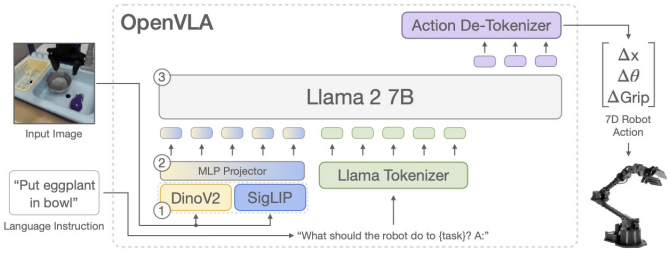


FIGURA 3.2: Arquitectura de OpenVLA. Fuente: [2].

3.4.2. Representación y generación de acciones

Una política robótica puede representar sus acciones mediante desplazamientos cartesianos, posiciones o velocidades articulares, cambios de orientación o comandos asociados al efector, dependiendo de la interfaz utilizada durante su entrenamiento y despliegue [2], [3]. Una acción de dimensión D puede escribirse de forma genérica como:

$$\mathbf{a}_t = \begin{bmatrix} a_{t,1} & a_{t,2} & \cdots & a_{t,D} \end{bmatrix}^T. \tag{3.9}$$

Una misma tarea puede representarse de manera diferente según la política. Ante una instrucción de *pick-and-place*, OpenVLA produce en cada inferencia un único vector que describe una acción relativa del efector, mientras que las políticas de la familia π pueden producir conjuntamente una secuencia de varios vectores de acción. Así, aunque ambas políticas respondan a la misma observación y al mismo objetivo, una proporciona una predicción individual y la otra un bloque temporal que representa la evolución prevista durante varios instantes internos.

Además, una acción puede definirse en términos absolutos o relativos. En el primer caso especifica directamente un estado o una referencia objetivo, mientras que en el segundo representa un incremento respecto al estado actual:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \Delta \mathbf{x}_t. \tag{3.10}$$

La interpretación temporal también forma parte de la definición de la acción: una componente incremental está asociada a la frecuencia o al intervalo temporal con el que fue definida durante el entrenamiento. Por ello, la dimensión del vector no basta para determinar su significado; deben conocerse además las unidades, el sistema de referencia, la normalización y la interpretación temporal de cada componente [3], [10].

3.4.3. Predicción autorregresiva y discretización

En una generación autorregresiva, el modelo construye la acción componente a componente, donde cada nuevo valor se predice teniendo en cuenta la observación, la instrucción y las componentes generadas anteriormente. Para poder utilizar este mecanismo de forma análoga a la generación de tokens de lenguaje, OpenVLA discretiza previamente las componentes continuas de la acción [2].

Una acción continua puede representarse mediante un conjunto finito de intervalos. Para una componente a, una operación de cuantización asigna el valor continuo a uno de K estados discretos:

$$a \longrightarrow q(a) \in \{1, \ldots, K\}. \tag{3.11}$$

Una vez discretizadas, las componentes pueden incorporarse al vocabulario utilizado por un modelo generativo. Para una acción de varias componentes, una factorización autorregresiva puede escribirse como:

$$p(\mathbf{a} \mid \mathbf{o}, \mathbf{l}) = \prod_{d=1}^{D} p\left(a_d \mid a_{1:d-1}, \mathbf{o}, \mathbf{l}\right). \tag{3.12}$$

Cada componente se genera condicionada por la observación, la instrucción y las componentes producidas previamente.

OpenVLA adopta este enfoque y convierte las componentes discretizadas de las acciones en *tokens* que pueden ser generados por el modelo de lenguaje [2]. Tras la inferencia se realiza la operación inversa para recuperar valores numéricos de acción.

Esta formulación permite reutilizar directamente mecanismos de generación autorregresiva desarrollados para modelos de lenguaje. Como contrapartida, la salida depende de la discretización empleada y de las convenciones con las que se representaron las acciones durante el entrenamiento.

3.4.4. *Flow matching* y bloques temporales de acciones

Una alternativa consiste en modelar directamente distribuciones sobre variables continuas. El *flow matching* permite aprender un campo vectorial que describe la transformación de una distribución inicial sencilla hacia una distribución objetivo [31].

De forma simplificada, puede expresarse mediante la ecuación diferencial:

$$\frac{d\mathbf{x}_\tau}{d\tau} = \mathbf{v}_\theta\left(\mathbf{x}_\tau, \tau, \mathbf{c}\right), \tag{3.13}$$

donde $\mathbf{v}_\theta$ es el campo vectorial aprendido, $\mathbf{c}$ representa el condicionamiento y τ corresponde a la variable continua utilizada durante el proceso generativo. Esta variable no debe confundirse con el instante físico t de una trayectoria robótica.

De forma intuitiva, el *flow matching* permite generar directamente valores continuos de acción en lugar de producirlos como una secuencia de tokens discretizados. En π_0, esta función recae sobre el denominado *action expert*, que se trata de un módulo neuronal aprendido que recibe las representaciones multimodales del modelo y genera las acciones robóticas continuas [3]. La Figura 3.3 muestra la arquitectura general presentada por sus autores.

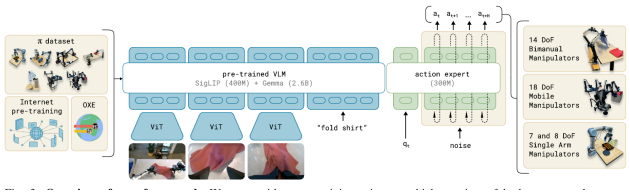


FIGURA 3.3: Arquitectura general de π_0 (Figura 3 del trabajo original). Fuente: [3].

En lugar de generar únicamente una acción, estas políticas pueden producir un bloque temporal o *action chunk*:

$$\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_H], \qquad \mathbf{a}_t \in \mathbb{R}^D, \tag{3.14}$$

donde H es el horizonte del bloque y D la dimensión de cada acción. De esta forma, una única inferencia proporciona una secuencia de acciones relacionadas temporalmente.

$\pi_{0.5}$ mantiene la generación continua de acciones de la familia π_0 e incorpora mecanismos destinados a combinar predicciones semánticas de alto nivel con acciones de bajo nivel [4]. En una tarea de manipulación, el nivel alto puede expresar qué objetivo debe alcanzarse, por ejemplo "colocar el objeto en el recipiente adecuado", mientras que el nivel bajo corresponde a las acciones numéricas necesarias para producir el movimiento. La Figura 3.4 reproduce la visión general presentada por sus autores.

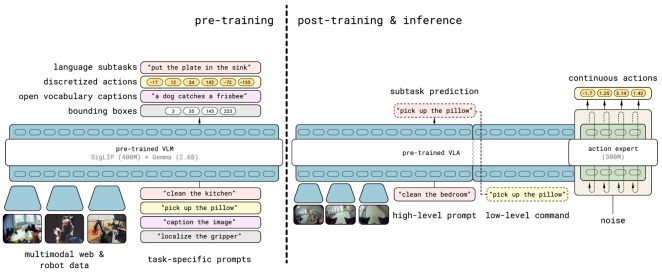


FIGURA 3.4: Visión general del entrenamiento y la arquitectura de $\pi_{0.5}$ (Figura 3 del trabajo original). Fuente: [4].

La generación de bloques permite modelar relaciones temporales entre varias acciones dentro de una misma inferencia. Sin embargo, el bloque se calcula condicionado por una observación determinada. Ejecutar una secuencia completa sin volver a observar el entorno no equivale a disponer de un controlador realimentado que actualiza continuamente su decisión. Esta diferencia resulta especialmente relevante al desplegar políticas de acción temporal sobre un sistema físico.

3.4.5. *Grounding* visual-lingüístico en la selección de entidades

Para ejecutar una instrucción como coloca el cilindro blanco en la zona amarilla, el sistema debe determinar qué elemento físico corresponde al cilindro blanco y cuál constituye la zona amarilla. La asociación entre una expresión lingüística y una entidad observable de la escena forma parte del problema de *grounding*.

El problema aparece tanto en la asociación de lenguaje con objetos tridimensionales como en tareas de manipulación condicionadas por instrucciones. La representación lingüística debe vincularse con una entidad concreta antes de que pueda definirse una acción física sobre ella [32], [33].

Conviene diferenciar este problema de otras tareas visuales. La clasificación asigna una categoría o contenido semántico a una imagen o región, mientras que la detección combina la identificación de instancias con su localización espacial. El *grounding* añade la necesidad de resolver qué entidad observada corresponde específicamente a una referencia lingüística [32], [33].

Para un conjunto de candidatos

$$\mathcal{C} = \{c_1, \ldots, c_N\}, \tag{3.15}$$

y una descripción lingüística l, una formulación sencilla consiste en seleccionar:

$$c^* = \arg\max_{c_i \in \mathcal{C}} s(c_i, l), \tag{3.16}$$

donde $s(c_i, l)$ cuantifica la compatibilidad entre cada candidato y la descripción.

Un modelo de alineamiento imagen–texto como SigLIP puede utilizarse para obtener este tipo de puntuaciones cuando recibe regiones visuales previamente delimitadas [7]. Estas deben ser obtenidas por otro procedimiento de percepción, pues SigLIP compara su contenido con las descripciones textuales, pero no determina por sí mismo sus límites espaciales.

El problema se vuelve más complejo cuando existen varias instancias similares, sinónimos, relaciones espaciales, referencias ambiguas o entidades que no están presentes en la escena. En estas situaciones, seleccionar siempre el candidato con mayor puntuación puede no ser suficiente, sino también debe considerarse si la evidencia permite realizar una asociación inequívoca.

El *grounding* visual-lingüístico debe distinguirse de la localización métrica, pues determinar que una región corresponde al objeto indicado por la instrucción no proporciona necesariamente una posición tridimensional adecuada para ejecutar una

trayectoria. La conexión entre ambas etapas requiere un sistema de percepción geométrica y una transformación entre los sistemas de coordenadas del sensor y del robot.

3.5. Geometría, cinemática y percepción tridimensional

La información producida por una política debe relacionarse finalmente con las variables mediante las que se describe y ejecuta el movimiento del manipulador. Para ello resulta necesario distinguir entre el espacio articular y el espacio cartesiano, establecer la relación cinemática entre ambos y expresar las mediciones procedentes de sensores externos en un sistema de coordenadas común [34], [35].

3.5.1. Espacio articular y espacio cartesiano

Un manipulador de n articulaciones puede describirse mediante el vector de coordenadas articulares

$$\mathbf{q} = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}^T, \tag{3.17}$$

que determina su configuración. Este conjunto de variables define el espacio articular del manipulador. El espacio cartesiano, denominado también espacio operacional en la literatura de robótica de manipulación, describe en cambio la posición y orientación del efector final respecto a un sistema de referencia [34].

Las tareas de manipulación suelen formularse de manera natural en este segundo espacio, mediante objetivos como alcanzar un objeto, elevarlo o situarlo en una posición determinada. Cuando la interfaz de ejecución utiliza referencias articulares, estos objetivos deben convertirse en configuraciones compatibles con la geometría y los límites del robot mediante su modelo cinemático.

También conviene distinguir entre camino y trayectoria. Un camino describe la evolución geométrica entre configuraciones o poses, mientras que una trayectoria incorpora además una parametrización temporal del movimiento [34]. En una tarea definida mediante varios puntos de paso, la continuidad entre las configuraciones articulares asociadas resulta relevante para evitar cambios innecesarios entre diferentes soluciones cinemáticas.

3.5.2. Cinemática directa e inversa

La cinemática directa relaciona una configuración articular con la pose del efector final:

$$^{B}\mathbf{T}_{E} = f(\mathbf{q}), \tag{3.18}$$

donde $^{B}\mathbf{T}_{E}$ representa la transformación del efector E respecto a la base B y $f(\cdot)$ depende de la geometría del manipulador [34].

La cinemática inversa plantea el problema contrario: determinar una o varias configuraciones articulares capaces de producir una pose objetivo,

$$\mathbf{q}^* \in f^{-1}\left({}^{B}\mathbf{T}_{E}^{*}\right). \tag{3.19}$$

A diferencia del problema directo, la cinemática inversa puede presentar múltiples soluciones. También puede no existir una configuración que alcance una pose situada fuera del espacio de trabajo del manipulador [34].

Las singularidades constituyen un problema diferente. El jacobiano, que relaciona localmente el movimiento articular con el movimiento cartesiano del efector, pierde rango en una configuración singular y se reduce la capacidad instantánea del manipulador para producir determinados movimientos cartesianos. Incluso en las proximidades de una singularidad, la inversión diferencial puede requerir variaciones articulares de gran magnitud, por lo que resulta necesario considerar el condicionamiento del problema además de la alcanzabilidad geométrica [34].

3.5.3. Jacobiano y cinemática inversa diferencial

El jacobiano geométrico relaciona localmente las velocidades articulares con las velocidades lineal y angular del efector:

$$\begin{bmatrix}\mathbf{v}\\ \boldsymbol{\omega}\end{bmatrix} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}, \tag{3.20}$$

donde $\mathbf{v}$ representa la velocidad lineal, $\boldsymbol{\omega}$ la velocidad angular y $\mathbf{J}(\mathbf{q})$ el jacobiano del manipulador [34].

Esta relación permite construir métodos iterativos de cinemática inversa. Si $\mathbf{e}$ representa el error cartesiano utilizado por el procedimiento, una actualización mediante mínimos cuadrados amortiguados (*damped least squares*, DLS) puede escribirse como

$$\Delta\mathbf{q} = \mathbf{J}^T\left(\mathbf{J}\mathbf{J}^T + \lambda^2\mathbf{I}\right)^{-1}\mathbf{e}, \tag{3.21}$$

donde λ es el factor de amortiguamiento [34]. Su introducción mejora el condicionamiento numérico de la inversión en las proximidades de configuraciones singulares, a costa de admitir un compromiso entre el seguimiento exacto del error y la magnitud de la actualización articular.

La configuración se actualiza iterativamente mediante

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \Delta\mathbf{q}_k. \tag{3.22}$$

Cuando una tarea no restringe completamente todos los grados de libertad, pueden utilizarse criterios secundarios para seleccionar entre configuraciones que satisfacen el objetivo principal. La teoría clásica de redundancia explota para ello los grados de libertad asociados al espacio nulo del jacobiano [34]. En una implementación concreta también pueden incorporarse criterios de continuidad para favorecer soluciones próximas a la configuración utilizada como referencia, manteniendo la convergencia cartesiana como objetivo principal.

3.5.4. Efector final y punto central de la herramienta

La pose asociada a la última articulación o a la brida de acoplamiento del manipulador no coincide necesariamente con el punto físico utilizado para interactuar con el entorno. Al instalar una pinza u otra herramienta aparece una transformación adicional entre la brida y el punto relevante para la tarea.

El punto central de la herramienta o TCP (*Tool Center Point*) define esta referencia. Si ${}^{E}\mathbf{T}_{TCP}$ representa la transformación entre la brida E y el TCP, su pose respecto a la base viene dada por

$$ {}^{B}\mathbf{T}_{TCP} = {}^{B}\mathbf{T}_{E}\,{}^{E}\mathbf{T}_{TCP}. \tag{3.23} $$

Esta transformación, denominada habitualmente transformación de herramienta, permite que las referencias cartesianas se expresen respecto al punto físico que debe alcanzar la tarea y no únicamente respecto a la brida del robot [35].

Un error en la estimación del TCP se propaga a las posiciones calculadas para la herramienta. Este aspecto resulta especialmente relevante en operaciones de agarre y deposición, en las que la posición de contacto debe corresponder al efector instalado.

3.5.5. Sistemas de coordenadas y transformaciones homogéneas

Una pose representa conjuntamente la posición y la orientación de un sistema respecto a otro sistema de referencia. En robótica es habitual trabajar simultáneamente con sistemas asociados a la base del manipulador, la brida, la herramienta, los sensores y los objetos de la escena [34], [35].

Una transformación rígida tridimensional entre dos sistemas puede representarse mediante una matriz homogénea

$$ {}^{A}\mathbf{T}_{B} = \begin{bmatrix} {}^{A}\mathbf{R}_{B} & {}^{A}\mathbf{t}_{B} \\ \mathbf{0}^{T} & 1 \end{bmatrix}, \tag{3.24} $$

donde ${}^{A}\mathbf{R}_{B} \in SO(3)$ expresa la orientación del sistema B respecto a A y ${}^{A}\mathbf{t}_{B}$ su traslación.

Para un punto expresado mediante coordenadas homogéneas ${}^{B}\mathbf{p}$, su representación en el sistema A se obtiene como

$$ {}^{A}\mathbf{p} = {}^{A}\mathbf{T}_{B}\,{}^{B}\mathbf{p}. \tag{3.25} $$

Las transformaciones pueden concatenarse. Si se conocen ${}^{A}\mathbf{T}_{B}$ y ${}^{B}\mathbf{T}_{C}$,

$$ {}^{A}\mathbf{T}_{C} = {}^{A}\mathbf{T}_{B}\,{}^{B}\mathbf{T}_{C}. \tag{3.26} $$

Esta propiedad permite relacionar de forma sistemática las referencias asociadas a la cámara, la base del robot, la brida y el TCP [34], [35].

3.5.6. Percepción 3D y calibración extrínseca *eye-to-hand*

Un sensor tridimensional proporciona una representación geométrica de la escena mediante un conjunto de puntos

$$\mathcal{P} = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_M\}, \qquad \mathbf{p}_j = \begin{bmatrix} x_j & y_j & z_j \end{bmatrix}^T. \tag{3.27}$$

Cuando la información visual y tridimensional se encuentra registrada, una región identificada en la imagen puede asociarse con los puntos 3D correspondientes. De esta forma pueden obtenerse posiciones geométricas de los elementos de la escena que posteriormente se expresan en el sistema de referencia utilizado por el robot.

En una configuración *eye-to-hand*, la cámara se encuentra fija respecto al entorno y observa externamente el espacio de trabajo del manipulador. Para utilizar en la planificación las coordenadas medidas por el sensor debe conocerse la transformación extrínseca

$$^{B}\mathbf{T}_{C}, \tag{3.28}$$

que relaciona el sistema de coordenadas de la cámara C con el sistema de la base B [34], [35].

Esta transformación puede estimarse mediante correspondencias tridimensionales conocidas en ambos sistemas. Dados N pares

$$\left({}^{C}\mathbf{p}_i, {}^{B}\mathbf{p}_i\right), \qquad i = 1, \ldots, N, \tag{3.29}$$

la rotación $\mathbf{R}$ y la traslación $\mathbf{t}$ pueden obtenerse resolviendo

$$(\mathbf{R}^*, \mathbf{t}^*) = \underset{\mathbf{R}\in SO(3),\, \mathbf{t}\in\mathbb{R}^3}{\arg\min} \sum_{i=1}^{N} \left\| {}^{B}\mathbf{p}_i - \left(\mathbf{R}\, {}^{C}\mathbf{p}_i + \mathbf{t}\right) \right\|_2^2. \tag{3.30}$$

Este problema corresponde al registro rígido por mínimos cuadrados de dos conjuntos de puntos tridimensionales con correspondencias conocidas [36].

Una vez estimada ${}^{B}\mathbf{T}_{C}$, cualquier punto observado en el sistema de la cámara puede transformarse al sistema de la base mediante la Ecuación 3.25. El procedimiento experimental utilizado para determinar esta transformación en la célula se describe en el Capítulo 4.

3.6. Transferencia de políticas VLA a sistemas robóticos

Una política robótica se entrena bajo unas determinadas condiciones de observación y acción. Cuando se despliega sobre una plataforma distinta, las predicciones del modelo deben relacionarse con los sensores, los sistemas de referencia, la cinemática y la interfaz de control del robot de destino. Por tanto, la transferencia de una política no depende únicamente de que esta interprete correctamente la escena, sino también

de que exista una correspondencia adecuada entre las representaciones utilizadas durante el entrenamiento y las variables físicamente ejecutables.

3.6.1. Espacios de observación y acción

Una política robótica establece una relación entre un espacio de observación $\mathcal{O}$ y un espacio de acciones $\mathcal{A}$:

$$\mathbf{o}_t \in \mathcal{O}, \qquad \mathbf{a}_t \in \mathcal{A}. \tag{3.31}$$

El espacio de observación determina la información disponible para la política y puede incluir imágenes, estado articular, pose del efector, información de la pinza u otras variables. El espacio de acciones establece, por su parte, las magnitudes producidas por el modelo y las convenciones necesarias para interpretarlas.

Estas definiciones forman parte del contrato bajo el que se aprende una política. Una modificación de las cámaras, del orden de las variables, de las unidades, de la normalización, del sistema de coordenadas o del significado de los canales de acción puede hacer que una misma entrada o salida numérica deje de representar la misma magnitud física.

3.6.2. Transferencia entre *embodiments*

En robótica, el término *embodiment* se utiliza para distinguir las diferentes realizaciones físicas mediante las que una política interactúa con el entorno. En el contexto de este trabajo, estas diferencias comprenden la estructura cinemática del manipulador, el efector final, la disposición de los sensores y la interfaz mediante la que se ejecutan las acciones.

Dos plataformas pueden recibir la misma instrucción y observar una tarea equivalente, pero requerir acciones expresadas mediante convenciones y variables de control diferentes para ejecutarla físicamente. Por ello, el despliegue de una política sobre un nuevo *embodiment* requiere establecer una interfaz entre la representación producida por el modelo y las variables del robot de destino [10].

De forma abstracta, dicha interfaz puede representarse en esta memoria como

$$\mathbf{a}^{\text{robot}} = g\left(\mathbf{a}^{\text{modelo}}, \mathbf{s}^{\text{robot}}, \mathcal{C}\right), \tag{3.32}$$

donde $g(\cdot)$ representa la transformación necesaria para adaptar la salida, $\mathbf{s}^{\text{robot}}$ contiene el estado relevante de la plataforma de destino y $\mathcal{C}$ agrupa las convenciones utilizadas para interpretar la acción.

Entre estas convenciones pueden encontrarse las unidades, la escala, el sistema de coordenadas, la representación absoluta o incremental, la frecuencia de ejecución, los límites físicos y la configuración del efector. En consecuencia, la transferencia semántica y la transferencia de acciones son problemas diferentes: una política puede identificar correctamente qué debe hacerse y, al mismo tiempo, producir una representación de acción que no pueda aplicarse directamente sobre otro robot.

3.6.3. Arquitecturas *end-to-end*, modulares e híbridas

La arquitectura de un sistema robótico determina cómo se organizan y comunican los componentes encargados de percepción, decisión, planificación y ejecución. Las arquitecturas robóticas pueden distribuir estas funciones entre módulos con responsabilidades diferenciadas, lo que permite estructurar sistemas complejos y definir interfaces explícitas entre sus componentes [37].

En una política aprendida con un planteamiento *end-to-end*, se busca modelar directamente la relación entre las observaciones disponibles y la acción:

$$(\mathbf{o}, \mathbf{l}, \mathbf{s}) \longrightarrow \pi_\theta \longrightarrow \mathbf{a}, \tag{3.33}$$

donde $\mathbf{l}$ representa la instrucción lingüística y $\mathbf{s}$ un posible estado adicional del robot. Los modelos VLA constituyen ejemplos de políticas aprendidas que integran información visual, lingüística y de acción, aunque la definición exacta de las entradas y salidas depende de cada modelo [2], [3].

En una arquitectura modular, por el contrario, distintas funciones pueden mantenerse separadas:

$$\text{percepción} \rightarrow \text{decisión} \rightarrow \text{planificación} \rightarrow \text{control}. \tag{3.34}$$

Esta separación permite que cada componente disponga de una interfaz y unas restricciones propias. La literatura sobre arquitecturas robóticas señala precisamente la necesidad de organizar y coordinar componentes de percepción, planificación y control dentro de un mismo sistema [37].

Entre ambos planteamientos pueden construirse arquitecturas híbridas, en las que una política aprendida proporciona determinadas decisiones o contribuciones de acción mientras otras funciones permanecen asignadas a módulos geométricos, cinemáticos o de control. Esta organización resulta especialmente útil cuando la política se despliega sobre una plataforma diferente de aquella asociada a su representación original de acciones.

3.6.4. Adaptación y acotación de acciones

Una forma de combinar una predicción aprendida con una referencia obtenida mediante un modelo geométrico consiste en utilizar la contribución de la política como una corrección sobre un objetivo nominal. Si $\mathbf{p}^{\text{geom}}$ representa la referencia geométrica y $\Delta\mathbf{p}^{\text{VLA}}$ la modificación aportada por la política, puede escribirse

$$\mathbf{p}^{\text{exec}} = \mathbf{p}^{\text{geom}} + \Delta\mathbf{p}^{\text{VLA}}. \tag{3.35}$$

Este planteamiento guarda relación conceptual con los métodos de control residual, en los que una política aprendida modifica una solución proporcionada por un controlador o política previa [38]. Esta relación es únicamente conceptual: no implica que las políticas VLA consideradas hayan sido entrenadas originalmente como políticas residuales.

Cuando una salida aprendida se transfiere entre plataformas, diferencias de escala, normalización o sistema de referencia pueden producir correcciones de magnitud incompatible con el robot de destino. Una interfaz puede limitar esta contribución mediante una saturación componente a componente:

$$\Delta \mathbf{p}^{\mathrm{lim}} = \mathrm{clip}\left(\Delta \mathbf{p}^{\mathrm{VLA}}, -\mathbf{b}, \mathbf{b}\right), \tag{3.36}$$

donde $\mathbf{b}$ contiene los límites establecidos para cada componente. La referencia resultante queda entonces

$$\mathbf{p}^{\mathrm{exec}} = \mathbf{p}^{\mathrm{geom}} + \Delta \mathbf{p}^{\mathrm{lim}}. \tag{3.37}$$

La elección de las componentes utilizadas, su transformación entre sistemas de referencia, su escala y los límites aplicados dependen de la plataforma y de la interfaz de integración. Por ello, estos parámetros constituyen decisiones metodológicas y no propiedades generales de la política original.

En conjunto, la transferencia de una política VLA a un nuevo manipulador puede entenderse como la combinación de una representación aprendida con una interfaz que hace explícitas las diferencias de observación, acción y geometría entre plataformas. El Capítulo 4 describe cómo estos principios se materializan en la arquitectura experimental desarrollada para el UR5e.

4 Metodología

4.1. Introducción

Este TFG se plantea como un proyecto de desarrollo e integración tecnológica orientado a diseñar y validar experimentalmente un sistema robótico basado en modelos VLA, concretamente OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID, en una célula real de *pick-and-place* basada en un robot UR5e.

La integración se organizó mediante una arquitectura híbrida en la que los modelos aportan información semántica y de acción, mientras que la percepción métrica, la adaptación al robot, la cinemática inversa, el control del efector y las comprobaciones de seguridad se resuelven mediante módulos externos.

Cada episodio parte de una captura inicial de la Photoneo, a partir de la cual se obtienen los candidatos de la escena y sus posiciones tridimensionales. La instrucción y la observación se procesan con la configuración correspondiente, se realiza el *grounding* y la salida del modelo se adapta antes de incorporarla a una trayectoria compatible con el UR5e. Durante la ejecución no se realizan nuevas inferencias ni replanificación; una adquisición posterior se utiliza únicamente para verificar el resultado físico.

La metodología se estructuró siguiendo estas etapas para poder distinguir los fallos de interpretación, percepción, adaptación, planificación y ejecución, en lugar de reducir cada ensayo únicamente al resultado final de la tarea.

4.2. Configuraciones y checkpoints evaluados

Para la experimentación se utilizaron las configuraciones OpenVLA-7B, `pi0_droid` y `pi05_droid`. Aunque las tres corresponden a modelos VLA preentrenados, cada una mantiene su propio *checkpoint*, su interfaz de inferencia y sus convenciones de entrada y salida.

En este contexto, un *checkpoint* contiene los parámetros aprendidos por un modelo que se cargan posteriormente para realizar inferencia. Dependiendo de la implementación, también puede estar asociado a configuraciones y estadísticas necesarias para interpretar correctamente sus entradas y salidas. Por tanto, los tres sistemas evaluados no comparten un mismo *checkpoint* ni una representación común de la acción.

Para OpenVLA se utilizó OpenVLA-7B a partir de su implementación oficial [2], [14]. El modelo fue entrenado con demostraciones procedentes de diferentes conjuntos incluidos en Open X-Embodiment, por lo que su entrenamiento no está asociado exclusivamente a un único manipulador [10]. Sin embargo, el hecho de que los datos

procedan de múltiples plataformas no implica que su salida pueda aplicarse directamente al UR5e. La representación de la acción, las convenciones utilizadas durante el entrenamiento y la interfaz de ejecución siguen siendo diferentes de las empleadas en la célula desarrollada en este trabajo.

Para la familia π se emplearon las configuraciones `pi0_droid` y `pi05_droid` proporcionadas por OpenPI [5], [17]. Aunque π_0 y $\pi_{0.5}$ fueron desarrollados utilizando datos procedentes de diferentes robots y tareas [3], [4], las configuraciones concretas utilizadas en este TFG están asociadas al dominio DROID. Este conjunto de datos emplea como plataforma común un manipulador Franka Panda de siete grados de libertad, por lo que sus observaciones, estados y acciones responden a convenciones diferentes de las utilizadas por el UR5e [12].

Las diferencias de *embodiment* afectan tanto a la estructura cinemática como al efector final, la disposición sensorial y el significado de los canales de acción. El UR5e utilizado dispone de seis articulaciones y trabaja con una pinza OnRobot 3FG15 y una cámara Photoneo externa, mientras que la plataforma DROID emplea un Franka Panda de siete grados de libertad y su propia configuración de sensores y efector. En consecuencia, las acciones generadas por `pi0_droid` y `pi05_droid` no constituyen comandos directamente ejecutables por el UR5e.

Esta incompatibilidad afecta también a OpenVLA. Aunque su salida contiene una acción relativa del efector y resulta más próxima a una representación cartesiana, tampoco se envía directamente al controlador del robot. En las tres configuraciones se emplea una interfaz de transferencia que adapta la información aportada por el modelo antes de incorporarla a una trayectoria compatible con la geometría y las restricciones de la célula. El procedimiento concreto utilizado para realizar esta adaptación se describe más adelante.

Por tanto, la comparación realizada en este trabajo no enfrenta tres controladores conectados directamente al UR5e, sino tres configuraciones VLA preentrenadas integradas mediante una arquitectura común de percepción, adaptación, planificación y seguridad.

4.3. Plataforma y arquitectura del sistema

La célula está formada por un robot colaborativo UR5e [39], una pinza OnRobot 3FG15 [40] y una cámara 3D Photoneo MotionCam-3D-S instalada en configuración fija *eye-to-hand* [41]. La cinta transportadora permaneció detenida y se utilizó como superficie de trabajo.

La inferencia se ejecutó en una NVIDIA Jetson AGX Orin [42], mientras que un ordenador central coordinó la captura, el procesamiento tridimensional, la planificación, la comunicación con el robot y el registro de resultados. Esta separación respondió a los elevados requisitos de cómputo y memoria de los modelos VLA evaluados. En la implementación utilizada, la carga de los pesos y la inferencia se apoyaron en aceleración mediante GPU y demandaron una capacidad de memoria considerable. La Jetson AGX Orin se destinó por ello a alojar los modelos, mientras que el ordenador central se reservó para los procesos de percepción, planificación y control [42]. Por este motivo, el ordenador central no ejecutó directamente las políticas, sino que se

comunicó con servicios de inferencia persistentes desplegados en la Orin. Esta organización es coherente con las interfaces remotas ofrecidas por OpenVLA y OpenPI, diseñadas para mantener separado el entorno de la política respecto al sistema encargado del control robótico [14], [18]. Los equipos se interconectaron mediante un conmutador D-Link DGS-105.

La comunicación con el UR5e y el envío de las trayectorias articulares se realizaron mediante ROS 2 y el controlador oficial `ur_robot_driver` [43], [44].

La pinza 3FG15 se controló desde ROS 2 mediante una versión local del paquete `onrobot_driver`, conectada por TCP a la unidad de control OnRobot instalada en la célula. El ejecutor envió la anchura de los dedos al controlador `finger_width_controller` publicando mensajes `std_msgs/msg/Float64MultiArray` en el tópico `/onrobot/finger_width_controller/commands`. La documentación oficial de OnRobot describe la pinza 3FG15, su compatibilidad con robots Universal Robots y la configuración de la Compute Box [40], [45]. La conexión mediante Modbus y la organización concreta mediante tópico y controlador corresponden a la integración local desarrollada para este trabajo.

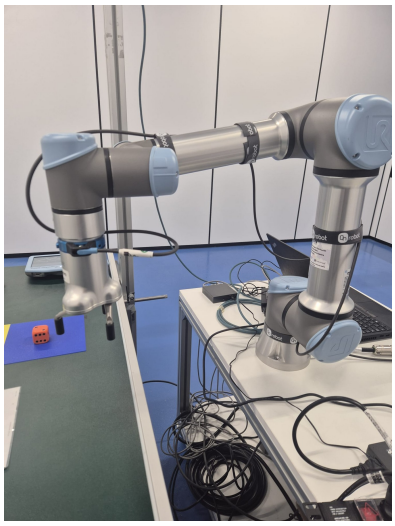


FIGURA 4.1: Vista general de la célula experimental.

El flujo general de cada episodio se resume como:

$$\begin{aligned} &\text{instrucción y captura} \rightarrow \text{percepción 3D y candidatos} \\ &\rightarrow \text{inferencia y } \textit{grounding} \rightarrow \text{adaptación y plan} \\ &\rightarrow \text{ejecución y verificación.} \end{aligned} \tag{4.1}$$

4.3.1. Entorno software y dependencias

El sistema se desarrolló principalmente en Python y se organizó en módulos para la adquisición, la percepción, la inferencia, el *grounding*, la adaptación de acciones, la planificación, la ejecución y el registro. El procesamiento numérico y geométrico se realizó con NumPy [46], mientras que OpenCV se utilizó para generar máscaras, analizar componentes conectados y extraer las regiones candidatas [47].

El modelo cinemático del UR5e se representó en MuJoCo mediante una cadena de cuerpos y articulaciones a la que se incorporó el TCP de la pinza OnRobot 3FG15. Para cada configuración articular, las funciones cinemáticas de MuJoCo permitieron actualizar la pose cartesiana del TCP y obtener sus jacobianos traslacional y rotacional. La API proporciona específicamente funciones para el cálculo del jacobiano de un *site*, utilizado como referencia del efector final [48], [49]. Sobre estas magnitudes se implementó en el código de este trabajo un procedimiento iterativo de cinemática inversa para determinar las configuraciones articulares asociadas a los puntos de paso.

OpenVLA-7B se desplegó a partir de su implementación oficial, basada en PyTorch y en la biblioteca Transformers de Hugging Face [14], [50], [51]. Las políticas π_0-DROID y $\pi_{0.5}$-DROID se ejecutaron mediante OpenPI, empleando la revisión oficial identificada por el *commit* `c23745b5ad24`, cuya versión completa y repositorio se documentan en [17].

El entorno de inferencia empleado utiliza JAX y Flax [52], [53] y se mantuvo separado del entorno encargado de controlar el robot mediante la interfaz remota de OpenPI [18].

Las configuraciones locales empleadas fueron `pi0_droid` y `pi05_droid`. Sus horizontes de acción, transformaciones de entrada y salida y estadísticas de normalización se comprobaron sobre la versión exacta del repositorio utilizada [5], [6]. Sobre estas herramientas se desarrolló la lógica específica de este trabajo.

4.4. Calibración y percepción tridimensional

4.4.1. Calibraciones geométricas

El TCP se obtuvo haciendo coincidir una referencia física fija en la cinta, definida mediante unas bridas plásticas, con el mismo punto de la pinza desde distintas orientaciones del efector. Este procedimiento sigue el método de enseñanza de la posición del TCP proporcionado por PolyScope, en el que un mismo punto fijo del espacio se alcanza desde diferentes orientaciones de la herramienta para estimar automáticamente la posición del TCP respecto a la brida del robot [54]. A partir de las poses registradas, el asistente de PolyScope estimó el desplazamiento del punto de trabajo respecto a la brida colocada en la garra del UR5e.

La transformación extrínseca entre el sistema de coordenadas de la Photoneo y la base del UR5e se estimó mediante varias correspondencias tridimensionales, en las que los puntos expresados en la base del robot se obtuvieron mediante contacto controlado, mientras que sus correspondencias se identificaron en la nube de puntos adquirida por la cámara.

El montaje incluyó un tablero con marcadores ArUco como apoyo visual [55]. No obstante, la transformación no se obtuvo mediante detección automática de los marcadores, sino mediante las correspondencias tridimensionales seleccionadas.

Los dos conjuntos de puntos se alinearon mediante la herramienta software CloudCompare, a partir de los pares de puntos equivalentes [56]. Matemáticamente, este procedimiento corresponde al problema de registro rígido por mínimos cuadrados

descrito en la Ecuación 3.30 del Capítulo 3. La matriz obtenida y utilizada durante los experimentos fue:

$$ {}^{B}\mathbf{T}_{C} = \begin{bmatrix} 0{,}041 & -0{,}998 & -0{,}056 & -219{,}488 \\ -0{,}999 & -0{,}041 & 0{,}000 & -952{,}756 \\ -0{,}002 & 0{,}056 & -0{,}998 & 866{,}163 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{4.2} $$

donde la traslación está expresada en milímetros. Para un punto homogéneo ${}^{C}\mathbf{p}$ expresado en el sistema de la cámara, su posición en la base se obtiene mediante:

$$ {}^{B}\mathbf{p} = {}^{B}\mathbf{T}_{C}\,{}^{C}\mathbf{p}. \tag{4.3} $$

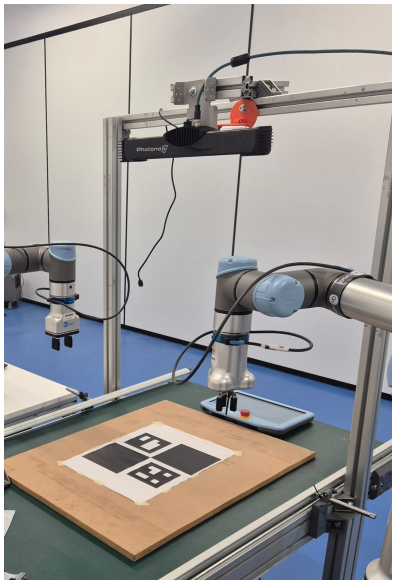


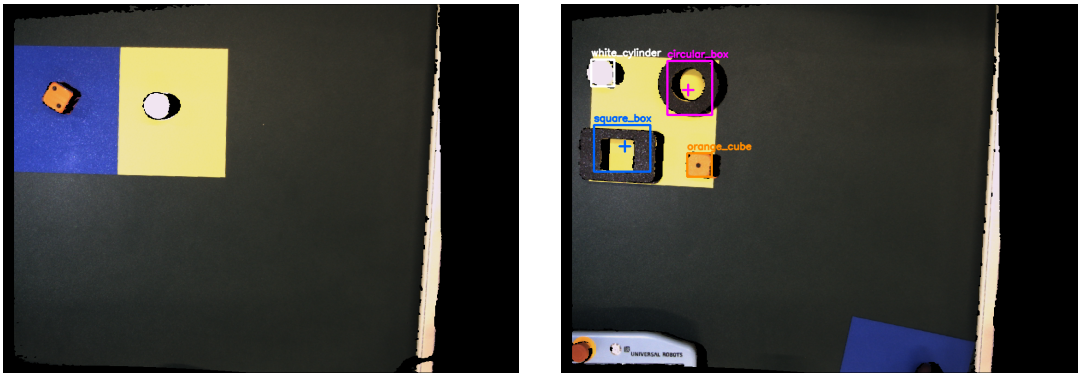
FIGURA 4.2: Montaje utilizado para la calibración entre la Photoneo y la base del UR5e.

4.4.2. Obtención de candidatos y posiciones 3D

Cada episodio comienza con una nueva adquisición de la escena mediante PhoXi Control. La captura proporciona la información visual y la nube de puntos correspondientes al mismo estado físico antes del movimiento [41].

A partir de la imagen se extraen candidatos de objeto y destino mediante color, forma, conectividad y restricciones geométricas. Las coordenadas se estiman con estadísticos robustos sobre los puntos tridimensionales válidos de cada región y posteriormente se transforman al sistema de la base mediante la Ecuación 4.2.

En la tarea de taxonomía por geometría, el procesamiento de los recipientes distingue entre la apertura interior y el borde. Esta separación permite situar el punto de deposición en el interior del recipiente y evita utilizar como destino el centro de la estructura exterior completa.



(a) Captura RGB de objeto–zona (b) Candidatos de clasificación por forma

FIGURA 4.3: Ejemplos de la información visual utilizada por el sistema y de los candidatos detectados.

4.5. *Grounding* e integración de los modelos

En este trabajo, el *grounding* corresponde a la selección del objeto y del destino físico a los que hace referencia la instrucción. El módulo de percepción genera previamente un conjunto reducido de candidatos, cada uno asociado a una región de la imagen y a una posición tridimensional obtenida mediante la Photoneo. Esta organización mantiene separadas la selección semántica de las entidades y la localización métrica utilizada posteriormente para construir la trayectoria.

La instrucción se normaliza mediante un *parser* que reconoce formulaciones equivalentes, conserva las relaciones espaciales y detecta referencias ambiguas o incompatibles con la escena. El *parser* se utiliza como normalizador, auditor y mecanismo de bloqueo. Las incompatibilidades se registran como fallos de la selección producida por el modelo y no se contabilizan como aciertos aunque el sistema impida su ejecución física.

Los modelos evaluados procesan la imagen y la instrucción, pero presentan espacios de acción distintos. En OpenVLA, la acción de siete componentes contiene tres componentes traslacionales del efector, tres asociadas a su orientación y una componente de pinza [2]. En las configuraciones DROID empleadas mediante OpenPI, los ocho canales corresponden a siete acciones de velocidad articular del manipulador Franka y una acción de pinza [6]. Las configuraciones utilizadas producen bloques de 10 acciones para `pi0_droid` y de 15 para `pi05_droid` [5], [18].

TABLA 4.1: Formas de salida de las configuraciones evaluadas.

Configuración	**Forma local**	**Información disponible**
OpenVLA	$(1, 7)$	Una única acción por inferencia.
π_0-DROID	$(10, 8)$	Secuencia temporal de diez acciones.
$\pi_{0.5}$-DROID	$(15, 8)$	Secuencia temporal de quince acciones.

En las políticas de la familia π, cada hipótesis objeto–destino se evaluó mediante perturbaciones controladas de la instrucción y de la evidencia visual. Se calcularon cuatro medidas de sensibilidad sobre la salida nativa del modelo: dos asociadas al

objeto y dos al destino, diferenciando en ambos casos la respuesta al texto y la suficiencia de la evidencia visual. Estas magnitudes cuantifican cambios relativos del bloque de acciones y no representan distancias físicas del UR5e.

En cada ronda, las cuatro medidas se dividieron por la mediana de sus valores positivos entre las hipótesis competidoras y se combinaron con pesos 0,40, 0,20, 0,25 y 0,15, respectivamente. La selección final minimizó la mediana de este coste normalizado a lo largo de las rondas; los votos individuales se conservaron únicamente como diagnóstico. Una selección incompatible con una instrucción inequívoca provocaba el bloqueo de la ejecución y se registraba como un fallo del modelo.

La única acción proporcionada por OpenVLA ofrecía menos evidencia temporal para obtener una selección explícita entre varios candidatos. Por este motivo se incorporó a su flujo un adaptador externo basado en SigLIP [7]. El servicio recibió los recortes de los candidatos con identificadores neutrales y calculó su compatibilidad con descripciones textuales del objeto y del destino. De esta forma, la correspondencia entre cada región y cada clase se resolvió mediante la evidencia visual, sin enviar al servicio una asociación previa entre identificadores y entidades.

Aunque OpenVLA utiliza internamente representaciones de SigLIP y DINOv2 [2], estas forman parte del procesamiento que conduce a la acción y no se exponen como una selección explícita de los candidatos extraídos de la captura. El adaptador externo no sustituyó el codificador de OpenVLA, no actuó como detector y no modificó su acción nativa; únicamente proporcionó la selección explícita del objeto y del destino. El efecto del adaptador se estudió mediante condiciones con y sin SigLIP.

4.6. Adaptación de las acciones al UR5e

Las salidas devueltas por los modelos se almacenaron antes de aplicar cualquier transformación propia de la interfaz de transferencia.

En OpenVLA, las seis primeras componentes de la acción se normalizaron mediante una calibración empírica previa del adaptador. Para cada canal se calculó la mediana de las acciones observadas y un semirrango robusto a partir de los percentiles de la distribución. Cada componente se centró respecto a su mediana, se dividió por dicho semirrango y se limitó al intervalo $[-1, 1]$. Esta normalización pertenece a la interfaz desarrollada en este trabajo y no debe confundirse con la normalización interna utilizada durante el entrenamiento de OpenVLA.

De las componentes normalizadas se utilizaron únicamente las tres traslacionales. Estas se expresaron en un marco local construido a partir de la dirección entre el objeto y el destino y se escalaron mediante un presupuesto cartesiano acotado. Como OpenVLA proporciona una única acción por inferencia, el residual resultante se reutilizó en las fases correspondientes del plan, aplicando en cada caso los límites establecidos para restringir su influencia.

En π_0-DROID y $\pi_{0.5}$-DROID, las salidas nativas corresponden al espacio de acciones de la plataforma DROID y, por tanto, no representan directamente desplazamientos cartesianos del UR5e. En la interfaz desarrollada para este trabajo se utilizó de forma heurística la información contenida en los tres primeros canales de cada bloque como una señal de corrección para construir residuales traslacionales acotados. El bloque se dividió en tres segmentos, asociados al agarre, la elevación y la aproximación al

destino, y la contribución obtenida en cada segmento se escaló y limitó antes de combinarse con la referencia geométrica calculada mediante la Photoneo.

La posición cartesiana utilizada en cada fase se obtuvo combinando la geometría calculada mediante la Photoneo con un residual traslacional acotado construido por la interfaz a partir de la salida del modelo:

$$\mathbf{p}_k^{\text{exec}} = \mathbf{p}_k^{\text{geom}} + \text{clip}\left(\Delta\mathbf{p}_k^{\text{VLA}}, -\mathbf{b}_k, \mathbf{b}_k\right), \tag{4.4}$$

donde $\mathbf{p}_k^{\text{geom}}$ es el punto de referencia obtenido a partir de la percepción tridimensional, $\Delta\mathbf{p}_k^{\text{VLA}}$ es la corrección construida por la interfaz a partir de la salida del modelo y $\mathbf{b}_k$ limita su influencia en cada fase.

La combinación de una referencia nominal con una corrección aprendida es conceptualmente afín a los enfoques de control residual, que superponen una contribución aprendida a una solución obtenida mediante control convencional [38]. En este trabajo, sin embargo, la corrección residual no procede de una política entrenada específicamente mediante aprendizaje por refuerzo, sino de la salida de un VLA preentrenado y se limita de forma determinista para cada fase.

La capa de adaptación evita ejecutar directamente acciones definidas para otros *embodiments*. La necesidad de comprobar explícitamente la semántica, la escala y el sistema de coordenadas de las acciones también ha sido identificada en despliegues recientes de modelos VLA sobre robots UR5 [22]. La normalización, la segmentación temporal y los límites utilizados en este trabajo pertenecen, no obstante, a la adaptación desarrollada para la célula evaluada.

Solo los residuales traslacionales resultantes de esta adaptación modificaron la trayectoria. La orientación del efector se mantuvo cenital y la secuencia principal de apertura y cierre de la pinza se gestionó externamente.

4.7. Trayectoria y seguridad

La trayectoria se construye mediante seis fases: preagarre, agarre, elevación, aproximación al destino, deposición y retirada. Para cada fase se resuelve la cinemática inversa de forma secuencial, utilizando la solución anterior como referencia para reducir cambios innecesarios de rama.

La cinemática inversa se resolvió mediante un procedimiento iterativo de mínimos cuadrados amortiguados (DLS), utilizando los jacobianos traslacional y rotacional del TCP calculados con MuJoCo. En cada iteración se corrigió el error entre la pose actual y la pose objetivo, limitando además el incremento articular y respetando los límites de las articulaciones. Para cada nuevo punto de paso se utilizó como inicialización la solución obtenida en el punto anterior, favoreciendo así la continuidad de la trayectoria [34].

Antes del movimiento se comprueban el estado inicial del robot, la validez de las coordenadas, la existencia de soluciones articulares, la continuidad de la trayectoria y el cumplimiento del espacio de trabajo. Las trayectorias articulares validadas se envían mediante la interfaz de control proporcionada por el controlador ROS 2 de Universal Robots [44].

Las anchuras de apertura y cierre se definieron como parámetros para adaptarlas a los objetos empleados en cada tarea. La ejecución se bloquea ante instrucciones ambiguas, selecciones incompatibles con la escena, valores no finitos, ausencia de solución cinemática o fallos de comunicación. Todas las pruebas físicas se realizaron bajo supervisión directa del operador.

4.8. Tareas implementadas

Las dos tareas se seleccionaron para introducir dificultades semánticas diferentes manteniendo una manipulación física sencilla y repetible. La primera permite estudiar la selección explícita de objeto y destino junto con relaciones espaciales, mientras que la segunda exige asociar el objeto con un destino definido por una propiedad geométrica. De este modo pueden aislarse problemas de grounding y razonamiento sin incorporar todavía las dificultades adicionales de operaciones de contacto o ensamblaje más complejas.

La primera tarea consiste en trasladar un cubo naranja o un cilindro blanco a una zona azul o amarilla. Parte de las instrucciones añade relaciones espaciales respecto a la zona, por lo que se evalúan tanto la selección del objeto y del destino como el cumplimiento de la colocación solicitada. Se ejecutaron diez casos con cada modelo, para un total de 30 episodios físicos.

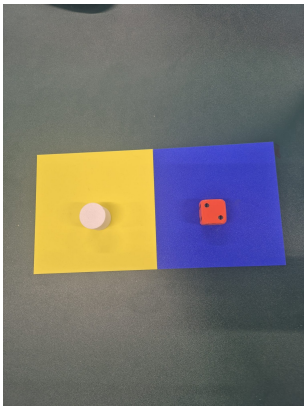
La segunda tarea evalúa la clasificación por forma: el cubo debe depositarse en el recipiente cuadrado y el cilindro en el circular. Se utilizaron tres disposiciones de la escena, cuatro instrucciones por disposición y los tres modelos, dando lugar a 36 episodios físicos.

Dentro de cada caso, los tres modelos recibieron la misma instrucción literal y se evaluaron sobre una disposición física equivalente. Se mantuvieron constantes el *home* operativo, las tolerancias de movimiento, la velocidad, los criterios de seguridad y los criterios utilizados para determinar el éxito de la tarea. Cada episodio físico comenzó desde el estado inicial definido para la campaña y utilizó una nueva captura de la Photoneo, de forma que la planificación se realizó siempre a partir del estado real de la escena correspondiente a ese ensayo.

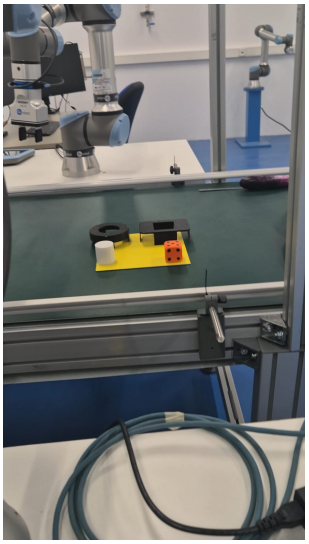
Para reducir la variabilidad asociada a la inferencia, las políticas de la familia π se ejecutaron con una semilla de inferencia fija. Asimismo, π_0-DROID y $\pi_{0.5}$-DROID se ejecutaron de forma separada, evitando mantener simultáneamente activos ambos servicios de inferencia. Estas condiciones se conservaron durante las campañas utilizadas para la comparación entre modelos.

Los ensayos físicos y los *replays* se trataron como modalidades experimentales diferentes. En los episodios físicos se adquirió nuevamente la escena, se generó y validó el plan y, cuando todas las comprobaciones fueron satisfactorias, se permitió el movimiento del robot. En los *replays*, por el contrario, se reutilizaron la observación y la información de escena previamente almacenadas y se deshabilitó el movimiento físico. Esto permitió variar la formulación lingüística o la configuración evaluada manteniendo fija la observación visual utilizada como entrada.

La campaña completa comprende 66 episodios físicos. Cada episodio incluye la preparación y captura de la escena, la inferencia y el *grounding*, la validación del plan, la ejecución y la verificación del resultado.



(a) Tarea de objeto–zona



(b) Tarea de clasificación por forma

FIGURA 4.4: Escenas representativas de las dos tareas evaluadas.

Además, se realizaron 186 replays sobre capturas guardadas para evaluar la robustez lingüística de los modelos: 150 correspondientes a la tarea objeto–zona y 36 a la tarea de clasificación por forma. Estos ensayos permitieron estudiar reformulaciones, sinónimos, ambigüedad y referencias a entidades ausentes manteniendo fija la observación visual utilizada como entrada.

De forma adicional, se realizaron 22 replays específicos para analizar la ablación del adaptador externo SigLIP en OpenVLA: 10 correspondientes a la Tarea 1 y 12 a la Tarea 2.

Este protocolo se relaciona con evaluaciones recientes de robustez lingüística en modelos VLA, en las que se modifican de forma controlada las expresiones de acción y las referencias a objetos para analizar la estabilidad de la política ante paráfrasis [24]. De forma complementaria, ICBench estudia el acoplamiento entre lenguaje y acción manteniendo fija la observación visual e introduciendo instrucciones incompatibles con la escena [25].

4.9. Métricas y análisis

La evaluación se organizó de acuerdo con las principales etapas del flujo, evitando reducir cada episodio a una única tasa binaria de éxito. Para el *grounding* se registraron la selección correcta del objeto y del destino, el rechazo de instrucciones ambiguas o no ejecutables y, en la tarea objeto–zona, el cumplimiento de la relación espacial solicitada.

En la parte física se comprobaron la validez del plan, la finalización de la ejecución, la deposición verificada y el cumplimiento estricto de la instrucción. Esta separación evita identificar automáticamente la terminación del controlador con el éxito de la tarea y es coherente con trabajos de evaluación real que incorporan una detección explícita del resultado [26].

Cuando la evidencia geométrica disponible lo permitió, también se conservaron magnitudes relacionadas con la precisión del agarre y de la deposición.

Los fallos se agruparon según su origen en lenguaje y *grounding*, percepción tridimensional, acción y adaptación, planificación y seguridad, o ejecución y verificación.

Debido al reducido número de episodios por configuración, las tasas de éxito se interpretaron principalmente de forma descriptiva y se acompañaron de conteos absolutos e intervalos de confianza del 95 % de Wilson, adecuados para proporciones binomiales con muestras pequeñas [57], [58]. Cuando se dispuso de resultados binarios emparejados para los mismos casos, se utilizó además la prueba exacta de McNemar, basada en los pares discordantes [58]. Se adoptó un nivel de significación de $\alpha = 0{,}05$. Estos contrastes se emplearon como apoyo a la interpretación de los resultados y no para establecer un ranking general entre arquitecturas.

Para mantener la trazabilidad de los experimentos, cada registro se asoció con la tarea, el caso, el modelo y la instrucción utilizados. Según el tipo de ensayo, se conservaron además la captura empleada como observación, la información tridimensional de la escena, la selección producida durante el *grounding*, la salida nativa del VLA antes de su adaptación, las correcciones finalmente incorporadas a la trayectoria y el resultado de las comprobaciones realizadas antes de permitir la ejecución.

En los episodios físicos, esta información se complementó con la evidencia obtenida tras el movimiento para distinguir entre la finalización del controlador y el resultado físico de la tarea. En los *replays*, la conservación de la observación original permitió volver a evaluar la misma escena modificando únicamente las condiciones previstas en cada ensayo.

Para analizar específicamente la contribución de los modelos se conservaron sus salidas nativas antes de la adaptación, las correcciones traslacionales aplicadas a la trayectoria y la respuesta de la acción ante modificaciones de la imagen o de la instrucción. También se registraron la latencia de inferencia y el número de consultas realizadas, con el fin de comparar la rapidez de respuesta de las distintas configuraciones sin confundirla con la duración total del ciclo robótico.

Cabe mencionar que las secuencias producidas por π_0 y $\pi_{0.5}$ se analizaron mediante A-PI, A-VI y A-AI, basadas en las diferencias sucesivas propuestas por Valle *et al.* para caracterizar la variación interna de acciones VLA [59]. El cálculo siguió la implementación pública de referencia asociada al artículo [60]: la segunda diferencia se divide por 2 y la tercera por 4 antes de realizar el promedio. Estos factores igualan la cota inducida por los operadores de diferencias de primer, segundo y tercer orden, cuyos coeficientes tienen normas ℓ_1 iguales a 2, 4 y 8, respectivamente. De este modo, tras aplicar los factores 1, $1/2$ y $1/4$, las tres medidas mantienen una escala de variación comparable para canales de acción acotados.

Estas métricas se seleccionaron para analizar si las diferencias de rendimiento observadas entre π_0-DROID y $\pi_{0.5}$-DROID podían estar relacionadas con diferencias en la dinámica interna de sus bloques de acciones. Dado que ambas configuraciones producen secuencias temporales en una única inferencia, resulta posible estudiar no solo la magnitud de sus acciones, sino también cómo varían a lo largo del bloque. A-PI cuantifica los cambios entre acciones consecutivas, A-VI caracteriza la variación de esos cambios y A-AI la variación de orden superior. Sus denominaciones no

implican que las magnitudes calculadas correspondan directamente a posición, velocidad o aceleración física del UR5e, sino que se aplican sobre los canales nativos de acción de cada política y se utilizan aquí como descriptores de la regularidad interna del bloque.

Para este trabajo se emplearon las siguientes agregaciones escalares sobre un bloque $\mathbf{A} = \{\mathbf{a}_1, \ldots, \mathbf{a}_H\}$ de dimensión D:

$$\text{A-PI} = \frac{1}{D(H-1)} \sum_{t=1}^{H-1} \sum_{d=1}^{D} \left| a_{t+1,d} - a_{t,d} \right|, \tag{4.5}$$

$$\text{A-VI} = \frac{1}{2D(H-2)} \sum_{t=1}^{H-2} \sum_{d=1}^{D} \left| a_{t+2,d} - 2a_{t+1,d} + a_{t,d} \right|, \tag{4.6}$$

$$\text{A-AI} = \frac{1}{4D(H-3)} \sum_{t=1}^{H-3} \sum_{d=1}^{D} \left| a_{t+3,d} - 3a_{t+2,d} + 3a_{t+1,d} - a_{t,d} \right|. \tag{4.7}$$

Estas métricas cuantifican, respectivamente, la variación de primer, segundo y tercer orden dentro del bloque. Se calcularon sobre los ocho canales devueltos por la interfaz DROID y, de forma separada, sobre los tres primeros canales utilizados por el adaptador desarrollado en este TFG. Como medida complementaria se calculó la distancia euclídea media entre acciones consecutivas en este subespacio de tres canales, normalizada por el número de transiciones del bloque.

OpenPI aplica las transformaciones de salida y la desnormalización asociadas a cada checkpoint antes de devolver las acciones. Tanto `pi0_droid` como `pi05_droid` utilizan estadísticas del dominio DROID, cargadas desde el archivo `norm_stats.json` asociado al checkpoint correspondiente. No se estableció la equivalencia numérica de dichas estadísticas entre ambos checkpoints; por tanto, sus salidas desnormalizadas no se trataron como una escala numérica común. En consecuencia, las métricas A-PI, A-VI y A-AI no se utilizaron para establecer comparaciones cuantitativas directas entre ambas políticas. Su análisis se centró en las diferencias entre episodios con y sin éxito dentro de cada configuración, donde la normalización permanece fija.

5 Resultados y discusión

5.1. Introducción

A la luz de las dos tareas experimentales definidas en el Capítulo 4, los resultados se analizan atendiendo a la ejecución física, la robustez del *grounding* ante variaciones lingüísticas y el comportamiento interno de las acciones. La campaña comprendió 66 episodios físicos, 30 correspondientes a la Tarea 1 y 36 a la Tarea 2, además de 186 *replays* de robustez lingüística y 22 *replays* específicos para la ablación de OpenVLA sin SigLIP.

Se distingue entre éxito operacional, correspondiente a la finalización física de la operación prevista, y cumplimiento estricto, que exige además satisfacer las condiciones semánticas y geométricas de la instrucción. Dado el reducido número de episodios físicos por configuración, las tasas obtenidas se interpretan principalmente de forma descriptiva y se acompañan de los conteos absolutos. Las métricas temporales A-PI, A-VI y A-AI se aplican únicamente a las políticas de la familia π, puesto que OpenVLA produce una salida nativa de un único paso; las magnitudes derivadas de las acciones se expresan en el espacio de salida devuelto por cada política.

5.2. Tarea 1: colocación de objetos sobre zonas

5.2.1. Resultados físicos

La primera tarea evaluó la selección del objeto y del destino, la ejecución del *pick-and-place* y, cuando correspondía, el cumplimiento de relaciones espaciales como izquierda, derecha, parte delantera o parte trasera de la zona. Cada configuración se evaluó en diez casos físicos.

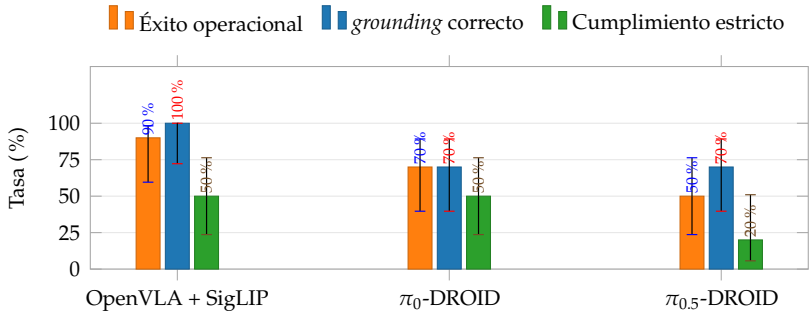


FIGURA 5.1: Resultados físicos por configuración en la tarea objeto–zona.

El sistema basado en OpenVLA con el adaptador externo SigLIP obtuvo el mayor éxito operacional, con 9 de 10 episodios, mientras que π_0-DROID alcanzó 7 de 10

y $\pi_{0.5}$-DROID 5 de 10. Al exigir cumplimiento estricto, OpenVLA + SigLIP y π_0-DROID obtuvieron 5 de 10 casos, frente a 2 de 10 de $\pi_{0.5}$-DROID.

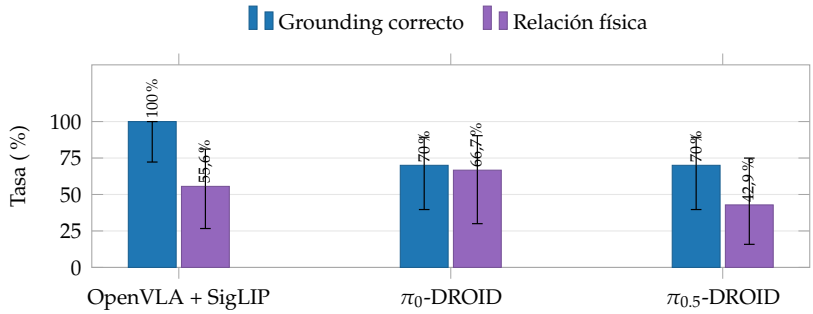


FIGURA 5.2: Grounding correcto y relación física en la tarea objeto–zona.

De los 30 episodios, 10 fallaron específicamente por no satisfacer la relación espacial solicitada, 6 por una selección incorrecta del objeto o del destino y 2 durante la verificación posterior a la colocación. Los 12 restantes cumplieron todos los criterios. Esta distribución confirma que la finalización del movimiento no equivale necesariamente al cumplimiento de la instrucción completa. Por ello, en este trabajo se mantuvieron separadas las etapas de *grounding*, ejecución y verificación del resultado, de forma coherente con evaluaciones reales que separan la ejecución de la detección explícita del resultado [26].

Nueve de los diez casos físicos incluían una relación espacial explícita; el restante evaluaba únicamente la pareja objeto–destino. La tasa relacional se calculó solo sobre los casos con relación explícita y *grounding* correcto: 5/9 para OpenVLA + SigLIP, 4/6 para π_0-DROID y 3/7 para $\pi_{0.5}$-DROID. Los denominadores difieren porque un fallo previo de selección impide evaluar posteriormente la relación espacial.

Dos situaciones observadas durante la campaña ilustran esta diferencia. En una de las tareas se solicitó colocar el cilindro blanco a la izquierda de la zona amarilla. OpenVLA produjo una preferencia hacia el frente, $\pi_{0.5}$ hacia la izquierda y π_0 hacia la zona posterior izquierda. Aunque los tres ciclos físicos se completaron, las preferencias espaciales producidas por los modelos no fueron equivalentes.

En otra tarea se solicitó colocar el cilindro blanco en el centro de la zona azul. Las tres configuraciones seleccionaron correctamente el objeto y el destino y completaron la colocación física sobre la zona. Sin embargo, las preferencias espaciales obtenidas de sus salidas no coincidieron literalmente con la petición: OpenVLA produjo derecha, $\pi_{0.5}$ izquierda y π_0 una preferencia posterior izquierda. Este resultado muestra la importancia de diferenciar la preferencia espacial inferida a partir de la salida del modelo del resultado geométrico finalmente alcanzado durante la ejecución.

5.2.2. Ablación de OpenVLA sin SigLIP

La contribución del decisor semántico externo se analizó comparando la configuración principal de OpenVLA + SigLIP con la evidencia consolidada de OpenVLA sin SigLIP. En los diez episodios físicos de la Tarea 1, OpenVLA + SigLIP seleccionó correctamente la pareja objeto–destino en 10/10 casos (100 %). La consolidación de los diez *replays* correspondientes sin SigLIP produjo 3/10 selecciones correctas (30 %).

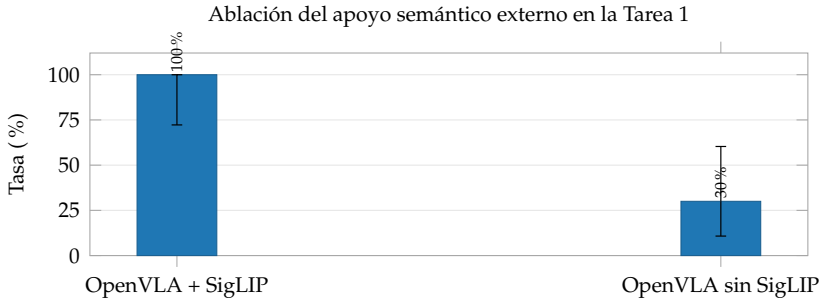


FIGURA 5.3: Selección correcta de la pareja objeto–destino con OpenVLA + SigLIP y en la consolidación de OpenVLA sin SigLIP en la Tarea 1.

5.2.3. Robustez lingüística

La robustez frente a variaciones de la instrucción se estudió mediante 150 *replays*, 50 por configuración. Para cada configuración se incluyeron 42 instrucciones positivas, en las que existía una pareja objeto–destino válida, y 8 instrucciones negativas o ambiguas que debían ser rechazadas. Las variantes incluyeron instrucciones canónicas, alias de objeto y destino, reformulaciones del comando, variaciones de la relación espacial y controles ambiguos o no soportados.

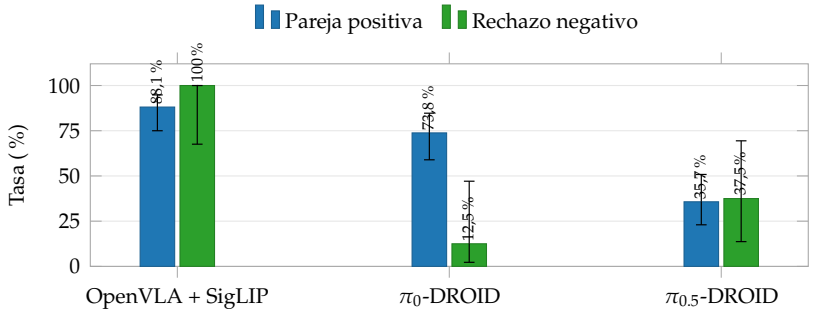


FIGURA 5.4: Robustez lingüística por configuración en los 150 *replays* de la tarea objeto–zona

OpenVLA + SigLIP resolvió correctamente 37/42 parejas positivas y rechazó 8/8 instrucciones negativas. π_0-DROID obtuvo 31/42 y 1/8, respectivamente, mientras que $\pi_{0.5}$-DROID obtuvo 15/42 y 3/8. OpenVLA + SigLIP fue, por tanto, la configuración más robusta tanto ante reformulaciones positivas como ante instrucciones que debían rechazarse. π_0 conservó una tasa elevada de parejas positivas, pero aceptó la mayoría de las negativas; $\pi_{0.5}$ mostró además una caída importante en la estabilidad de la pareja objeto–destino.

Se observa que los alias combinados redujeron el acierto al 16,7 %, frente al 62,5 % para los alias de objeto y al 70,4 % para los alias de objetivo, lo que indica que la robustez observada ante variaciones aisladas no se mantuvo al combinarlas.

5.2.4. Comportamiento de las acciones

Las métricas A-PI, A-VI y A-AI, definidas en la Sección 4.9, se aplicaron únicamente a π_0-DROID y $\pi_{0.5}$-DROID, ya que OpenVLA produce una única acción por inferencia. La Figura 5.7 resume su comportamiento sobre los tres primeros canales utilizados por el adaptador.

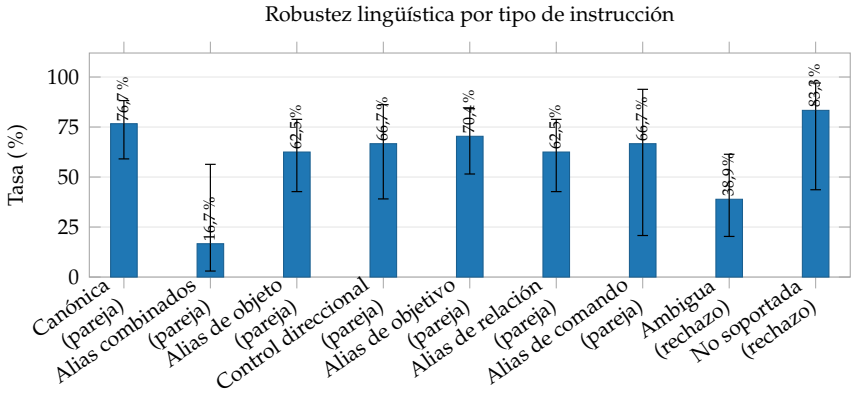


FIGURA 5.5: Tasa de acierto agregada por tipo de variación lingüística en la tarea objeto–zona.

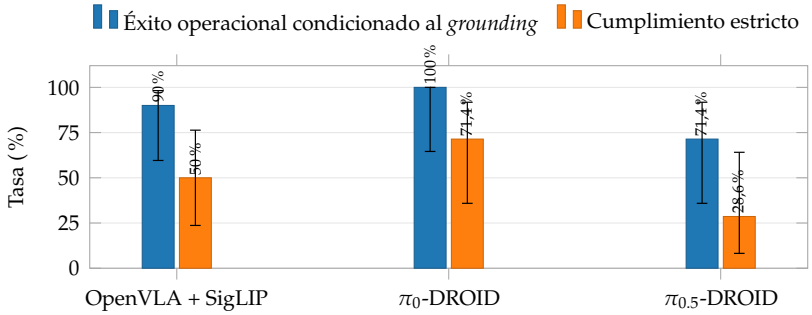


FIGURA 5.6: Éxito operacional y cumplimiento estricto condicionados a un *grounding* correcto en la tarea objeto–zona.

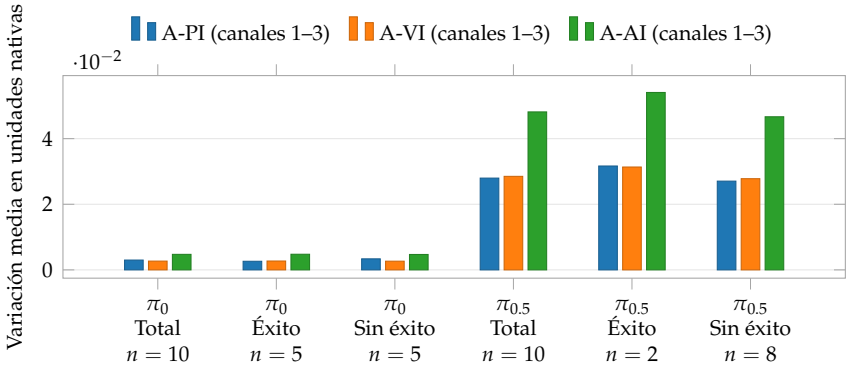


FIGURA 5.7: Métricas temporales internas de los bloques de acción en la tarea objeto–zona.

El análisis dentro de cada configuración muestra diferencias entre los episodios con y sin cumplimiento estricto. En π_0, los cinco episodios con cumplimiento estricto redujeron A-PI un 22,1 % respecto a los cinco sin éxito, mientras que A-VI y A-AI se mantuvieron prácticamente invariantes y la distancia por transición en esos tres canales disminuyó un 21,6 %. En $\pi_{0.5}$ no apareció una relación monotónica equivalente: los dos éxitos estrictos presentaron A-PI, A-VI y A-AI aproximadamente un 17 %, 13 % y 16 % superiores a los ocho episodios sin éxito. Por tanto, en esta tarea una menor variabilidad no actúa como predictor independiente del resultado.

5.3. Tarea 2: clasificación por forma

5.3.1. Resultados físicos

La segunda tarea exigió asociar un objeto con el recipiente compatible con su forma. Se utilizaron tres disposiciones físicas de la escena y cuatro instrucciones por disposición y modelo, dando lugar a 36 episodios físicos.

Las tres disposiciones modificaron deliberadamente la geometría de la escena sin alterar la tarea semántica. El *Layout 1*, correspondiente a las 12 primeras pruebas, utilizó la configuración inicial que consistió en los objetos a clasificar posicionados uno al lado del otro y enfrentados con las cajas en función de su forma. En el *Layout 2*, para los 12 casos siguientes, se intercambiaron las posiciones de las cajas. Por último, para el *Layout 3*, los últimos 12 casos, se mantuvieron los recipientes en la disposición del segundo *layout* y se recolocaron únicamente el cubo naranja y el cilindro blanco. De esta forma se introdujeron cambios espaciales controlados manteniendo constantes las instrucciones y el resto del protocolo.

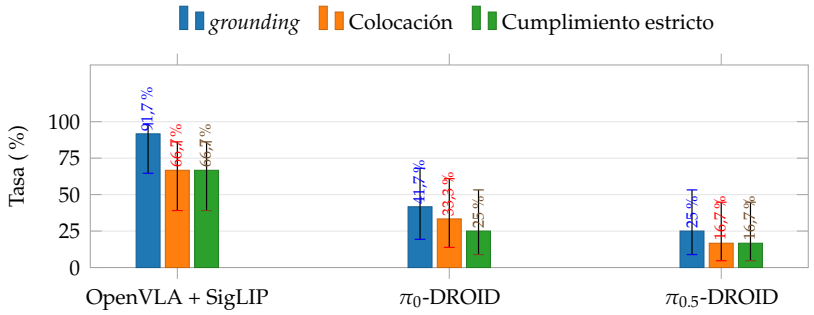


FIGURA 5.8: Resultados físicos por configuración en la tarea de clasificación por forma.

OpenVLA + SigLIP obtuvo 11/12 *groundings* correctos, 8/12 colocaciones correctas y 8/12 cumplimientos estrictos. π_0-DROID obtuvo 5/12, 4/12 y 3/12, mientras que $\pi_{0.5}$-DROID obtuvo 3/12, 2/12 y 2/12, respectivamente.

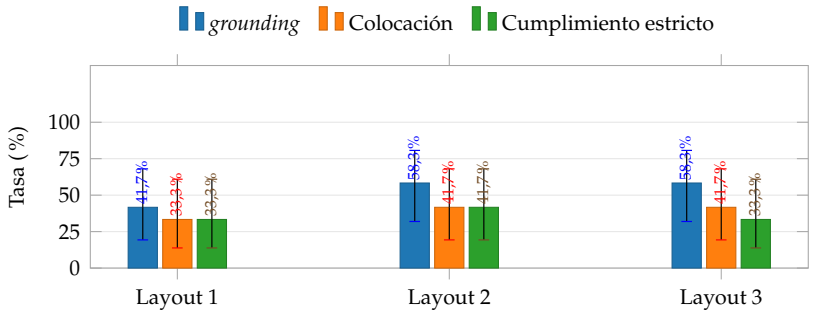


FIGURA 5.9: Resultados físicos por disposición de la escena en la tarea de clasificación por forma.

La limitación más notoria que se encontró fue el *grounding*: 17 de los 36 episodios fallaron antes de intentar la ejecución física debido a una selección incorrecta del objeto o del recipiente. Entre los 19 episodios que superaron esta etapa, 14 alcanzaron una colocación correcta y 13 cumplieron todos los criterios. Por tanto, la mayor pérdida de rendimiento se produjo antes del movimiento, y no durante la manipulación una vez seleccionada la pareja correcta.

El análisis por disposición muestra además que el resultado no dependió de una única geometría favorable. El *Layout 1* obtuvo 5/12 *groundings* correctos (41,7 %), 4/12 colocaciones (33,3 %) y 4/12 éxitos estrictos (33,3 %). Los *Layouts 2* y 3 alcanzaron ambos 7/12 *groundings* (58,3 %) y 5/12 colocaciones (41,7 %); el cumplimiento estricto fue 5/12 (41,7 %) en el segundo y 4/12 (33,3 %) en el tercero. El intercambio de cajas y la posterior recolocación de los objetos no eliminaron, por tanto, la dificultad de asociación semántica, aunque los dos últimos *layouts* mejoraron el *grounding* respecto al primero. Esta variación entre disposiciones justifica haber mantenido tres escenas físicas en lugar de extraer conclusiones de una única configuración.

5.3.2. Ablación de SigLIP

La contribución del adaptador visual se evaluó mediante una ablación de OpenVLA. La configuración física con SigLIP obtuvo 11 *groundings* correctos de 12 (91,7 %), mientras que los 12 *replays* sobre las mismas disposiciones sin el apoyo externo de SigLIP produjeron 3 aciertos de 12 (25,0 %).

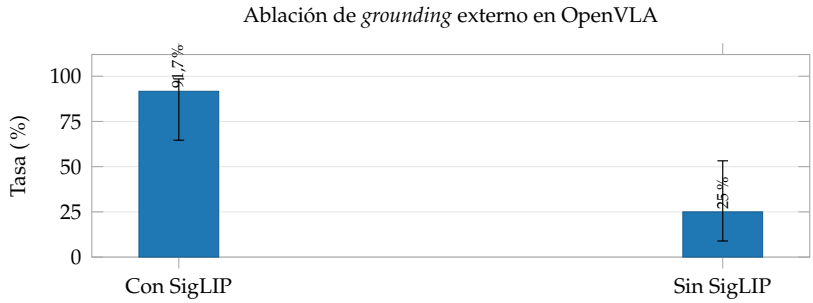


FIGURA 5.10: Comparación del *grounding* de OpenVLA con y sin el apoyo externo de SigLIP en la tarea de clasificación por forma.

El descenso afectó tanto a instrucciones implícitas como explícitas, ya que sin SigLIP se obtuvieron 2/6 aciertos en las formulaciones que exigían asociar la forma del objeto con el recipiente y 1/6 cuando el recipiente se nombraba explícitamente. Por ello, el fallo no puede atribuirse únicamente a la inferencia "cubo–cuadrado" o "cilindro–circular"; también aparecieron errores cuando la clase del recipiente estaba escrita en la instrucción. En esta configuración, SigLIP externo actuó como un apoyo determinante para la selección de candidatos físicos, sin modificar los parámetros de OpenVLA.

5.3.3. Robustez lingüística

La tarea de clasificación por forma se complementó con 36 *replays* lingüísticos, 12 por configuración, y se procedió a evaluar de la misma manera que en la tarea anterior.

OpenVLA + SigLIP obtuvo 7/8 instrucciones positivas correctas y rechazó 4/4 negativas. π_0-DROID obtuvo 4/8 y 1/4, y $\pi_{0.5}$-DROID 1/8 y 2/4. El patrón encontrado reproduce lo observado en la Tarea 1, y es que π_0 conserva mejor las asociaciones positivas que $\pi_{0.5}$, mientras que ambos modelos π rechazan con poca fiabilidad las instrucciones negativas.

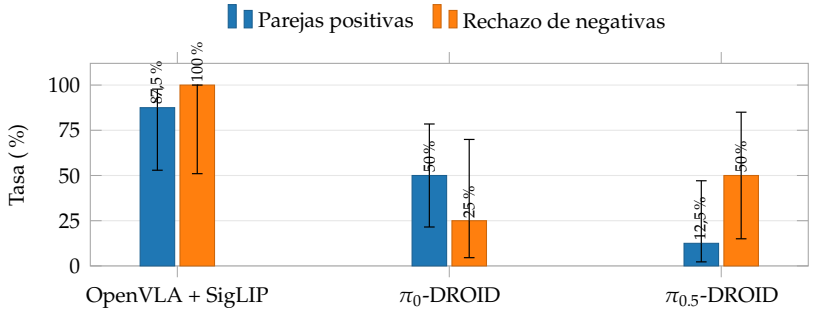


FIGURA 5.11: Robustez lingüística por configuración en la tarea de clasificación por forma.

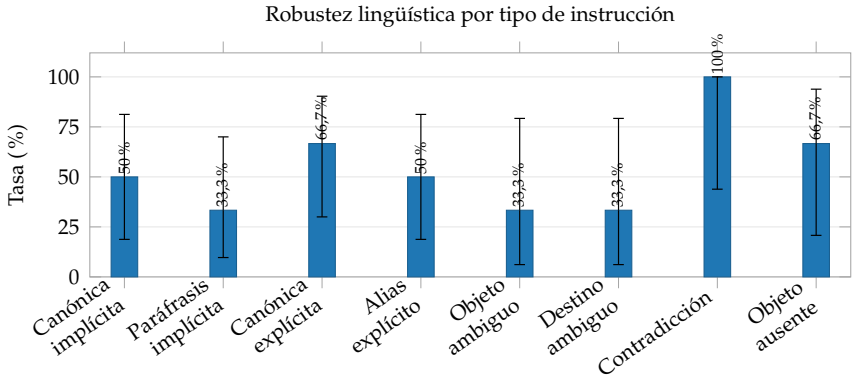


FIGURA 5.12: Tasa de acierto agregada por tipo de variación lingüística en la tarea de clasificación por forma.

5.3.4. Comportamiento de las acciones

Las métricas temporales se analizaron de nuevo dentro de cada configuración de la familia π, comparando los episodios con y sin cumplimiento estricto.

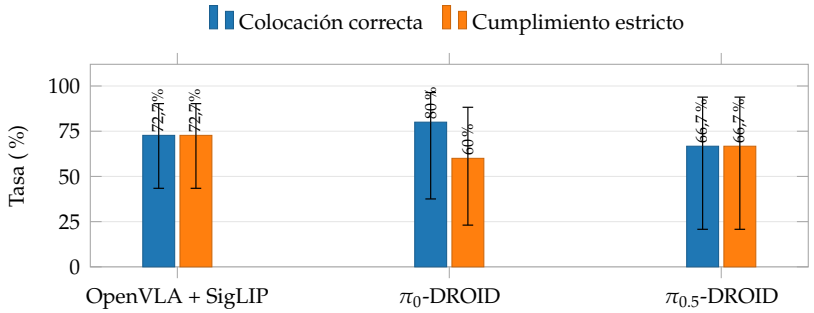


FIGURA 5.13: Resultado de la acción condicionado a un *grounding* correcto en la tarea de clasificación por forma.

A diferencia de la Tarea 1, al separar los episodios por resultado apareció en esta tarea un patrón consistente dentro de ambos modelos. En π_0, los tres éxitos estrictos presentaron reducciones del 27 %, en A-PI, del 16 %, en A-VI y del 19 % en A-AI respecto a los nueve casos sin éxito; la distancia por transición en esos tres canales disminuyó un 27 %. En $\pi_{0.5}$, los dos éxitos estrictos redujeron A-PI aproximadamente un 15 %, A-VI un 11 % y A-AI un 16 %, mientras que la distancia por transición en esos tres canales disminuyó aproximadamente un 13 %.

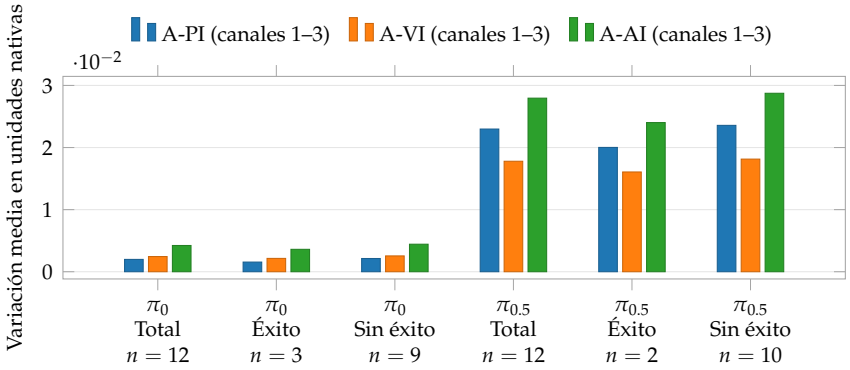


FIGURA 5.14: Métricas temporales internas de los bloques de acción en la tarea de clasificación por forma.

En ambos modelos, los subconjuntos con éxito estricto presentaron en esta tarea una menor variación interna que los casos sin éxito.

5.4. Discusión

Una vez obtenidos todos los resultados se puede afirmar: en la Tarea 1, el principal fallo fue la relación espacial, ya que la selección objeto–destino fue relativamente sólida, pero las referencias direccionales limitaron el cumplimiento estricto. Por otro lado, en la Tarea 2 el problema dominante fue el *grounding* objeto–recipiente, ya que solo 19 de los 36 episodios alcanzaron una selección correcta antes de la ejecución. Esta diferencia confirma la utilidad de evaluar por separado selección semántica, razonamiento espacial, acción y resultado físico.

Las ablaciones de SigLIP refuerzan esta lectura, pues OpenVLA + SigLIP alcanzó 10/10 parejas objeto–destino correctas en la Tarea 1 frente a 3/10 sin el decisor externo, y 11/12 frente a 3/12 en la Tarea 2. Dado que OpenVLA + SigLIP y las políticas π emplearon mecanismos de selección distintos, estas cifras caracterizan las configuraciones integradas y no únicamente la capacidad de *grounding* del VLA.

Los intervalos de confianza de Wilson reflejan además la incertidumbre asociada al reducido tamaño de las muestras. Esta fue especialmente amplia en las comparaciones físicas de la Tarea 1, mientras que las ablaciones de SigLIP y el *grounding* de la Tarea 2 mostraron separaciones más claras entre las proporciones estimadas.

Como las configuraciones se evaluaron sobre los mismos casos, se utilizó la prueba exacta de McNemar para comprobar si los desacuerdos entre dos configuraciones eran equilibrados o si una de ellas acertaba con mayor frecuencia precisamente en los casos en los que la otra fallaba. En la Tarea 1 ninguna comparación física entre configuraciones alcanzó significación estadística ($p > 0{,}05$). En la Tarea 2, OpenVLA + SigLIP presentó diferencias significativas frente a π_0-DROID en *grounding* ($p = 0{,}031$) y frente a $\pi_{0.5}$-DROID en *grounding* ($p = 0{,}008$), colocación ($p = 0{,}031$) y cumplimiento estricto ($p = 0{,}031$). La ablación de SigLIP también mostró diferencias significativas en ambas tareas ($p = 0{,}016$ en la Tarea 1 y $p = 0{,}008$ en la Tarea 2).

En los *replays* lingüísticos de la Tarea 1, OpenVLA + SigLIP superó a $\pi_{0.5}$-DROID en la selección correcta de las instrucciones positivas (37/42 frente a 15/42, $p < 0{,}001$), mientras que π_0-DROID también superó a $\pi_{0.5}$-DROID (31/42 frente a 15/42, $p < 0{,}001$). En las instrucciones negativas, OpenVLA + SigLIP rechazó correctamente 8/8

casos frente a 1/8 de π_0-DROID ($p = 0{,}016$). Las restantes comparaciones de la Tarea 1 se mantuvieron como resultados descriptivos.

En los *replays* lingüísticos de la Tarea 2, la comparación de las instrucciones positivas mostró una diferencia significativa entre OpenVLA + SigLIP y $\pi_{0.5}$-DROID ($p = 0{,}031$), mientras que las restantes comparaciones positivas y las correspondientes a instrucciones negativas no alcanzaron significación estadística.

En resumen, este comportamiento es coherente con la sensibilidad a paráfrasis y al desacoplamiento entre lenguaje y acción descrita en trabajos recientes [24], [25].

La interpretación debe considerar además el objetivo original de $\pi_{0.5}$. El modelo fue concebido para mejorar la generalización en entornos nuevos y para abordar tareas de mayor horizonte mediante una combinación de razonamiento semántico de alto nivel y generación continua de acciones [4]. $\pi_{0.5}$-DROID se evaluó en tareas cortas de *pick-and-place*, sobre una observación inicial fija y mediante la interfaz residual de transferencia desarrollada para el UR5e. Por tanto, el protocolo no reproduce el régimen de largo horizonte del sistema original ni explota todas las características para las que fue diseñado.

A esta limitación del protocolo se añade una cautela específica en el análisis de las métricas temporales. A-PI, A-VI y A-AI se calcularon sobre salidas desnormalizadas asociadas a checkpoints distintos, por lo que sus magnitudes absolutas no se utilizaron para establecer una comparación cuantitativa directa entre π_0-DROID y $\pi_{0.5}$-DROID. Su utilidad se centró en analizar, dentro de cada política, cómo variaba la dinámica interna de los bloques entre episodios con y sin éxito. Esta precaución es coherente con OpenPI, donde la semántica del espacio de acciones, las estadísticas de normalización y las transformaciones asociadas al checkpoint forman parte del contrato necesario para interpretar correctamente las predicciones [5], [6], [17].

Frente al entorno DROID, en este trabajo cambiaron el manipulador, el efector final, la configuración sensorial y la interfaz de control. Bajo estas condiciones, π_0-DROID obtuvo un mayor cumplimiento físico que $\pi_{0.5}$-DROID en las dos tareas.

La comparación entre éxitos y fallos impide, no obstante, identificar la variabilidad como causa única del rendimiento. En la Tarea 1, los éxitos de $\pi_{0.5}$ no coincidieron con los bloques menos variables; en la Tarea 2 sí apareció una reducción consistente de A-PI, A-VI y A-AI en los subconjuntos exitosos. Las métricas caracterizan la dinámica interna de las acciones y ayudan a detectar problemas de transferencia, pero deben analizarse junto con el *grounding*, la relación espacial, la adaptación y la verificación final [59].

6 Conclusiones y líneas futuras

6.1. Conclusiones

En atención a lo expuesto en los resultados y la discusión de este TFG, la campaña experimental realizada permite concluir que la integración de modelos VLA preentrenados en una configuración experimental diferente de las asociadas a sus datos y checkpoints de origen es viable, pero requiere una interfaz explícita entre la representación aprendida, la percepción métrica y la ejecución física. Los resultados muestran además que el rendimiento final no depende de una única etapa del sistema, sino de la interacción entre interpretación lingüística, *grounding*, generación de acciones, adaptación al robot y verificación del resultado.

La arquitectura desarrollada permitió integrar la percepción tridimensional de la escena, el *grounding* visual-lingüístico, la inferencia VLA, la adaptación de las acciones y una ejecución protegida y validada. Esta separación hizo posible determinar no solo si el robot completaba una tarea, sino también en qué etapa se originaban los fallos.

En relación con los objetivos definidos al inicio del trabajo, se consideran alcanzados los siguientes resultados:

1. Se estudiaron y desplegaron OpenVLA, π_0-DROID y $\pi_{0.5}$-DROID, caracterizando sus entradas, sus formas de salida y sus requisitos de inferencia y ejecución.
2. Se integró la percepción tridimensional con el UR5e y se estableció una referencia métrica común entre la cámara, la base del robot y el TCP del efector.
3. Se desarrolló una estrategia de transferencia entre las políticas preentrenadas y la plataforma experimental, separando la contribución aprendida de la percepción geométrica, la cinemática, la pinza y las restricciones de ejecución.
4. Se diseñó y ejecutó un protocolo con dos tareas complementarias que permitió analizar de forma diferenciada lenguaje, *grounding*, generación de acciones, ejecución física y verificación final.
5. Se compararon las tres configuraciones integradas bajo condiciones experimentales equivalentes y se caracterizaron diferencias de rendimiento y modos de fallo.

Por el tamaño muestral de las campañas físicas, las tasas obtenidas deben interpretarse de forma descriptiva y no como evidencia de un ranking universal entre arquitecturas. Bajo el protocolo y las interfaces desarrolladas, OpenVLA + SigLIP obtuvo los mejores resultados globales y π_0-DROID presentó un comportamiento más favorable que $\pi_{0.5}$-DROID en las tareas evaluadas. Las ablaciones mostraron además una

contribución clara del *grounding* externo: OpenVLA + SigLIP obtuvo 10/10 y 11/12 selecciones correctas en las Tareas 1 y 2, frente a 3/10 y 3/12 en los *replays* sin SigLIP; esta comparación no constituye una evaluación física pareada de la capacidad motora de OpenVLA.

La principal conclusión metodológica es que la evaluación física de un VLA no debería reducirse a una única tasa de éxito: separar lenguaje, *grounding*, acción, adaptación, ejecución y verificación permite identificar las limitaciones. De cara al desensamblaje robotizado, los resultados indican que, antes de aumentar la complejidad de la manipulación, deben reforzarse especialmente el *grounding*, el razonamiento espacial y la adaptación de las acciones.

6.2. Líneas futuras

Como continuación directa, resulta de interés estudiar la adaptación específica al *embodiment* mediante *fine-tuning* o técnicas equivalentes, comparando esta estrategia con la transferencia sin entrenamiento utilizada en este trabajo. También sería útil evaluar esquemas de ejecución en lazo cerrado, en los que el entorno se vuelva a observar tras fragmentos más cortos del bloque de acciones y la política pueda corregir su predicción durante la tarea.

La ampliación del protocolo a más objetos, disposiciones, relaciones espaciales y orientaciones permitiría comprobar la generalización de los resultados. Una extensión especialmente relevante sería aumentar la influencia directa de la política robótica sobre la orientación del efector y estudiar tareas con obstáculos o cambios dinámicos de escena.

Finalmente, los resultados obtenidos muestran que la interfaz de adaptación de acciones merece un análisis específico. Comparar diferentes estrategias de normalización, escalado y utilización de los *action chunks*, manteniendo constante el modelo y la tarea, permitiría aislar mejor qué parte de las diferencias observadas procede de la política y cuál de su transferencia a la plataforma física. Esta separación sería un paso necesario antes de trasladar la arquitectura a tareas de manipulación menos controladas. En particular, su transferencia posterior a operaciones de desensamblaje requeriría además tratar componentes parcialmente ocluidos, interacciones de contacto, control de fuerza y herramientas específicas.

A Resultados experimentales completos

Este apéndice reúne la información individual de los episodios físicos utilizados en la evaluación presentada en el Capítulo 5. Su finalidad es complementar los resultados agregados de la memoria y facilitar la trazabilidad entre cada caso experimental y las tasas de éxito presentadas en las figuras y el análisis.

La campaña física estuvo formada por 66 episodios: 30 correspondientes a la Tarea 1, de colocación de objetos sobre zonas, y 36 correspondientes a la Tarea 2, de clasificación por forma.

A.1. Tarea 1: colocación de objetos sobre zonas

La primera tarea estuvo formada por diez casos evaluados con cada una de las tres configuraciones consideradas: OpenVLA + SigLIP, π_0-DROID y $\pi_{0.5}$-DROID. En total se realizaron 30 episodios físicos.

Para cada ensayo se registraron la instrucción utilizada, el objeto y el destino esperados, el resultado del *grounding*, la ejecución física y el cumplimiento estricto de la instrucción. Esta separación permite distinguir los fallos de selección de aquellos producidos durante la manipulación o durante la verificación final.

TABLA A.1: Casos físicos definidos para la Tarea 1.

Caso	Instrucción	Objetivo esperado
C01	*put the white cylinder in the center of the blue zone*	cilindro blanco → zona azul, centro
C02	*put the white cylinder on the left side of the yellow zone*	cilindro blanco → zona amarilla, izquierda
C03	*place the orange cube inside the right side of the blue zone*	cubo naranja → zona azul, derecha
C04	*put the orange cube in the blue zone*	cubo naranja → zona azul
C05	*move the white cylinder to the right-hand side of the yellow zone*	cilindro blanco → zona amarilla, derecha
C06	*place the orange die near the rear side of the yellow area*	cubo naranja → zona amarilla, parte trasera
C07	*place the white cylinder near the front side of the yellow zone*	cilindro blanco → zona amarilla, parte delantera
C08	*place the white cylindrical object inside the right side of the yellow target area*	cilindro blanco → zona amarilla, derecha
C09	*place the orange block near the back side of the blue target area*	cubo naranja → zona azul, parte trasera
C10	*place the cylindrical white object near the back side of the blue region*	cilindro blanco → zona azul, parte trasera

TABLA A.2: Resultados individuales de los 30 episodios físicos de la Tarea 1.

Caso	Modelo	G	Rel.	Op.	Ver.	Estr.	Etapa de fallo
C01	OpenVLA + SigLIP	Sí	Sí	Sí	Sí	Sí	–
C01	π_0-DROID	Sí	Sí	Sí	Sí	Sí	–
C01	$\pi_{0.5}$-DROID	Sí	Sí	Sí	Sí	Sí	–
C02	OpenVLA + SigLIP	Sí	No	Sí	Sí	No	Relación espacial
C02	π_0-DROID	Sí	Sí	Sí	Sí	Sí	–
C02	$\pi_{0.5}$-DROID	Sí	Sí	Sí	Sí	Sí	–
C03	OpenVLA + SigLIP	Sí	No	Sí	Sí	No	Relación espacial
C03	π_0-DROID	Sí	No	Sí	Sí	No	Relación espacial
C03	$\pi_{0.5}$-DROID	Sí	No	Sí	Sí	No	Relación espacial
C04	OpenVLA + SigLIP	Sí	N/A	Sí	Sí	Sí	–
C04	π_0-DROID	Sí	N/A	Sí	Sí	Sí	–
C04	$\pi_{0.5}$-DROID	No	N/A	No	No	No	Grounding
C05	OpenVLA + SigLIP	Sí	Límite (1 mm)	Sí	Sí	No	Relación espacial
C05	π_0-DROID	No	No medible	No	No	No	Grounding
C05	$\pi_{0.5}$-DROID	No	No medible	No	No	No	Grounding
C06	OpenVLA + SigLIP	Sí	Sí	Sí	Sí	Sí	–
C06	π_0-DROID	Sí	Sí	Sí	Sí	Sí	–
C06	$\pi_{0.5}$-DROID	Sí	No	Sí	Sí	No	Relación espacial
C07	OpenVLA + SigLIP	Sí	Sí	Sí	Sí	Sí	–
C07	π_0-DROID	Sí	No	Sí	Sí	No	Relación espacial
C07	$\pi_{0.5}$-DROID	Sí	Sí	No	No	No	Verificación post-place

Continúa en la página siguiente

Tabla A.2 – continuación

Caso	Modelo	G	Rel.	Op.	Ver.	Estr.	Etapa de fallo
C08	OpenVLA + SigLIP	Sí	Sí	No	No	No	Verificación post-place
C08	π_0-DROID	No	No medible	No	No	No	Grounding
C08	$\pi_{0.5}$-DROID	No	No medible	No	No	No	Grounding
C09	OpenVLA + SigLIP	Sí	Límite (1 mm)	Sí	Sí	No	Relación espacial
C09	π_0-DROID	Sí	Sí	Sí	Sí	Sí	–
C09	$\pi_{0.5}$-DROID	Sí	No	Sí	Sí	No	Relación espacial
C10	OpenVLA + SigLIP	Sí	Sí	Sí	Sí	Sí	–
C10	π_0-DROID	No	No medible	No	No	No	Grounding
C10	$\pi_{0.5}$-DROID	Sí	No	No	Sí	No	Relación espacial

G: *grounding* correcto; *Rel.*: cumplimiento físico de la relación espacial; *Op.*: éxito operacional; *Ver.*: verificación posterior a la deposición; *Estr.*: cumplimiento estricto. En la columna de relación, "N/A" indica que el caso no incluía una relación espacial y "No medible" que un fallo previo de *grounding* impidió evaluarla.

A.2. Tarea 2: clasificación por forma

La segunda tarea estuvo formada por tres disposiciones físicas de la escena. En cada disposición se evaluaron cuatro instrucciones con las tres configuraciones, dando lugar a un total de 36 episodios físicos.

Los casos contemplaron tanto instrucciones explícitas, en las que se indicaba directamente el recipiente de destino, como instrucciones implícitas en las que el sistema debía asociar la forma del objeto con el recipiente correspondiente.

TABLA A.3: Tipos de caso físicos definidos para la Tarea 2.

Prueba	Tipo	Instrucción	Objetivo esperado
Cubo–forma	Implícita	*put the orange cube in the container that matches its shape*	cubo naranja → recipiente cuadrado
Cubo–explícita	Explícita	*put the orange cube in the square box*	cubo naranja → recipiente cuadrado
Cilindro–forma	Implícita	*put the white cylinder in the container that matches its shape*	cilindro blanco → recipiente circular
Cilindro–explícita	Explícita	*put the white cylinder in the circular box*	cilindro blanco → recipiente circular

TABLA A.4: Resultados individuales de los 36 episodios físicos de la Tarea 2.

Caso	L.	Modelo	Prueba	G	Ejec.	Col.	Estr.	Resultado
C01	1	OpenVLA + SigLIP	C–forma	Sí	Sí	Sí	Sí	Éxito
C02	1	OpenVLA + SigLIP	C–expl.	Sí	Sí	No	No	Fallo de deposición
C03	1	OpenVLA + SigLIP	Ci–forma	No	No	No	No	Fallo de grounding
C04	1	OpenVLA + SigLIP	Ci–expl.	Sí	Sí	Sí	Sí	Éxito
C05	1	π_0-DROID	C–forma	No	No	No	No	Fallo de grounding
C06	1	π_0-DROID	C–expl.	No	No	No	No	Fallo de grounding
C07	1	π_0-DROID	Ci–forma	No	No	No	No	Fallo de grounding
C08	1	π_0-DROID	Ci–expl.	Sí	Sí	Sí	Sí	Éxito
C09	1	$\pi_{0.5}$-DROID	C–forma	No	No	No	No	Fallo de grounding
C10	1	$\pi_{0.5}$-DROID	C–expl.	No	No	No	No	Fallo de grounding
C11	1	$\pi_{0.5}$-DROID	Ci–forma	No	No	No	No	Fallo de grounding
C12	1	$\pi_{0.5}$-DROID	Ci–expl.	Sí	Sí	Sí	Sí	Éxito
C13	2	OpenVLA + SigLIP	C–forma	Sí	Sí	Sí	Sí	Éxito
C14	2	OpenVLA + SigLIP	C–expl.	Sí	Sí	Sí	Sí	Éxito
C15	2	OpenVLA + SigLIP	Ci–forma	Sí	Sí	Sí	Sí	Éxito
C16	2	OpenVLA + SigLIP	Ci–expl.	Sí	Sí	Sí	Sí	Éxito
C17	2	π_0-DROID	C–forma	No	No	No	No	Fallo de grounding
C18	2	π_0-DROID	C–expl.	No	No	No	No	Fallo de grounding
C19	2	π_0-DROID	Ci–forma	Sí	Sí	No	No	Fallo de deposición
C20	2	π_0-DROID	Ci–expl.	No	No	No	No	Fallo de grounding
C21	2	$\pi_{0.5}$-DROID	C–forma	No	No	No	No	Fallo de grounding
C22	2	$\pi_{0.5}$-DROID	C–expl.	No	No	No	No	Fallo de grounding
C23	2	$\pi_{0.5}$-DROID	Ci–forma	Sí	Sí	No	No	Fallo de deposición
C24	2	$\pi_{0.5}$-DROID	Ci–expl.	Sí	Sí	Sí	Sí	Éxito
C25	3	OpenVLA + SigLIP	C–forma	Sí	Sí	Sí	Sí	Éxito
C26	3	OpenVLA + SigLIP	C–expl.	Sí	Sí	No	No	Fallo de deposición
C27	3	OpenVLA + SigLIP	Ci–forma	Sí	Sí	No	No	Fallo de deposición
C28	3	OpenVLA + SigLIP	Ci–expl.	Sí	Sí	Sí	Sí	Éxito
C29	3	π_0-DROID	C–forma	No	No	No	No	Fallo de grounding
C30	3	π_0-DROID	C–expl.	Sí	Sí	Sí	No	Fallo de retirada
C31	3	π_0-DROID	Ci–forma	Sí	Sí	Sí	Sí	Éxito
C32	3	π_0-DROID	Ci–expl.	Sí	Sí	Sí	Sí	Éxito

Continúa en la página siguiente

Tabla A.4 – continuación

Caso	L.	Modelo	Prueba	G	Ejec.	Col.	Estr.	Resultado
C33	3	$\pi_{0.5}$-DROID	C–forma	No	No	No	No	Fallo de grounding
C34	3	$\pi_{0.5}$-DROID	C–expl.	No	No	No	No	Fallo de grounding
C35	3	$\pi_{0.5}$-DROID	Ci–forma	No	No	No	No	Fallo de grounding
C36	3	$\pi_{0.5}$-DROID	Ci–expl.	No	No	No	No	Fallo de grounding

L.: *layout*; *G*: *grounding* correcto; *Ejec.*: ejecución intentada; *Col.*: colocación correcta; *Estr.*: cumplimiento estricto. Las abreviaturas C–forma, C–expl., Ci–forma y Ci–expl. corresponden, respectivamente, a cubo con instrucción implícita por forma, cubo con instrucción explícita, cilindro con instrucción implícita por forma y cilindro con instrucción explícita.

A.3. Contribución traslacional aplicada por la interfaz VLA

Además de las salidas nativas de los modelos, durante los episodios se conservaron los residuales traslacionales incorporados a la trayectoria después de la adaptación descrita en la Sección 4.6. Este registro permite cuantificar la magnitud con la que la información de acción procedente de cada configuración modificó las referencias geométricas obtenidas mediante la percepción tridimensional.

El análisis se restringe a los episodios que superaron correctamente la etapa de *grounding*, ya que un fallo previo en la selección del objeto o del destino bloqueaba la ejecución antes de alcanzar la etapa de adaptación de acciones. De los 66 episodios físicos de la campaña, 43 alcanzaron esta etapa: 24 en la Tarea 1 y 19 en la Tarea 2. Los 23 episodios restantes no se consideran como residuales nulos, puesto que en ellos la contribución traslacional no llegó a aplicarse.

La Tabla A.5 muestra la norma euclídea media del residual traslacional finalmente aplicado en las fases de agarre, elevación y deposición. Los valores se expresan en milímetros en el sistema de referencia de la base del UR5e.

TABLA A.5: Magnitud media de la contribución traslacional aplicada por la interfaz VLA en los episodios que superaron correctamente el *grounding*.

Tarea	Configuración	n	Pick [mm]	Lift [mm]	Place [mm]
Tarea 1	OpenVLA + SigLIP	10	7,00	13,84	2,83
Tarea 1	π_0-DROID	7	8,87	16,67	2,20
Tarea 1	$\pi_{0.5}$-DROID	7	7,47	27,39	4,00
Tarea 2	OpenVLA + SigLIP	11	8,08	12,85	3,47
Tarea 2	π_0-DROID	5	8,75	17,54	1,79
Tarea 2	$\pi_{0.5}$-DROID	3	8,54	22,25	3,62

Las tres configuraciones produjeron correcciones del orden de milímetros. En la fase de agarre, las magnitudes medias se situaron aproximadamente entre 7 y 9 mm en ambas tareas. La mayor contribución apareció en la fase de elevación, donde los

valores medios oscilaron entre 12,85 y 27,39 mm. En la deposición, las correcciones fueron menores, con valores medios comprendidos entre 1,79 y 4,00 mm.

Para estos episodios los residuales recogidos en la tabla coinciden con las correcciones incorporadas a la trayectoria después de la adaptación y acotación establecidas por la interfaz.

Estas magnitudes deben interpretarse como una característica de cada configuración integrada, es decir, del modelo junto con la interfaz de transferencia desarrollada en este trabajo. No representan directamente la magnitud de las acciones nativas de los VLA, ya que OpenVLA y las configuraciones DROID emplean espacios de acción y procedimientos de adaptación diferentes.

Asimismo, estos resultados permiten cuantificar cuánto se modificó la referencia geométrica en los episodios ejecutables, pero no permiten determinar de forma causal si dicha modificación mejoró o empeoró el resultado físico. Como se indicó en la Sección 1.4, no se realizó una campaña física equivalente con $\Delta p^{\mathrm{VLA}} = 0$ que permitiera aislar experimentalmente el efecto del residual respecto a una trayectoria basada únicamente en la referencia geométrica.

B Ensayos complementarios de robustez y ablación

Además de los episodios físicos, se realizaron ensayos mediante *replay* sobre observaciones previamente almacenadas. Estos ensayos permitieron estudiar la respuesta de las configuraciones ante modificaciones controladas de la instrucción y analizar el efecto del adaptador externo SigLIP sin repetir el movimiento físico del robot.

En total se realizaron 186 *replays* de robustez lingüística: 150 correspondientes a la Tarea 1 y 36 a la Tarea 2. A ellos se añadieron 22 *replays* específicos para estudiar la ablación del adaptador externo SigLIP en OpenVLA.

Los registros individuales de los 186 ensayos lingüísticos se incluyen también en el material complementario entregado junto con esta memoria. En este apéndice se presentan sus resultados desglosados por configuración y tipo de variación, junto con los resultados individuales de la ablación.

B.1. Robustez lingüística

En los ensayos de robustez lingüística se mantuvo fija la observación visual y se modificó de forma controlada la formulación de la instrucción. Las variantes incluyeron instrucciones canónicas, paráfrasis, alias de objetos y destinos, variaciones de relaciones espaciales, formulaciones ambiguas y referencias a entidades no disponibles o incompatibles con la escena.

En las instrucciones positivas se consideró correcta la selección cuando la pareja objeto–destino obtenida coincidía con la esperada. En los ensayos negativos se evaluó si la configuración rechazaba correctamente una instrucción ambigua, contradictoria o no compatible con la escena.

B.1.1. Tarea 1: colocación de objetos sobre zonas

Para la Tarea 1 se realizaron 50 *replays* por configuración, para un total de 150. Cada configuración recibió 42 instrucciones positivas y 8 instrucciones negativas que debían ser rechazadas.

La Tabla B.1 muestra los resultados obtenidos por cada configuración.

OpenVLA + SigLIP mantuvo la pareja objeto–destino correcta en 37 de las 42 instrucciones positivas y rechazó las 8 instrucciones negativas. En π_0-DROID se obtuvieron 31 parejas positivas correctas y un rechazo negativo, mientras que $\pi_{0.5}$-DROID alcanzó 15 parejas correctas y tres rechazos.

TABLA B.1: Resultados de robustez lingüística por configuración en la Tarea 1.

Configuración	Positivas	Correctas	Negativas	Rechazadas
OpenVLA + SigLIP	42	37	8	8
π_0-DROID	42	31	8	1
$\pi_{0.5}$-DROID	42	15	8	3

La Tabla B.2 agrupa los resultados según el tipo de variación lingüística. En las categorías positivas se contabiliza la selección correcta de la pareja objeto–destino; en las categorías negativas se contabiliza el rechazo correcto de la instrucción.

TABLA B.2: Desglose por configuración y tipo de variación lingüística en la Tarea 1.

Tipo de variación	OpenVLA + SigLIP	π_0-DROID	$\pi_{0.5}$-DROID	Total
Canónica	10/10 (100 %)	8/10 (80 %)	5/10 (50 %)	23/30 (76,7 %)
Alias combinados	0/2 (0 %)	0/2 (0 %)	1/2 (50 %)	1/6 (16,7 %)
Alias de objeto	5/8 (62,5 %)	7/8 (87,5 %)	3/8 (37,5 %)	15/24 (62,5 %)
Control direccional	4/4 (100 %)	2/4 (50 %)	2/4 (50 %)	8/12 (66,7 %)
Alias de destino	9/9 (100 %)	8/9 (88,9 %)	2/9 (22,2 %)	19/27 (70,4 %)
Alias de relación	8/8 (100 %)	5/8 (62,5 %)	2/8 (25 %)	15/24 (62,5 %)
Alias de comando	1/1 (100 %)	1/1 (100 %)	0/1 (0 %)	2/3 (66,7 %)
Instrucción ambigua	6/6 (100 %)	0/6 (0 %)	1/6 (16,7 %)	7/18 (38,9 %)
Entidad no soportada	2/2 (100 %)	1/2 (50 %)	2/2 (100 %)	5/6 (83,3 %)

El desglose muestra que una misma tasa agregada puede ocultar comportamientos muy diferentes entre configuraciones. OpenVLA + SigLIP mantuvo el 100 % de acierto en las instrucciones canónicas, los controles direccionales, los alias de destino y de relación, y rechazó correctamente todos los casos ambiguos y no soportados. π_0-DROID conservó un comportamiento elevado en instrucciones canónicas y alias de objeto y destino, pero no rechazó ninguno de los seis casos ambiguos. $\pi_{0.5}$-DROID presentó una mayor degradación en varias categorías, especialmente en los alias de destino y de relación.

Los alias combinados constituyeron la condición positiva más desfavorable de forma agregada, con un único acierto entre seis ensayos. Este resultado muestra que la estabilidad frente a una modificación lingüística aislada no implica necesariamente robustez cuando se modifican simultáneamente varias expresiones de la instrucción.

B.1.2. Tarea 2: clasificación por forma

Para la Tarea 2 se realizaron 12 *replays* por configuración, para un total de 36. Cada configuración recibió 8 instrucciones positivas y 4 negativas.

Los resultados por configuración se recogen en la Tabla B.3.

TABLA B.3: Resultados de robustez lingüística por configuración en la Tarea 2.

Configuración	Positivas	Correctas	Negativas	Rechazadas
OpenVLA + SigLIP	8	7	4	4
π_0-DROID	8	4	4	1
$\pi_{0.5}$-DROID	8	1	4	2

La distribución por tipo de instrucción se muestra en la Tabla B.4.

TABLA B.4: Desglose por configuración y tipo de variación lingüística en la Tarea 2.

Tipo de variación	**OpenVLA + SigLIP**	**π_0-DROID**	**$\pi_{0.5}$-DROID**	**Total**
Canónica implícita	2/2 (100 %)	1/2 (50 %)	0/2 (0 %)	3/6 (50 %)
Paráfrasis implícita	2/2 (100 %)	0/2 (0 %)	0/2 (0 %)	2/6 (33,3 %)
Canónica explícita	2/2 (100 %)	2/2 (100 %)	0/2 (0 %)	4/6 (66,7 %)
Alias explícito	1/2 (50 %)	1/2 (50 %)	1/2 (50 %)	3/6 (50 %)
Objeto ambiguo	1/1 (100 %)	0/1 (0 %)	0/1 (0 %)	1/3 (33,3 %)
Destino ambiguo	1/1 (100 %)	0/1 (0 %)	0/1 (0 %)	1/3 (33,3 %)
Contradicción	1/1 (100 %)	1/1 (100 %)	1/1 (100 %)	3/3 (100 %)
Objeto ausente	1/1 (100 %)	0/1 (0 %)	1/1 (100 %)	2/3 (66,7 %)

El desglose de la Tarea 2 vuelve a mostrar diferencias que no son visibles en la tasa agregada. OpenVLA + SigLIP resolvió correctamente todas las instrucciones canónicas implícitas, las paráfrasis implícitas y las instrucciones canónicas explícitas. π_0-DROID mantuvo los dos aciertos en las formulaciones canónicas explícitas, pero falló las dos paráfrasis implícitas. $\pi_{0.5}$-DROID no resolvió correctamente ninguna de las instrucciones canónicas implícitas, paráfrasis implícitas o canónicas explícitas de este conjunto, aunque obtuvo un acierto en la categoría de alias explícito.

En los casos negativos, las tres configuraciones rechazaron correctamente las instrucciones contradictorias. En cambio, solo OpenVLA + SigLIP rechazó los casos de objeto y destino ambiguos, mientras que ante el objeto ausente el rechazo fue correcto para OpenVLA + SigLIP y $\pi_{0.5}$-DROID.

B.2. Ablación del adaptador externo SigLIP

La contribución del adaptador externo SigLIP utilizado junto con OpenVLA se analizó mediante 22 *replays* específicos: 10 correspondientes a la Tarea 1 y 12 a la Tarea 2.

En cada replay se reutilizó la observación almacenada y se eliminó el apoyo externo de SigLIP para la selección explícita de candidatos. De esta forma se pudo analizar la selección de OpenVLA sin dicho adaptador manteniendo fija la evidencia visual disponible.

B.2.1. Ablación en la Tarea 1

Los diez casos corresponden a las instrucciones definidas en la Tabla A.1. Con SigLIP, la configuración integrada obtuvo una selección objeto–destino correcta en los 10 casos. En los replays sin SigLIP, la selección fue correcta en 3 de los 10.

La Tabla B.5 muestra el resultado individual y la selección obtenida en la condición sin SigLIP.

La conservación de la pareja correcta en C04, C07 y C09 explica los 3/10 aciertos registrados sin el adaptador. En los siete casos restantes cambió el objeto, el destino o ambos respecto a la pareja esperada.

TABLA B.5: Ablación de SigLIP en los diez casos de la Tarea 1.

Caso	Selección sin SigLIP	Con SigLIP	Sin SigLIP
C01	cubo naranja → zona amarilla	Sí	No
C02	cubo naranja → zona azul	Sí	No
C03	cilindro blanco → zona azul	Sí	No
C04	cubo naranja → zona azul	Sí	Sí
C05	cubo naranja → zona amarilla	Sí	No
C06	cubo naranja → zona azul	Sí	No
C07	cilindro blanco → zona amarilla	Sí	Sí
C08	cubo naranja → zona amarilla	Sí	No
C09	cubo naranja → zona azul	Sí	Sí
C10	cubo naranja → zona azul	Sí	No

B.2.2. Ablación en la Tarea 2

En la Tarea 2 se realizaron 12 replays sin SigLIP, correspondientes a los casos de OpenVLA de los tres *layouts*. La configuración con SigLIP había obtenido 11/12 selecciones correctas, mientras que la condición sin SigLIP obtuvo 3/12.

TABLA B.6: Ablación de SigLIP en los doce casos de OpenVLA de la Tarea 2.

Caso	L.	Prueba	Selección sin SigLIP	Con	Sin
C01	1	Cubo–forma	cilindro blanco → recipiente circular	Sí	No
C02	1	Cubo–explícita	cubo naranja → recipiente circular	Sí	No
C03	1	Cilindro–forma	sin selección válida	No	No
C04	1	Cilindro–explícita	cubo naranja → recipiente cuadrado	Sí	No
C13	2	Cubo–forma	cubo naranja → recipiente cuadrado	Sí	Sí
C14	2	Cubo–explícita	cilindro blanco → recipiente cuadrado	Sí	No
C15	2	Cilindro–forma	cilindro blanco → recipiente cuadrado	Sí	No
C16	2	Cilindro–explícita	cilindro blanco → recipiente circular	Sí	Sí
C25	3	Cubo–forma	cubo naranja → recipiente circular	Sí	No
C26	3	Cubo–explícita	cilindro blanco → recipiente cuadrado	Sí	No
C27	3	Cilindro–forma	cilindro blanco → recipiente circular	Sí	Sí
C28	3	Cilindro–explícita	cubo naranja → recipiente circular	Sí	No

L. indica el *layout*. Las columnas "Con" y "Sin" indican si la pareja objeto–destino fue correcta con el adaptador externo SigLIP y en el replay sin dicho adaptador, respectivamente.

Los tres aciertos sin SigLIP correspondieron a C13, C16 y C27. La condición de referencia con SigLIP falló únicamente en C03, por lo que la comparación global fue de 11/12 selecciones correctas con SigLIP frente a 3/12 sin SigLIP.

B.3. Trazabilidad de los replays

Los resultados individuales utilizados para construir las tablas de robustez lingüística se conservan en el fichero `resultados/linguistica.csv` del material complementario. Los registros de la ablación se conservan por separado para cada tarea.

Esta organización permite relacionar los resultados agregados presentados en el Capítulo 5 con las instrucciones, selecciones y resultados individuales, sin reproducir en la memoria las 186 filas completas del conjunto de replays lingüísticos.

C Implementación software y reproducibilidad

Este apéndice describe la organización del software desarrollado para la integración experimental presentada en el Capítulo 4 y muestra fragmentos representativos de la implementación. El objetivo no es reproducir íntegramente el código fuente dentro de la memoria, sino relacionar las principales decisiones metodológicas con las funciones que las materializan.

El material complementario entregado contiene los principales scripts empleados para la adquisición de la escena, percepción, inferencia, *grounding*, adaptación de acciones, planificación, supervisión y ejecución física.

C.1. Organización del código

La implementación se organizó de forma modular. El programa principal coordina el episodio experimental, mientras que los módulos auxiliares se encargan de funciones concretas de percepción, preparación de la escena, supervisión, ejecución y *grounding* externo.

La Tabla C.1 resume los 13 scripts incluidos en el material complementario y su responsabilidad principal.

TABLA C.1: Archivos de código incluidos en el material complementario.

Archivo	Función principal
`principal.py`	Integración principal del sistema VLA y coordinación del flujo completo de cada episodio.
`flujo.py`	Preparación de la escena, normalización de instrucciones y coordinación de distintas etapas auxiliares del flujo experimental.
`supervisor.py`	Supervisión y planificación asociada principalmente a las configuraciones π_0-DROID y $\pi_{0.5}$-DROID.
`ejecucion.py`	Ejecución física protegida del UR5e y del efector OnRobot 3FG15, junto con funciones de planificación y cinemática.
`vision_escena.py`	Construcción de la representación multiobjeto de la escena a partir de la captura tridimensional.
`vision_zonas.py`	Procesamiento geométrico y obtención de candidatos para la tarea objeto–zona.
`vision_forma.py`	Procesamiento específico de objetos y recipientes para la tarea de clasificación por forma.
`photoneo/servidor.py`	Servicio utilizado para solicitar una nueva adquisición mediante la Photoneo.

Continúa en la página siguiente

Tabla C.1 – continuación

Archivo	Función principal
`photoneo/captura.py`	Cliente encargado de solicitar y validar una nueva captura tridimensional.
`siglip/zonas_cliente.py`	Cliente del servicio de *grounding* externo empleado en la Tarea 1.
`siglip/zonas_servidor.py`	Servidor SigLIP utilizado para evaluar los candidatos de objeto y destino de la Tarea 1.
`siglip/forma_cliente.py`	Cliente del servicio SigLIP específico de la clasificación por forma.
`siglip/forma_servidor.py`	Servidor SigLIP utilizado para comparar los candidatos de objeto y recipiente de la Tarea 2.

C.2. Secuencia general de un episodio

El punto de entrada principal de la integración es `principal.py`. A partir de una instrucción y una observación de la escena, el programa coordina la inferencia del modelo, el *grounding*, la adaptación de las acciones, la construcción del plan y las comprobaciones necesarias antes de permitir una ejecución física.

De forma simplificada, la secuencia implementada puede representarse como:

instrucción y escena → inferencia VLA → *grounding* → adaptación de acciones → cinemática y planificación → supervisión → ejecución.

La salida nativa de cada modelo se conserva antes de aplicar la adaptación específica de la interfaz. El código comprueba además que la forma de las acciones recibidas coincida con la esperada para cada configuración:

```
EXPECTED_SHAPES = {
    "openvla": (1, 7),
    "pi05_droid": (15, 8),
    "pi0_droid": (10, 8),
}
```

CÓDIGO C.1: Formas de salida esperadas por la integración.

Esta comprobación permite detener el flujo antes de la planificación cuando la respuesta recibida no satisface el contrato esperado para el modelo.

C.3. Grounding y neutralización de candidatos

Una de las decisiones de implementación relevantes corresponde al tratamiento de los candidatos enviados al adaptador externo SigLIP. Las entidades obtenidas por los módulos de percepción contienen internamente información suficiente para relacionarlas con la geometría de la escena. Sin embargo, dicha identidad semántica no se transmite directamente al servicio SigLIP.

El cliente asigna identificadores neutrales a las regiones candidatas y envía únicamente el identificador y la caja delimitadora correspondiente. La relación entre el identificador neutral y la entidad física original permanece en el cliente.

El Código C.2 muestra el procedimiento utilizado en el cliente de la Tarea 2.

```python
def neutralize(
    candidates: list[dict[str, Any]],
    *,
    role: str,
) -> tuple[list[dict[str, Any]], dict[str, dict[str, Any]]]:
    if not isinstance(candidates, list) or len(candidates) != 2:
        count = len(candidates) if isinstance(candidates, list) else "invalid"
        raise ShapeSiglipGroundingError(
            f"{role}: se requieren dos candidatos; recibidos={count}"
        )

    neutral, mapping = [], {}

    for index, candidate in enumerate(candidates, start=1):
        if not isinstance(candidate, dict):
            raise ShapeSiglipGroundingError(
                f"{role}[{index}]: candidato inválido"
            )

        local_key = str(
            candidate.get(
                "key",
                candidate.get("canonical", candidate.get("name", "")),
            )
        ).strip()

        if not local_key:
            raise ShapeSiglipGroundingError(
                f"{role}[{index}]: falta identidad local"
            )

        neutral_id = f"{role}_{index:02d}"

        neutral.append(
            {
                "candidate_id": neutral_id,
                "bbox_xyxy": finite_bbox(
                    candidate.get("bbox_xyxy"),
                    label=f"{role}.{local_key}",
                ),
            }
        )
        mapping[neutral_id] = candidate

    return neutral, mapping
```

CÓDIGO C.2: Neutralización de candidatos antes de enviarlos al servicio SigLIP.

Esta separación evita que el identificador enviado al servicio revele directamente si una región corresponde al cubo naranja, al cilindro blanco o a uno de los recipientes. La correspondencia semántica debe obtenerse a partir de la evidencia visual y de las descripciones textuales proporcionadas.

C.3.1. Selección y comprobaciones del grounding externo

El servidor SigLIP calcula puntuaciones de compatibilidad para los candidatos de cada rol. La selección se acompaña de comprobaciones sobre el margen entre alternativas y sobre la coherencia entre la clase solicitada y la asignación obtenida para el candidato.

```python
for class_index, class_name in enumerate(class_names):
    query_logits = logits[:, class_index]
    probabilities = softmax(query_logits)
    order = np.argsort(query_logits)[::-1]
```

```
    best, second = int(order[0]), int(order[1])

    probability_margin = float(
        probabilities[best] - probabilities[second]
    )

    class_results[class_name] = {
        "selected_candidate_id": candidate_ids[best],
        "selected_probability": float(probabilities[best]),
        "probability_margin": probability_margin,
        "query_margin_pass": probability_margin >= MIN_QUERY_MARGIN,
    }

checks = []
for class_name in class_names:
    query = class_results[class_name]
    assignment = candidate_results[
        query["selected_candidate_id"]
    ]
    mutual_top1 = assignment["selected_class"] == class_name

    checks.append(
        {
            "class_name": class_name,
            "selected_candidate_id": query["selected_candidate_id"],
            "mutual_top1": mutual_top1,
            "pass": bool(
                mutual_top1
                and query["query_margin_pass"]
                and assignment["assignment_margin_pass"]
            ),
        }
    )
```

CÓDIGO C.3: Comprobaciones de consistencia utilizadas en el grounding SigLIP.

Estas funciones corresponden al apoyo externo de SigLIP descrito en la Sección 4.5. No sustituyen el codificador interno de OpenVLA ni modifican su acción nativa.

C.4. Adaptación de las acciones VLA

La incompatibilidad entre los espacios de acción originales y la interfaz del UR5e se resuelve mediante una capa explícita de adaptación. Las salidas de los modelos no se envían directamente al controlador del robot.

C.4.1. Adaptación de OpenVLA

Para OpenVLA se utiliza la única acción generada por la inferencia y se construye a partir de sus componentes traslacionales un residual métrico expresado en el marco de trabajo de la tarea.

```
if model == "openvla":
    calibration = load_openvla_calibration()
    task_frame = core.build_task_frame(
        object_xyz_m,
        target_xyz_m,
    )

    metric_residual, openvla_report = (
        core.openvla_translation_residual(
            raw_action=matrix[0],
            calibration=calibration,
```

```
            task_frame=task_frame,
            translation_budget_m=OPENVLA_NATIVE_BUDGET_M,
            gain=1.0,
        )
    )

    metric_residual = finite_vector(
        metric_residual,
        3,
        "translation.openvla_metric_residual",
    )

    native_signals = {
        "initial": metric_residual.copy(),
        "middle": metric_residual.copy(),
        "result": metric_residual.copy(),
    }
```

CÓDIGO C.4: Construcción del residual traslacional para OpenVLA.

La misma corrección traslacional se reutiliza posteriormente en las fases correspondientes del plan. Las componentes de orientación y pinza de la acción nativa no se aplican directamente al UR5e.

C.4.2. Adaptación de los bloques de acciones DROID

Las configuraciones `pi0_droid` y `pi05_droid` producen bloques temporales de acciones. Como se explicó en la Sección 4.6, los canales DROID corresponden originalmente al espacio de acciones de la plataforma Franka y no a desplazamientos cartesianos del UR5e.

La implementación utiliza de forma heurística los tres primeros canales como señal para construir una corrección traslacional. El bloque se divide temporalmente en tres partes y se calcula la media de cada una.

```
thirds = _temporal_thirds(
    matrix,
    f"translation.{model}",
)

initial_mean = thirds[0].mean(axis=0)
middle_mean = thirds[1].mean(axis=0)
result_mean = thirds[2].mean(axis=0)

grouped_model_translation = np.asarray(
    [
        initial_mean[:3],
        middle_mean[:3],
        result_mean[:3],
    ],
    dtype=float,
)

metric_model_deltas = (
    np.clip(
        grouped_model_translation,
        -1.0,
        1.0,
    )
    * DROID_TRANSLATION_SCALE_M
)

model_to_base = np.asarray(
    runtime.MODEL_TO_BASE,
    dtype=float,
```

```
31 ).reshape(3, 3)
32
33 metric_base_deltas = (
34     model_to_base @ metric_model_deltas.T
35 ).T
```

CÓDIGO C.5: Construcción de señales traslacionales a partir de los bloques DROID.

Este fragmento muestra explícitamente que no se realiza una conversión cinemática de las siete velocidades articulares de Franka a velocidades cartesianas del UR5e. Los tres primeros canales se utilizan como una señal heurística, que posteriormente se escala, transforma y limita antes de modificar las referencias geométricas.

C.4.3. Acotación de los residuales

Antes de aplicar cualquier corrección a los puntos geométricos se comprueba que el residual y el presupuesto sean vectores finitos y se realiza una saturación componente a componente.

```
1  def _clip_metric_residual(
2      *,
3      proposed: np.ndarray,
4      budget: np.ndarray,
5      label: str,
6  ) -> tuple[np.ndarray, dict[str, Any]]:
7      proposed = finite_vector(
8          proposed,
9          3,
10         f"{label}.proposed",
11     )
12     budget = finite_vector(
13         budget,
14         3,
15         f"{label}.budget",
16     )
17
18     if np.any(budget <= 0.0):
19         raise MainStop(
20             f"{label}: presupuesto no positivo"
21         )
22
23     applied = np.clip(
24         proposed,
25         -budget,
26         budget,
27     )
```

CÓDIGO C.6: Acotación de los residuales métricos utilizados por la interfaz.

La operación corresponde directamente a la acotación expresada en la Ecuación 4.4. El residual resultante modifica una referencia obtenida mediante la percepción tridimensional, en lugar de reemplazarla por una acción VLA ejecutada directamente.

C.5. Cinemática inversa mediante DLS

Una vez construidos los objetivos cartesianos de las diferentes fases, es necesario obtener configuraciones articulares compatibles con el UR5e. El procedimiento de

cinemática inversa se implementó de forma iterativa utilizando los jacobianos traslacional y rotacional del TCP obtenidos mediante MuJoCo.

El núcleo de la actualización DLS se muestra en el Código C.7.

```
jacp = np.zeros((3, model.nv), dtype=float)
jacr = np.zeros((3, model.nv), dtype=float)
mujoco.mj_jacSite(model, data, jacp, jacr, tcp_id)

J = np.vstack([
    pos_w * jacp[:, dof_idx],
    axis_w * jacr[:, dof_idx],
])

e = np.concatenate([
    pos_w * pos_err,
    axis_w * axis_err,
])

A = J @ J.T + (damping ** 2) * np.eye(6)

try:
    Ainv_e = np.linalg.solve(A, e)
    Ainv_J = np.linalg.solve(A, J)
    dq_task = J.T @ Ainv_e
    J_pinv = J.T @ Ainv_J
except np.linalg.LinAlgError:
    A_pinv = np.linalg.pinv(A)
    dq_task = J.T @ (A_pinv @ e)
    J_pinv = J.T @ (A_pinv @ J)

q_err_seed = _angle_delta_to_seed(q)
N = np.eye(J.shape[1], dtype=float) - J_pinv @ J
dq_null = -cont_null_w * q_err_seed
dq = dq_task + N @ dq_null

max_abs = float(np.max(np.abs(dq))) if dq.size else 0.0
if max_abs > max_step:
    dq *= max_step / max_abs

q = q + dq
```

CÓDIGO C.7: Núcleo de la resolución iterativa de cinemática inversa mediante DLS.

La matriz amortiguada $JJ^{\mathrm{T}} + \lambda^2 I$ corresponde a la formulación DLS presentada en la Sección 3.5.3. La implementación incorpora además una contribución secundaria en el espacio nulo para favorecer configuraciones próximas a la semilla y limita el incremento articular máximo de cada iteración.

La continuidad entre fases se refuerza utilizando como referencia la configuración obtenida en el punto anterior. De esta forma, la resolución cinemática no se limita a comprobar la alcanzabilidad de cada punto de forma independiente.

C.6. Supervisión previa a la ejecución física

La construcción satisfactoria de una trayectoria no implica automáticamente que el robot pueda comenzar a moverse. Antes de la ejecución se vuelve a consultar el estado real del UR5e y se compara con el estado utilizado durante la generación del plan.

```
q_start, tcp_start = runtime.current_robot_state(node)

```

```
q_home = finite_vector(
    preplan["q_home"],
    6,
    "real.q_home",
)
tcp_home = finite_vector(
    preplan["tcp_home"],
    3,
    "real.tcp_home",
)

q_error = float(
    np.max(
        np.abs(q_start - q_home)
    )
)
tcp_error = float(
    np.linalg.norm(tcp_start - tcp_home)
)

if q_error > 0.02 or tcp_error > 0.005:
    raise MainStop(
        "El robot cambió después del preplan: "
        f"q={q_error:.4f}rad, "
        f"tcp={tcp_error:.4f}m"
    )
```

CÓDIGO C.8: Comprobación del estado real del UR5e antes de ejecutar el plan.

Si el estado actual difiere del utilizado durante la planificación por encima de las tolerancias definidas, la ejecución se detiene y el plan no se envía al robot. Esta comprobación evita ejecutar una trayectoria calculada para una configuración que ya no corresponde al estado físico de la célula.

A esta comprobación se añaden las validaciones realizadas durante la generación del plan: valores finitos, límites articulares, existencia de soluciones cinemáticas, continuidad entre configuraciones y cumplimiento de los límites establecidos para cada fase.

C.7. Ejecución física y control del efector

Una vez superadas las comprobaciones anteriores, el módulo de ejecución recorre las fases de aproximación, agarre, elevación, transporte, deposición y retirada. Las trayectorias articulares validadas se envían al UR5e y el efector OnRobot 3FG15 se controla de forma externa a la acción nativa del VLA.

La anchura de la pinza se envía mediante la interfaz ROS 2 utilizada en la célula:

`/onrobot/finger_width_controller/commands`.

La apertura y el cierre de la pinza forman, por tanto, parte de la capa determinista de ejecución. Del mismo modo, la orientación del efector permanece definida por la estrategia de planificación y no por las componentes rotacionales de las acciones VLA.

C.8. Soporte para replays

La implementación permite reutilizar escenas almacenadas sin solicitar movimiento del robot. Esta modalidad conserva la observación original y permite repetir la inferencia o modificar la formulación lingüística manteniendo fija la evidencia visual.

Esta separación se utilizó en los ensayos de robustez lingüística y en las ablaciones descritas en el Apéndice B. Los resultados generados mediante *replay* se trataron de forma independiente de los episodios físicos y no se utilizaron como ejecuciones reales adicionales.

Bibliografía

[1] AIMEN Centro Tecnológico. «Automatización y Robótica.» Información institucional sobre las líneas de I+D+i en automatización y robótica. (2026), dirección: https://www.aimen.es/idi/automatizacion-y-robotica (visitado 13-08-2026).

[2] M. J. Kim, K. Pertsch, S. Karamcheti et al., «OpenVLA: An Open-Source Vision-Language-Action Model,» en *Proceedings of the 8th Conference on Robot Learning*, ép. Proceedings of Machine Learning Research, vol. 270, PMLR, 2025, págs. 2679-2713. dirección: https://proceedings.mlr.press/v270/kim25c.html.

[3] K. Black, N. Brown, D. Driess et al., «π_0: A Vision-Language-Action Flow Model for General Robot Control,» en *Proceedings of Robotics: Science and Systems XXI*, Los Angeles, CA, USA, jun. de 2025. DOI: 10.15607/RSS.2025.XXI.010. dirección: https://www.roboticsproceedings.org/rss21/p010.html.

[4] Physical Intelligence, K. Black, N. Brown et al., «$\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization,» *arXiv preprint arXiv:2504.16054*, 2025. DOI: 10.48550/arXiv.2504.16054. arXiv: 2504.16054 [cs.RO]. dirección: https://arxiv.org/abs/2504.16054.

[5] Physical Intelligence. «OpenPI Training and Inference Configurations.» Commit `c23745b5ad24`; configuraciones `pi0_droid` y `pi05_droid`. (2026), dirección: https://github.com/Physical-Intelligence/openpi/blob/c23745b5ad24e98f66967ea795a07b2588ed6c79/src/openpi/training/config.py (visitado 03-08-2026).

[6] Physical Intelligence. «OpenPI Normalization Statistics and Action Space Definitions.» Commit `c23745b5ad24`. (2026), dirección: https://github.com/Physical-Intelligence/openpi/blob/c23745b5ad24e98f66967ea795a07b2588ed6c79/docs/norm_stats.md (visitado 03-08-2026).

[7] X. Zhai, B. Mustafa, A. Kolesnikov y L. Beyer, «Sigmoid Loss for Language Image Pre-Training,» en *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, págs. 11 975-11 986. dirección: https://openaccess.thecvf.com/content/ICCV2023/html/Zhai_Sigmoid_Loss_for_Language_Image_Pre-Training_ICCV_2023_paper.html.

[8] B. Zitkovich et al., «RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,» en *Proceedings of the 7th Conference on Robot Learning*, ép. Proceedings of Machine Learning Research, vol. 229, PMLR, 2023, págs. 2165-2183. dirección: https://proceedings.mlr.press/v229/zitkovich23a.html.

[9] A. Brohan et al., «RT-1: Robotics Transformer for Real-World Control at Scale,» en *Robotics: Science and Systems XIX*, 2023. dirección: https://www.roboticsproceedings.org/rss19/p025.html.

[10] Open X-Embodiment Collaboration, A. O'Neill, A. Rehman, A. Gupta et al., «Open X-Embodiment: Robotic Learning Datasets and RT-X Models,» en *2024 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2024. DOI: 10.1109/ICRA57147.2024.10611477. dirección: https://ieeexplore.ieee.org/document/10611477.

[11] Octo Model Team, D. Ghosh, H. Walke et al., «Octo: An Open-Source Generalist Robot Policy,» en *Robotics: Science and Systems XX*, 2024. dirección: https://www.roboticsproceedings.org/rss20/p090.html.

[12] A. Khazatsky, K. Pertsch, S. Nair et al., «DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset,» en *Proceedings of Robotics: Science and Systems XX*, Delft, Netherlands, jul. de 2024. DOI: 10.15607/RSS.2024.XX.120. dirección: https://www.roboticsproceedings.org/rss20/p120.html.

[13] S. Ramos, S. Girgin, L. Hussenot et al., «RLDS: An Ecosystem to Generate, Share and Use Datasets in Reinforcement Learning,» *arXiv preprint arXiv:2111.02767*, 2021. arXiv: 2111.02767. dirección: https://arxiv.org/abs/2111.02767.

[14] M. J. Kim et al. «OpenVLA: Official Implementation and Deployment Documentation.» Repositorio oficial. (2025), dirección: https://github.com/openvla/openvla (visitado 05-08-2026).

[15] M. Oquab, T. Darcet, T. Moutakanni et al., «DINOv2: Learning Robust Visual Features without Supervision,» *Transactions on Machine Learning Research*, 2024. dirección: https://arxiv.org/abs/2304.07193.

[16] L. Beyer, A. Steiner, A. S. Pinto et al., «PaliGemma: A Versatile 3B VLM for Transfer,» *arXiv preprint arXiv:2407.07726*, 2024. DOI: 10.48550/arXiv.2407.07726. arXiv: 2407.07726 [cs.CV]. dirección: https://arxiv.org/abs/2407.07726.

[17] Physical Intelligence. «OpenPI: Open-Source Models and Packages for Robotics.» Versión utilizada: commit c23745b5ad24. (2026), dirección: https://github.com/Physical-Intelligence/openpi/tree/c23745b5ad24e98f66967ea795a07b2588ed6c79 (visitado 05-08-2026).

[18] Physical Intelligence. «OpenPI: Remote Inference.» Commit c23745b5ad24. (2026), dirección: https://github.com/Physical-Intelligence/openpi/blob/c23745b5ad24e98f66967ea795a07b2588ed6c79/docs/remote_inference.md (visitado 05-08-2026).

[19] T. Z. Zhao, V. Kumar, S. Levine y C. Finn, «Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,» en *Proceedings of Robotics: Science and Systems XIX*, Daegu, Republic of Korea, jul. de 2023. DOI: 10.15607/RSS.2023.XIX.016.

[20] T. Wang, D. Bhatt, X. Wang y N. Atanasov, «Cross-Embodiment Robot Manipulation Skill Transfer using Latent Space Alignment,» *arXiv preprint arXiv:2406.01968*, 2024. DOI: 10.48550/arXiv.2406.01968. arXiv: 2406.01968 [cs.RO].

[21] M. J. Kim, C. Finn y P. Liang, «Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success,» *arXiv preprint arXiv:2502.19645*, 2025. DOI: 10.48550/arXiv.2502.19645. arXiv: 2502.19645 [cs.RO]. dirección: https://arxiv.org/abs/2502.19645.

[22] M. Hochedel y M. Lalonde, «Vision-Language-Action Models: Experimental Insights from a Real-World UR5 Platform,» *arXiv preprint arXiv:2606.30456*, 2026. DOI: 10.48550/arXiv.2606.30456. arXiv: 2606.30456 [cs.RO]. dirección: https://arxiv.org/abs/2606.30456.

[23] S. Zhang, Z. Xu, P. Liu et al., «VLABench: A Large-Scale Benchmark for Language-Conditioned Robotics Manipulation with Long-Horizon Reasoning Tasks,» *arXiv preprint arXiv:2412.18194*, 2024. DOI: 10.48550/arXiv.2412.18194. arXiv: 2412.18194 [cs.RO]. dirección: https://arxiv.org/abs/2412.18194.

[24] C. Kim, M. Kim, M. Kang, H. Kim y D. Jung, «LIBERO-Para: A Diagnostic Benchmark and Metrics for Paraphrase Robustness in VLA Models,» *arXiv preprint arXiv:2603.28301*, 2026. DOI: 10.48550/arXiv.2603.28301. arXiv: 2603.28301 [cs.RO]. dirección: https://arxiv.org/abs/2603.28301.

[25] N. Zhang, B. Zhu, S. Zhou y J. Chen, «Restoring Linguistic Grounding in VLA Models via Train-Free Attention Recalibration,» *arXiv preprint arXiv:2603.06001*, 2026. DOI: 10.48550/arXiv.2603.06001. arXiv: 2603.06001 [cs.RO]. dirección: https://arxiv.org/abs/2603.06001.

[26] Z. Zhou, P. Atreya, Y. L. Tan, K. Pertsch y S. Levine, «AutoEval: Autonomous Evaluation of Generalist Robot Manipulation Policies in the Real World,» en *Proceedings of the 9th Conference on Robot Learning*, ép. Proceedings of Machine Learning Research, vol. 305, PMLR, 2025, págs. 1997-2017. dirección: https://proceedings.mlr.press/v305/zhou25a.html.

[27] Q. Li, B. Yin, W. Huang et al., «Vision-Language-Action Safety: Threats, Challenges, Evaluations, and Mechanisms,» *arXiv preprint arXiv:2604.23775*, 2026. DOI: 10.48550/arXiv.2604.23775. arXiv: 2604.23775 [cs.RO]. dirección: https://arxiv.org/abs/2604.23775.

[28] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016. dirección: https://www.deeplearningbook.org/.

[29] A. Vaswani, N. Shazeer, N. Parmar et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems*, vol. 30, 2017. dirección: https://arxiv.org/abs/1706.03762.

[30] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,» en *International Conference on Learning Representations*, 2021. dirección: https://arxiv.org/abs/2010.11929.

[31] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel y M. Le, «Flow Matching for Generative Modeling,» en *International Conference on Learning Representations*, 2023. dirección: https://arxiv.org/abs/2210.02747.

[32] J. Thomason, M. Shridhar, Y. Bisk, C. Paxton y L. Zettlemoyer, «Language Grounding with 3D Objects,» en *Proceedings of the 5th Conference on Robot Learning*, ép. Proceedings of Machine Learning Research, vol. 164, PMLR, 2022, págs. 1691-1701. dirección: https://proceedings.mlr.press/v164/thomason22a.html.

[33] G. Tziafas, Y. Xu, A. Goel, M. Kasaei, Z. Li y H. Kasaei, «Language-Guided Robot Grasping: CLIP-Based Referring Grasp Synthesis in Clutter,» en *Proceedings of the 7th Conference on Robot Learning*, ép. Proceedings of Machine Learning Research, vol. 229, PMLR, 2023, págs. 1450-1466. dirección: https://proceedings.mlr.press/v229/tziafas23a.html.

[34] B. Siciliano, L. Sciavicco, L. Villani y G. Oriolo, *Robotics: Modelling, Planning and Control*. London: Springer, 2009. DOI: 10.1007/978-1-84628-642-1.

[35] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2.ª ed. Springer, 2017, ISBN: 978-3-319-54412-0. DOI: 10.1007/978-3-319-54413-7.

[36] K. S. Arun, T. S. Huang y S. D. Blostein, «Least-Squares Fitting of Two 3-D Point Sets,» *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-9, n.º 5, págs. 698-700, 1987. DOI: 10.1109/TPAMI.1987.4767965.

[37] D. Kortenkamp, R. Simmons y D. Brugali, «Robotic Systems Architectures and Programming,» en *Springer Handbook of Robotics*, B. Siciliano y O. Khatib, eds., 2.ª ed., Cham: Springer, 2016, págs. 283-306. DOI: 10.1007/978-3-319-32552-1_12.

[38] T. Johannink, S. Bahl, A. Nair et al., «Residual Reinforcement Learning for Robot Control,» en *2019 International Conference on Robotics and Automation (ICRA)*, IEEE, 2019, págs. 6023-6029. DOI: 10.1109/ICRA.2019.8794127. dirección: https://doi.org/10.1109/ICRA.2019.8794127.

[39] Universal Robots, *Especificaciones técnicas del UR5e*, Universal Robots, 2026. dirección: https://www.universal-robots.com/manuals/es/HTML/SW5_25/Content/prod-usr-man/complianceUR5e/H_g5_sections/appendix_g5/tech_spec_sheet.htm (visitado 05-08-2026).

[40] OnRobot, *3FG15: pinza de tres dedos*, OnRobot, 2026. dirección: https://onrobot.com/es/productos/3fg15-pinza-de-tres-dedos (visitado 05-08-2026).

[41] Photoneo, *PhoXi Control User Manual*, Zebra Technologies Slovakia, 2026. dirección: https://docs.photoneo.com/docs/PXC/PXC_manual.html (visitado 05-08-2026).

[42] NVIDIA, *Jetson AGX Orin Developer Kit User Guide*, NVIDIA Corporation, 2025. dirección: https://docs.nvidia.com/jetson/agx-orin-devkit/user-guide/latest/introduction.html (visitado 05-08-2026).

[43] Open Robotics, *ROS 2 Documentation: Topics, Services and Actions*, Open Robotics, 2026. dirección: https://docs.ros.org/en/humble/How-To-Guides/Topics-Services-Actions.html (visitado 05-08-2026).

[44] Universal Robots, *Universal Robots ROS 2 Driver Documentation*, Universal Robots, 2026. dirección: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/doc/ur_robot_driver/ur_robot_driver/doc/usage/toc.html (visitado 05-08-2026).

[45] OnRobot. «Configuración predeterminada de la Compute Box.» Documentación oficial de OnRobot. (2021), dirección: https://learn.onrobot.com/es/configuracion-predeterminada-de-compute-box?area=41 (visitado 06-08-2026).

[46] C. R. Harris, K. J. Millman, S. J. van der Walt et al., «Array Programming with NumPy,» *Nature*, vol. 585, n.º 7825, págs. 357-362, 2020. DOI: 10.1038/s41586-020-2649-2.

[47] G. Bradski, «The OpenCV Library,» *Dr. Dobb's Journal of Software Tools*, 2000.

[48] E. Todorov, T. Erez e Y. Tassa, «MuJoCo: A Physics Engine for Model-Based Control,» en *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2012, págs. 5026-5033. DOI: 10.1109/IROS.2012.6386109.

[49] Google DeepMind. «MuJoCo API Reference: Kinematic and Jacobian Functions.» Documentación de las funciones cinemáticas y de cálculo de jacobianos, incluida `mj_jacSite`. (2026), dirección: https://mujoco.readthedocs.io/en/3.3.6/APIreference/APIfunctions.html (visitado 09-08-2026).

[50] A. Paszke, S. Gross, F. Massa et al., «PyTorch: An Imperative Style, High-Performance Deep Learning Library,» en *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[51] T. Wolf, L. Debut, V. Sanh et al., «Transformers: State-of-the-Art Natural Language Processing,» en *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, págs. 38-45. DOI: 10.18653/v1/2020.emnlp-demos.6.

[52] J. Bradbury, R. Frostig, P. Hawkins et al. «JAX: Composable Transformations of Python and NumPy Programs.» (2018), dirección: https://github.com/jax-ml/jax (visitado 05-08-2026).

[53] J. Heek, A. Levskaya, A. Oliver et al. «Flax: A Neural Network Library and Ecosystem for JAX.» (2020), dirección: https://github.com/google/flax (visitado 05-08-2026).

[54] Universal Robots. «Teaching TCP Position.» Documentación oficial de PolyScope 5 para la determinación automática de la posición del TCP. (2026), dirección: https://www.universal-robots.com/manuals/EN/HTML/SW5_22/Content/prod-usr-man/software/PolyScope/content/installation_g5/teaching_TCP_position.htm (visitado 09-08-2026).

[55] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas y M. J. Marín-Jiménez, «Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,» *Pattern Recognition*, vol. 47, n.º 6, págs. 2280-2292, 2014. DOI: 10.1016/j.patcog.2014.01.005.

[56] CloudCompare, *Align: Point Pairs Picking*, CloudCompare, 2026. dirección: https://www.cloudcompare.org/doc/wiki/index.php/Align (visitado 05-08-2026).

[57] E. B. Wilson, «Probable Inference, the Law of Succession, and Statistical Inference,» *Journal of the American Statistical Association*, vol. 22, n.º 158, págs. 209-212, 1927. DOI: 10.1080/01621459.1927.10502953.

[58] A. Agresti, *An Introduction to Categorical Data Analysis*, 3.ª ed. Wiley, 2018, ISBN: 978-1-119-40526-9.

[59] P. Valle, C. Lu, S. Ali y A. Arrieta, «Evaluating Uncertainty and Quality of Visual Language Action-Enabled Robots,» *arXiv preprint arXiv:2507.17049*, 2025. DOI: 10.48550/arXiv.2507.17049. arXiv: 2507.17049 [cs.RO]. dirección: https://arxiv.org/abs/2507.17049.

[60] P. Valle. «VLA_UQ: implementación de las métricas de incertidumbre sobre acciones.» Implementación pública consultada en el commit `75bff92d9ad4`; A-VI divide la segunda diferencia por 2 y A-AI divide la tercera diferencia por 4. (2025), dirección: https://github.com/pablovalle/VLA_UQ/blob/75bff92d9ad4b3f8c033108bfb1e9dd19af686bd/uncertainty/uncertainty_metrics.py (visitado 06-08-2026).